



Notes Source

Metigator Linq Course + Microsoft Documents



Created By
Abdelrhman Elsaid

LINQ Learning Path

00-**Introduction** (Pure vs Impure Functions)

01-**Delegates**, Lambdas, Functional Refactoring

02-**LINQ Introduction**

03-**Projection Select** (String, Math, DTO)

04-**Projection** (SelectMany & Zip)

05-**Sorting** (OrderBy & IComparer)

06-**Data Partitioning** (Skip, Take, Chunk, Pagination)

07-**Quantifiers** (Any, All, Contains)

08-**Grouping Data** (GroupBy, ToLookup)

09-**Join Operations** (Join, Group Join)

10-**Generation Operations** (Empty, DefaultIfEmpty, Range, Repeat)

11-**Element Operations** (ElementAt, First & Last, Single)

12-**Equality Operations** (SequenceEqual)

13-**Concatenation**

14-**Aggregate Operations**

15-**Set Operations** (Except, Intersect, Union)

16-**Expression Trees**

17-**IQueryable vs IEnumerable**

18-**Converting Data Types** (AsEnumerable, AsQueryable, Cast, OfType)

19-**Converting Data Types** (ToArray, ToList, ToDictionary, ToLookup)

20-**Custom LINQ Extension Methods**

21-**LINQ Anatomy**

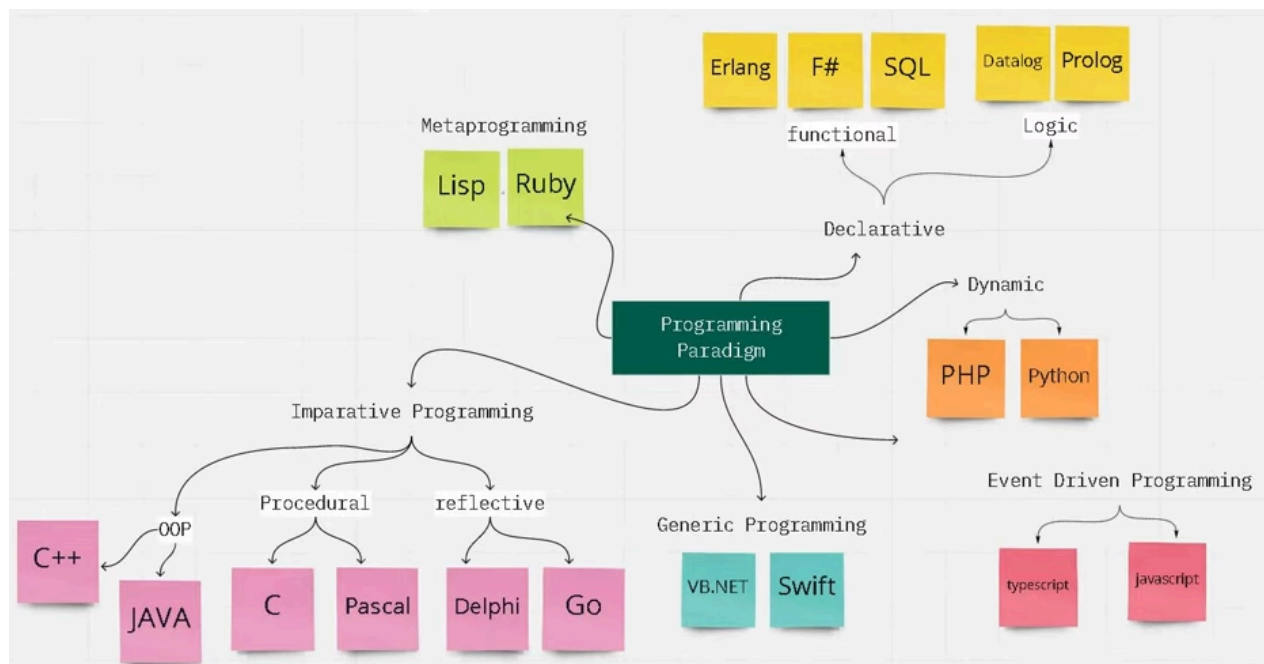
22-**Material For Practice**

Introduction

مقدمة عامة

السلام عليكم. النهاردة بنبدأ أول حلقة في سلسلة جديدة ومهمة جداً وهي الـ **لينك (LINQ)**.

- **المشكلة:** ناس كتير بتستخدم LINQ، بس للأسف مش بتستخدمها بالطريقة المثلى.
- **السبب:** فيه مفاهيم وأساسيات (Fundamentals) غايبة عنهم.
- **الهدف من السلسلة:** إننا نتقن الأساسيات دي قبل ما ندخل في الكود، عشان لما نستخدم LINQ تكون إنتاجيتنا (Productivity) عالية وأداء الكود (Performance) بكفاءة عالية.
- **بيئة العمل:** عملنا Solution فاضي، وكل مفهوم هنشرحه هنعمله في Project منفصل، وفي الآخر الكود كله هيترفع على GitHub عشان تطبقوا عليه.



1. نماذج البرمجة (Programming Paradigms)

قبل ما نكتب كود، لازم نتكلم عن أنواع أو "نماذج" البرمجة الموجودة، لأن كل لغة بتتبع نموذج معين. الأنواع الرئيسية هي:

أ) Declarative Programming

- النموذج ده بينقسم لشقين:

1. الـ **Functional**: زي لغات F، Erlang، #، و SQL. ده بيعتمد على الـ Expressions اللي بتوصف العمليات (Describe Operations).

2. الـ **Logic**: زي لغات Prolog و Datalog. ده بيعتمد على الحقائق والقواعد (Facts & Rules).

ب) Dynamic Programming

- النموذج ده بينفذ سلوك وقت الترجمة في وقت التشغيل (Executes Compile Time Behavior at Runtime).
- يعني بيعمل حساباته والـ Evaluation والبرنامج شغال.
- أمثلة: Python و PHP .

ج) Event Driven Programming

- هنا الـ Event هو اللي بيحرك العمليات (Event drives the operations).
- أمثلة: JavaScript و TypeScript .

د) Generic Programming Support

- ده اللي بنعمل فيه Data Structures و Operations بتدعم أنواع مختلفة (Generics).
- أمثلة: VB.NET و SWIFT .

هـ) Imperative Programming

- ده بيستخدم أوامر (Commands) وجمل (Statements) عشان يقول للبرنامج "إزاي يشتغل" (How to operate).
بينقسم لـ شقين:
- 1. الـ **Object Oriented**: زي ++C و Java . البيانات بتبقى Fields والسلوك بيبقى Methods . (قائمة على Objects).
- 2. الـ **Procedural**: زي C و Pascal . بيبقى عندي Procedures و Sub-procedures .

و) Reflective Programming

- أمثلة: Delphi و Go .

ز) Metaprogramming

- هنا بنستخدم كود البرنامج نفسه كأنه بيانات (Code as Data).
- أمثلة: Lisp و Ruby .

2. لغة #C وتعدد النماذج (C# as a Multi-Paradigm Language)

حسب تعريف ويكيبيديا ومايكروسوفت، لغة #C تعتبر **Multi-Paradigm General Purpose Language**، يعني فيها كل الحاجات اللي قولناها دي:

- الـ **Declarative**: بتستخدم الـ Attributes .
- الـ **Dynamic**: فيها الـ dynamic keyword .
- الـ **Functional**: بتدعم First Class Functions باستخدام Delegates و Lambda Expressions .
- الـ **Event Driven**: فيها Events .
- الـ **Generic**: بتعمل Support للـ Generics .
- الـ **Procedural**: بتدعم الـ Static Methods .

- الـ **Imperative**: فيها Statements زي (if , else , while , for , goto , switch).
- الـ **Object Oriented**: الـ Object هو المواطن الأول (First Citizen) واللغة Class-Based.
- الـ **Metaprogramming**: فيها Expression Trees و Reflection .

3. الفرق بين Statement و Expression

عشان نفهم LINQ صح، لازم نفرق بين المفهومين دول بدقة حسب تعريف مايكروسوفت.

أولاً: الـ Statement

- مايكروسوفت بتقولك: "أي سطر كود بيتتهي بـ Semicolon (;) هو Statement".
- أمثلة:
- الـ Declaration : تعتبر Statement.
- الـ Assignment : تعتبر Statement.
- الـ Initialization (تعريف وإسناد Value في نفس الوقت): تعتبر Statement.
- البلوكات (Blocks):
- لو الجملة بتتكون من أكثر من سطر، بنعملهم Group بداخل Block { ... } .
- زي if , while , switch , foreach . الـ Body بتاعهم ممكن يحتوي على أكثر من Statement.

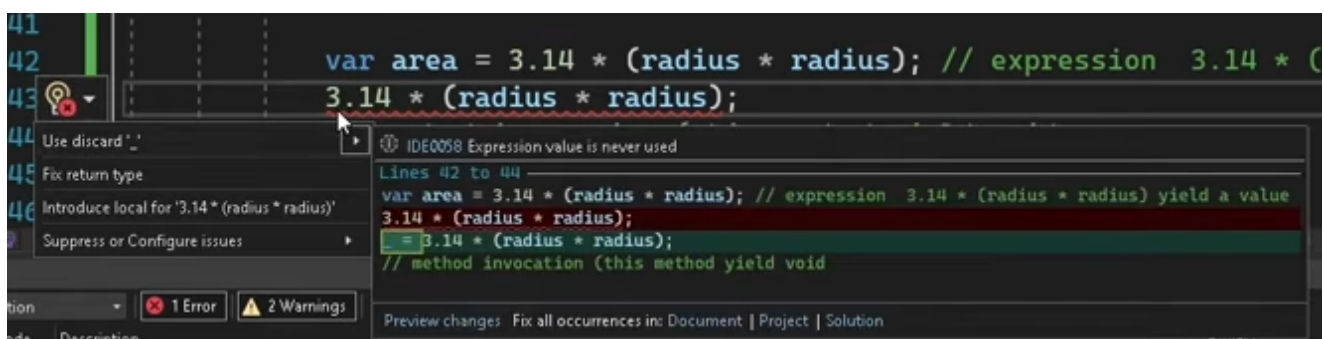
ثانياً: الـ Expression

- تعريفه: Expression yields a value (هو تعبير بيرجع قيمة).
- أمثلة:

1. الـ Mathematical Expression

```
1 // الجملة دي على بعضها statement
2 // لأنه بيطلع قيمة expression لكن الجزء اليمين (5 * 10) ده
3 var area = 10 * 5;
```

ملحوظة: لو حاولت تحط 5 * 10 في سطر لوحدها هيديك Error بيقول: Expression value is never used . لازم تستخدم القيمة دي (تحطها في متغير أو تبعثها لميثود)



2. الـ Method Invocation

- مناداة الميثود تعتبر Expression.
- سؤال: طيب لو الميثود بترجع void ؟
- الإجابة: الـ void تعتبر قيمة بس "غير موجودة" أو "غير مُعرفة". لما الميثود بتكون void الكومبايلر مش مستني منك ترجع قيمة فعلية، لكن عملية الاستدعاء نفسها technically expression.

```
1 Console.WriteLine("Hi Linq");
```

3. الـ Object Creation:

- استخدام الـ new operator عشان تعمل Object جديد يعتبر Expression.

4. الدوال النقية وغير النقية (Pure vs. Impure Functions)

ده أهم كونسبيبت هيتبني عليه شغلنا في LINQ. عشان نفهمه عملنا مشروع اسمه PureVersusImpureFunctions .

أ) الدوال غير النقية (Impure Functions)

الدالة بتكون Impure لو عملت أي حاجة من دول:

1. التعديل على متغير عام (Global Variable Mutation)

لو الدالة عدلت في متغير موجود بره نطاقها (Global).

C#

```
1
2 internal class Program
3 {
4     // Global Variable
5     // متعرف بره الـ Main
6     static List<int> list = new List<int>() { 1, 2, 3, 4, 5 };
7     static public void AddNumber(int number)
8     {
9         list.Add(number);
10    }
11
12    static void Main(string[] args)
13    {
14
15        AddNumber(6);
16
17        foreach(int number in list)
18        {
19            Console.WriteLine(number);
20        }
```

```

21
22     }
23
24     }

```

- **المشكلة:** يحصل **Side Effect** (أثر جانبي). لو المتغير ده مستخدم في مكان تاني، الكود هيبوظ هناك. وده بيخلي الـ Testing صعب جداً.
- **القاعدة:** الـ Pure Function بتعدل بس على الـ Local Variables، ممنوع تلمس الـ Global.

2. التعديل على الباراميتير (Reference Mutation)

لو بعت باراميتير للدالة وعدلت عليه (سواء كان Object أو باستخدام ref).

C#

```

1 public void AddInteger(ref int x)
2 {
3     x = x + 1; // بتاع المتغير اللي مبعوت Reference هنا عدلنا على الـ
4 }

```

- **المشكلة:** المتغير الأصلي اللي عند الـ Caller (لما مررته من الـ Main يعني) قيمته اتغيرت. لو هو كان معتمد إن القيمة تفضل زي ما هي، الكود بتاعه هيضرب. دي برضه **Impure**.

3. التفاعل مع العالم الخارجي (Interaction with Outside World)

لو الدالة بتكلم حاجة بره سيطرتها، زي Database ، Console ، DateTime ، Random ، أو API .

C#

```

1 public int GetRandomNumber()
2 {
3     Random rnd = new Random();
4     return rnd.Next(); // النتيجة مش متوقعة ومش ثابتة
5 }
6
7 // تعمل التالي Main وفي الـ
8 AddInteger(GetRandomNumber());
9

```

- **السبب:** الـ Pure Function لازم تكون **Deterministic**، يعني "2 = 1 + 1" دايماً. لكن الـ Random كل مرة بيديك قيمة مختلفة، فمبقدش أتوقع النتيجة (Cannot expect the result).
- لذلك، أي دالة بتستخدم IO Stream أو HTTP Call تعتبر **Impure**.

(ب) الدوال النقية (Pure Functions)

عشان نحول الدالة لـ Pure، لازم نطبق الشروط دي:

1. نفس المدخلات تعطي نفس المخرجات دائماً (Always same input -> same output).
2. مفيش آثار جانبية (No Side Effects).
3. مفيش تعامل مع متغيرات خارجية (No interaction with outer world).

مثال عملي: تحويل دالة Pure لـ Impure

عايزين نضيف رقم لقائمة، بس بطريقة Pure:

C#

```
1 // Pure Function
2 public List<int> AddIntegerPure(List<int> oldList, int numberToAdd)
3 {
4     // 1. عملنا ليست جديدة (Local Variable)
5     var newList = new List<int>();
6
7     // 2. نسخنا القيم القديمة للسنة الجديدة (بدون ما نعدل القديمة)
8     newList.AddRange(oldList);
9
10    // 3. ضفنا الرقم الجديد على الليست الجديدة
11    newList.Add(numberToAdd);
12
13    // 4. رجعنا الليست الجديدة
14    return newList;
15 }
```

• الشرح:

- أنا ما عدلتش على oldList خالص.
- أنا عملت newList وحطيت فيها الداتا وضفت عليها.
- كده oldList فضلت زي ما هي، والـ newList فيها التعديل.
- لو طبعنا oldList بعد العملية هنلاقيها ما تغيرتش.

مميزات الـ Pure Functions:

- الـ Easier to Understand: سهولة الفهم.
- الـ Easier to Test: سهولة الاختبار لأن مفيش مفاجآت.
- الـ Thread Safe: (ضمنياً لأنها مش بتشارك State).

أمثلة جاهزة في #C:

- كلاس Math كله Pure Functions.
- الـ Math.Max(10, 20) : دائماً هترجع 20.
- الـ Math.Cos(x) : دائماً هترجع نفس القيمة لنفس الـ x.

ملخص سريع (Summary)

الـ **Pure Function**: يتأخذ Input، يتعمل Process، وترجع Output جديد، من غير ما تلمس أي حاجة حواليتها.
الـ **Impure Function**: بتغير في الـ State بتاع البرنامج (Global vars)، أو بتعتمد على حاجات بتتغير (Random/Time).

Delegates Lambdas Functional Refactoring

1. مقدمة واسترجاع السريع

- الهدف: في الحلقة اللي فاتت عرفنا إن الـ **Functional Programming** بتعتمد بشكل أساسي على الـ **Functions**، بس مش أي **function**، لازم تكون **Pure Function**.
- خصائص الـ **Pure Function**:
 - مفيش **External Dependency** (ما تعتمدش على حاجة من بره).
 - مبتغيرش الـ **State** ولا الـ **Argument**.
 - مفيش **Imperative Code** (زي الـ **loops** المعقدة)، لأن الـ **Functional Programming** بتحب الـ **Declarative Code**.
- طرق تمثيل الـ **Function** في **#C**:
 - الـ **Methods**.
 - الـ **Delegates**.
 - الـ **Lambda Expressions**.

2. مفهوم الـ **Delegate** (المشكلة والحل)

المشكلة: تمرير الـ **Function** كـ **Parameter**

في لغات زي **JavaScript**، تقدر تبعث الـ **Function** كـ **parameter** عادي جداً لأن الـ **Function** هناك بيتعامل كـ **Object**. لكن في **#C**، الـ **Function** هو **Behavior** (سلوك) فمينفعش تبعته كـ **Argument** مباشرة.

الحل: الـ **Delegate**

الـ **Delegate** هو عبارة عن **Type** بيتمثل مرجع (**Reference**) لـ **Method** معينة.

- عشان تبعث **m1** method لـ **method** ثانية **m2**، لازم **m2** تستقبل **Delegate** بيتمثل **m1**.

مثال توضيحي (**Custom Delegate**):

لو عندنا ميثود بترجع **void** وما بتاخدش **parameters**:

C#

```
1 // تعريف الـ Delegate
2 delegate void mDelegate();
3
4 // الـ Delegate الميثود اللي هتستقبل الـ
5 static void m2(mDelegate del) {
6     del(); // تنفيذ الميثود المبعوثة
7 }
```

3. الـ Generic Delegates (حل مايكروسوفت)

بدل ما كل شوية تعرف Delegate خاص بيك (Custom Delegate)، مايكروسوفت في الـ .NET عملت لك 3 أنواع Generic تغطي كل احتياجاتك.

أنواع الـ Generic Delegates:

1. الـ **Action**: ميثود لا ترجع قيمة (void) ويمكن تاخذ parameters.

2. الـ **Func**: ميثود ترجع قيمة ويمكن تاخذ parameters.

3. الـ **Predicate**: ميثود بتاخذ parameter ودائماً ترجع **Boolean** (بتستخدم للـ check أو الشرط).

ملحوظة: المهندس يفضل استخدام `<Func<T, bool>` بدلاً من `<Predicate<T>` لأنه ببشوف كلمة `Func` معبرة أكثر (More Descriptive).

4. التطبيق العملي: الانتقال من Procedural لـ Functional

المهندس عمل مشروع عشان يوضح الفرق بين الطريقتين.

هيكل البيانات (Setup)

- كلاس `Employee` فيه بيانات (.ID, FirstName, Salary, Department, etc).
- كلاس `Repository` فيه ميثود `LoadEmployees` بترجع `List` عشوائية من 100 موظف للتجربة.

الطريقة القديمة (Procedural Way)

دي الطريقة اللي بنكتب بيها الكود العادي واللي فيها مشاكل كتير. السيناريو: عايزين نعمل 10 استعلامات (Queries) مختلفة على الموظفين (مثلاً: اللي اسمهم يبدأ بـ 'M'، اللي في قسم HR، اللي راتبهم < 1000، إلخ).

شكل الكود السيء:

C#

```
1 // مثال لميثود بتجيب الموظفين اللي اسمهم ببدا بحروف معينة
2 public static IEnumerable<Employee> GetEmployeesByFirstName(string value)
3 {
4     var employees = Repository.LoadEmployees(); // 1. External Dependency
5     foreach (var e in employees) // 2. Imperative Code (Loop)
6     {
7         if (e.FirstName.ToLower().StartsWith(value.ToLower()))
8         {
9             yield return e;
10        }
11    }
```

- عيوب الطريقة دي:

1. الـ **Not Pure**: يعتمد على **Repository (External Dependency)**.
2. فيها نوع من الـ **Imperative**: فيها **foreach** و **if** كتير.
3. التكرار: عشان تعمل 10 فلاتر، هنضطر نكتب 10 **methods** شبه بعض بالظبط مع اختلاف الشرط بس (مرة **StartsWith** ، مرة **Salary > x** ، إلخ).
4. صعوبة الـ **Testing**: صعب تعمل **Unit Test** للكود ده بسبب اعتماده على الداتا بيز أو الـ **Repository** مباشرة.

الطريقة الحديثة (Functional Way)

هنا هنحول الكود لـ **Pure Function** ونتخلص من التكرار باستخدام مبدأ **Higher Order Function** (إن الـ **function** تستقبل **function** ثانية كـ **parameter**).

خطوة 1: التخلص من الـ Dependency

بدل ما الميثود تروح تحيب الداتا بنفسها، إحنا هنبعث الداتا ليها كـ **Parameter (source)**.

خطوة 2: التخلص من تكرار اللوجيك (Filter Logic)

بدل ما نكتب ميثود لكل شرط، إحنا هنخلي الميثود تستقبل الشرط نفسه كـ **Parameter**. الشرط ده هيكوّن عبارة عن **Delegate** بياخد **Employee** ويرجع **bool**.

الكود بعد التحسين **(Functional Style)**:

C#

```

1 public static IEnumerable<Employee> Filter(IEnumerable<Employee> source,
2 Func<Employee, bool> predicate)
3 {
4     foreach (var employee in source)
5     {
6         // هنا بننفذ الشرط اللي المبرمج بعتّه
7         if (predicate(employee))
8         {
9             yield return employee;
10        }
11    }
12 }
```

طريقة الاستدعاء (Calling the Function)

دلوقتي نقدر نستخدم ميثود واحدة بس **Filter** عشان ننفذ الـ 10 استعلامات، عن طريق تمرير الـ **Logic** باستخدام **Lambda Expression**.


```

1 // 1. كل الموظفين اللي اسمهم يبدأ بـ 'ma'
2 var q1 = Filter(employees, e => e.FirstName.ToLower().StartsWith("ma"));
3
4 // 2. كل الموظفين في قسم الـ HR
5 var q2 = Filter(employees, e => e.Department == "HR");
6
7 // 3. وراتبهم أقل من قيمة معينة (Female) موظفات
8 var q3 = Filter(employees, e => e.Gender == "Female" && e.Salary < 5000);

```

المميزات هنا:

- الـ **Flexibility**: مرونة عالية، لو عايز شرط جديد مش محتاج تعدل في ميثود الـ `Filter` ، بس بتغير الـ `Lambda Expression` وأنت بتناديها.
- الـ **Less Code**: استبدلنا 10 ميثودز بميثود واحدة `generic`.
- الـ **Separation of Concerns**: فصلنا اللوجيك (الشرط) عن عملية التكرار والفلتر.

5. التحسين الأخير: Extension Method 🚀

لحد هنا استخدمنا `Functional Programming`، بس لسه شكل الاستدعاء مش أحسن حاجة (`Filter(list, condition)`). عشان نخلي الكود مقروء أكثر ونمهد لـ `LINQ`، هنستخدم **Extension Methods**.

الفكرة:

هنحط كلمة `this` قبل أول `parameter` في ميثود الـ `Filter` ، وده هيجعلنا نقدر ننادي الميثود كأنها جزء من الـ `List` نفسها.

تعديل الكود:

```

1 public static class ExtensionFunctional
2 {
3     // ضفنا كلمة this
4     public static IEnumerable<T> Filter<T>(this IEnumerable<T> source,
5     Func<T, bool> predicate)
6     {
7         foreach (var item in source)
8         {
9             if (predicate(item))
10            {
11                yield return item;
12            }
13        }
14    }
15 }

```

```
13     }
14 }
```

شكل الاستدعاء الجديد (Fluent Syntax):

C#

```
1 // الكود بقى بيتقري زي الإنجليزي
2 var result = employees.Filter(e => e.Salary > 3000)
3 .Print(); // Print ك extension method عملنا كمان
```

ملاحظات ومصطلحات هامة جداً

1. الـ Higher Order Functions

هي أي function بتعمل حاجة من الاتنين:

1. تستقبل function كـ parameter (زي ما عملنا في Filter لما استقبلت predicate).
2. أو بترجع function كـ return value. مكتبة LINQ كلها قائمة على المبدأ ده.

2. الـ Yield Return

المهندس ذكر إنه هيشرحها بالتفصيل في فيديو لوحده، بس وضح إنها بتعمل **State Machine**.

- الفائدة: بتخلي الميثود ترجع البيانات عنصر ورا عنصر (Lazy Evaluation) بدل ما تحمل كل البيانات في الميموري وترجعهم مرة واحدة. لما الـ Caller يطلب العنصر التالي، الـ yield بتكمل شغل من مكان ما وقفت.

3. الـ Pipelining

استخدام الـ Extension Methods بيخلينا نقدر نركب العمليات ورا بعضها (Pipeline)، يعني نفلتر وبعدين نطبع، وبعدين نعمل حاجة تالته في سطر واحد متصل، وده هنشوف قوته في الحلقة الجاية.

الخاتمة



- قدرنا نحول كود Procedural طويل ومعقد لكود Functional بسيط، Pure، و Flexible.
- استخدمنا Action، Func، Predicate، بدل ما نعرف Delegates خاصة بيها.
- استخدمنا Lambda Expressions عشان نبعث اللوجيك (Behavior).
- استخدمنا Extension Methods عشان نحسن شكل الكود وقابليته للقراءة.
- الحلقة الجاية: هنتكلم عن الـ Composition والـ Pipelining بشكل أعمق.

LINQ Introduction

1. مقدمة: إيه هو الـ LINQ أصلاً؟

في البداية، لازم نعرف إحنا بنتكلم عن إيه. الـ LINQ اختصار لـ **Language Integrated Query**. ببساطة، مايكروسوفت فكرت إنها تعمل "مكون" (Component) أو Framework صغير يوحد طريقة الاستعلام (Query) عن البيانات مهما كان مصدرها.

المشكلة اللي بيحلها

زمان، لو عايز تكلم الـ Database كنت بتكتب SQL، ولو عايز تكلم XML بتكتب بطريقة ثانية، ولو بتدور في List في الميموري بتكتب Loop. الـ LINQ وحدت كل ده:

- عايز تكلم **Database**؟ بتستخدم LINQ to SQL.
- عايز تستعلم عن **Object** في الميموري؟ بتستخدم LINQ to Objects.
- عايز تطلب ملف **XML** وتحوله لـ **Object**؟ بتستخدم LINQ to XML.

أهم الميزات الأساسية

1. الـ **Unified Query Syntax**: طريقة كتابة واحدة لكل المصادر.
2. الـ **Type Safe**: يعني الكومبايلر بيحميك من أخطاء الـ **Type Conflict**. مش هيطلعك Error زي "Unable to convert type X to Y" في وقت التشغيل (Runtime)، لأ، أنت بتكتب وانت عارف نوع الـ **Object** إيه.
3. الـ **Intellisense Support**: بما إنها **Type Safe**، الـ **Visual Studio** هيساعدك وانت بتكتب الكود.

2. شروط استخدام LINQ (على مين تشتغل؟)

مش أي حاجة ينفع نطبق عليها LINQ. الشرط الأساسي عشان تقدر تستخدم LINQ على أي **Collection** أو **Local Object** في الميموري، لازم الـ **Object** ده يكون مطبق (Implemented) للـ **Interface** اللي اسمه **IEnumerable**.

- سواء بشكل مباشر أو غير مباشر (عن طريق التوارث).
- لو مش عارف يعني إيه **IEnumerable**، ارجع للحلقة الخاصة بيه (شرحناها قبل كده).

ملحوظة: الـ LINQ داخلياً هي عبارة عن مجموعة من **Query Operators** معمولة على شكل **Extension Methods**. عشان كده لازم تكون فاهم الفرق بين الـ **Pure Functions** والـ **Impure Functions** ومفاهيم الـ **Functional Programming**.

3. تطبيق عملي: من كود يدوي لـ LINQ

في المحاضرة اللي فاتت، عملنا **Function** بايديناسمها **Filter** بتأخذ **Delegate** عشان نصفي لسته الموظفين. النهاردة هنشوف إزاي الـ LINQ بتعمل نفس الشغل ده بسطر واحد.

عندنا لستة موظفين Employees (مطبقة IEnumerable)، وعازين نطلع الناس اللي اسمهم بيبدأ بحرف 'S' وهم "إناث" (Female).

بدل ما نكتب Filter ميثود خاصة بيها، بنسأل الـ LINQ: "يا لينك، عندك ميثود جاهزة للفلتر؟". الرد: "أيوه، عندي ميثود اسمها Where".

- بتاخذ Source (اللي هو اللسته).
- وبتاخذ Predicate (الشرط أو الـ Delegate).

الكود باستخدام LINQ (طريقتين للكتابة)

فيه طريقتين لكتابة جمل الـ LINQ، والاتنين بيطلعوا نفس النتيجة ومفيش واحدة أحسن من الثانية بشكل مطلق، كل واحدة ليها وقتها.

1. الـ Method Syntax (Fluent API)

دي الطريقة اللي شبه استدعاء الميثود العادية (زي ما عملنا list.Filter المرة اللي فاتت).

C#

```

1  // ده Namespace لازم تعمل import
2  using System.Linq;
3
4  static void Main(string[] args)
5  {
6
7      var dataSource = DataBaseLoad.LoadEmployees();
8
9
10     var femalsStartWithS = dataSource.Where((e) =>
e.FirstName.ToLowerInvariant().StartsWith("s") && e.Gender == "female");
11
12     foreach (var employee in femalsStartWithS)
13     {
14         Console.WriteLine(employee);
15     }
16
17 }
```

- ميزتها: مفضلة لما بتتعامل مع List في الميموري (LINQ to Objects).
- الكومبايلر في الآخر بيحول أي كود لـ Method Syntax في الـ IL (Intermediate Language).

2. الـ Query Syntax (SQL-like)

دي طريقة شبيه لغة الـ SQL بالظبط.

عشان تتقن الطريقة دي امشى بالترتيب التالى (مع معظم الحالات الى هتقابلك)

1-Define Location. ---> **from () in ()**

2-Define Conditioning or sorting etc.. ---> **where**

3-return value using Select. ---> **select ()**

C#

```
1 var femalsStartWithS = from e in employees
2                           where e.FirstName.StartsWith("S") && e.Gender ==
   "Female"
3                           select e;
```

- ميزتها: مفضلة لما بتتعامل مع Database أو لما يكون عندك استعلامات معقدة زي Join أو Select من أكثر من مكان، بنكون أسهل في القراءة.

4. الفخ الكبير: Eager vs Deferred Execution

دي أهم جزئية في المحاضرة، ولازم تفهمها عشان ما تقعش في مشاكل الـ Performance أو Bugs غريبة.

التجربة العملية (لستة الأرقام)

```
1 internal class Program
2 {
3     static void Main(string[] args)
4     {
5         List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
6
7         IEnumerable<int> evenNumbers = numbers.Where(x => x % 2 == 0);
8
9         numbers.Add(10);
10        numbers.Add(12);
11        numbers.Remove(4);
12
13        foreach (var n in evenNumbers)
14        {
15            Console.Write($"{n} ");
16        }
17
18        Console.ReadKey();
19    }
20 }
```

تخيل عندنا لسته أرقام:
عملنا كويري بـ LINQ عشان نجيب الأرقام الزوجية (Even Numbers):

ركز في اللي هيحصل دلوقتي بالترتيب:

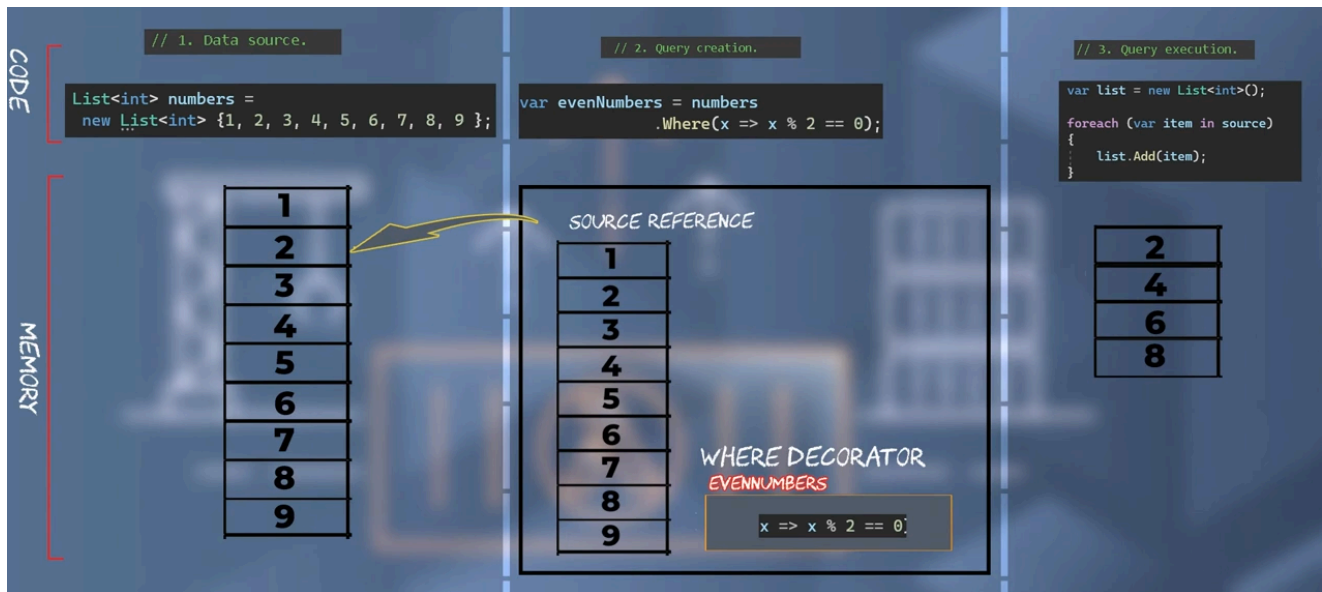
1. عرفنا الـ Query فوق (evenNumbers).
2. قمنا ضايفين رقم جديد للسته الأصلية: `numbers.Add(12)`;
3. مسحنا رقم من اللسته الأصلية: `numbers.Remove(4)`;
4. بعد كده عملنا `foreach` عشان نطبع الـ `evenNumbers`.

السؤال هنا: نفكر الطباعة هتطلع إيه؟

- الفريق الأول (بيفكر Sequential): هيقولك الكود بيمشي سطر سطر، فالكويري اتعملت قبل الإضافة والحذف، يبقى النتيجة هتكون بناءً على اللسته القديمة (يعني 2 و 4 بس).
- الفريق الثاني (الفاهم LINQ): هيقولك لأ، النتيجة هتكون (2, 6, 8, 10, 12) والـ 4 مش هتظهر.

الإجابة الصحيحة: الفريق الثاني هو اللي صح!

ليه؟ بسبب حاجة اسمها **Deferred Execution** (التنفيذ المؤجل) أو **Lazy Loading**.



يعني إيه **Deferred Execution**؟

لما بتكتب جملة `Where`، الكود مش بيتنفذ في لحظتها.

- الـ LINQ بتعمل "Construct" للكويري بس.
- بتعمل Object في الميموري (Wrapper) شايل حاجتين:
 1. الـ Reference للسته الأصلية (numbers).
 2. الشرط اللي أنت كتبتة (الـ Predicate).
- **التنفيذ الفعلي (Enumeration)** بيحصل بس لما تبدأ "تعد" أو تستخدم البيانات، وده بيحصل لما تعمل `foreach` أو `Count()` أو `ToList()`.
- لما الـ `foreach` اشتغلت، راحت بصت على اللسته "في حالتها الحالية" (بعد الإضافة والحذف) وطبقت عليها الشرط.

قاعدة ذهبية: طول ما أنت ما عملتش Iteration (تعداد)، أي تعديل في الداتا الأصلية هيسمع معاك في النتيجة.

استثناءات (Immediate Execution)

فيه بعض الميثودز في LINQ بتعمل تنفيذ فوري ومش بتستنى، زي:

- الـ `Count()`
- الـ `First()`
- الـ `ToList()`
- الـ `ToArray()` دول بيرجعوا قيمة محددة (Scalar) أو لسته جديدة فوراً، فبيعملوا Flush للكويري في ساعتها.

5. تشرح الـ LINQ (تحت الغطاء)

إزاي الـ LINQ بتشتغل داخلياً؟

1. الـ Wrapper Object

لما بتكتب `numbers.Where(...)`:

- الـ LINQ مش بتعمل لسته جديدة في الميموري (عشان توفر مساحة).
- هي بتعمل **Decorator** (غلاف) حوالين اللسته الأصلية. الغلاف ده بيقول: "أنا عندي ريفرنس للبيانات دي، ولما حد يطلب مني داتا، همشي الشرط ده عليها الأول".

2. مشكلة الـ ArrayList (Non-Generic)

لو عندك `ArrayList` (اللي هي بتقبل أي نوع بيانات: `int, string, object`):

C#

```
1 var collection = new ArrayList();
2 collection.Add(1);
3 collection.Add("Text");
4 // collection.Where(...) // هيديك Error!
```

- الـ LINQ مش بتشتغل مع الـ Collections القديمة دي مباشرة لأنها مش `<IEnumerable>`.
- الـ `Where` بتشتغل بس مع `<IEnumerable>` (لازم تكون Type Safe).

3. طرق كتابة كود LINQ المتعددة

في المحاضرة المهندس وضع إننا نقدر نكتب نفس اللوجيك بـ 3 طرق:

1. الـ **Extension Method** (الأكثر شيوعاً):

C#

```
1 var result = numbers.Where(x => x % 2 == 0);
```

2. الـ Static Method (نادرة الاستخدام بس هي الأصل):

C#

```
1 var result = Enumerable.Where(numbers, x => x % 2 == 0);
```

3. الـ Query Syntax:

C#

```
1 // Select n fROM numbers where n % 2 = 0;  
2 var result = from n in numbers  
3               where n % 2 == 0  
4               select n;
```

الثلاثة بيتحولوا لنفس الـ IL Code وبيدوا نفس النتيجة ونفس الأداء.

6. مثال معقد: Query on Query

يمكن تبني كويري فوق كويري، والاتنين يفضلوا **Lazy Evaluation**.

C#

```
1  
2 var employees = DataBaseLoad.LoadEmployees();  
3  
4 // 1. هات كل الموظفين الذكور  
5 var maleEmployees = employees.Where(e => e.Gender == "Male");  
6  
7 // 2. HR من الذكور دول، هات اللي شغالين في الـ  
8 var maleInHR = maleEmployees.Where(e => e.Department == "HR");  
9  
10 // 3. اطبع  
11 foreach(var emp in maleInHR) { ... }
```

- في الخطوة 1 و 2، مفيش أي تنفيذ حصل. مجرد تعريفات.
- التنفيذ الحقيقي حصل بس في الخطوة 3 (الـ `foreach`).
- ده بيخلي الكود **Efficient** جداً في الميموري لأنه مش بيعمل لستات وسيطة (Intermediate Lists).

7. ليه LINQ؟ (الخلاصة)

المهندس عصام لخص ليه الـ LINQ بقت مطلب أساسي لأي مبرمج:

1. الـ **Familiar Language**: لغة شبه الإنجليزي، سهولة القراءة والفهم (Descriptive). الكود يشرح نفسه.
2. الـ **Less Coding**: بدل ما تكتب 10 سطور Loop و If conditions، بتكتب سطر واحد.
3. الـ **Standard**: نفس السنتاكس بتستخدمه مع SQL, XML, Objects.
4. الـ **Type Safety**: اكتشاف الأخطاء وقت الـ Compilation مش وقت الـ Runtime.
مثال على ده
5. الـ **Intellisense Support**: دعم قوي من الـ IDE بيسهل الكتابة.

Projection Select (String, Math, DTO)

مقدمة: يعني إيه Projection؟

في الحلقة دي هنتكلم عن موضوع الـ **Projection Operations** في الـ LINQ. ببساطة، الـ Projection Operation هي عملية تحويل الـ Object من حالة لحالة أخرى.

- تشبيهه بالـ **Database**: لما كنا بنشتغل SQL ونعمل **SELECT** لأعمدة معينة (Columns)، إحنا كده بنعمل **Projection**. يعني أنا مش عايز كل الأعمدة تعرضها لي، أنا بختار أعمدة محددة وبسقط الباقي. نفس الفكرة موجودة في الـ LINQ.

أهداف عملية الـ Projection

إحنا بنلجأ للـ Projection لثلاثة أسباب رئيسية:

1. الـ **Constructing a New Type**: عايز أعمل Type جديد. البيانات المخزنة عندي مش عايز أعرضها كلها للـ Client (العميل).
 - السبب: ممكن تكون البيانات الكاملة حساسة (Sensitive) ومش عايز أكشفها، أو البيانات مش بتهم العميل كلها.
 - الحل: بنستخدم حاجة اسمها **DTO (Data Transfer Object)**. ده مجرد مسمى لـ Object وظيفته نقل البيانات، فبحول الـ Object الأصلي لـ DTO وأبعته للعميل.
2. الـ **Select Specific Property**: أنا عايز أبعت خاصية معينة بس (زي عمود واحد في الداتا بيز) مش الـ Object كله.
3. الـ **Perform Calculation**: ممكن أكون عايز أعمل عملية حسابية على البيانات قبل ما أرجعها.

العمليات المتاحة في LINQ Projection

عندنا 3 عمليات أساسية هنشرحهم بالتفصيل:

1. **Select**
2. **SelectMany**
3. **Zip**

تجهيز بيئة العمل (Setup)

قبل ما نبدأ في الأمثلة، المهندس عمل تعديلات على الـ Solution اللي اسمه **LinqTutorial**:

- استخدمنا **Shared Project** وفيه كلاس **Employee**.
- التعديلات على كلاس **Employee**:
 - تم إضافة Attributes جديدة: **Id**, **FirstName**, **LastName**, **Email**, **Male**.
 - تم إضافة **Skills List<string>**: دي لسته بمهارات الموظف (مثلاً: ASP.NET, CSS, Oracle).
 - موظف له مهاراته الخاصة.
 - عمل **Override** لدالة **ToString**: عشان يعرض المهارات مدمجة (**Joined**) بشكل نصي مقروء.

1. شرح الـ Select

الـ Select هي عملية تحويل عناصر موجودة في List إلى شكل آخر باتباع دالة تحويل (Transformation Function).

المثال الأول: تحويل الحروف (String Transformation)

لو عندنا مصفوفة (Array) من الكلمات وعازين نحول كل كلمة من حروف صغيرة (Small) لحروف كبيرة (Capital).

الكود:

C#

```
1 // Setup: Array of strings
2 string[] words = { "active", "angular", "asp", "c#", "core" };
3
4 // باستخدام LINQ Select
5 // x = words = input
6 var result = words.Select((x) => x.ToUpper());
7
8 // الطباعة
9 foreach (var word in result)
10 {
11     Console.WriteLine(word);
12 }
```

توضيح الكود:

• الـ Select يتاخذ دالة (Func) بتحول من Input String لـ Output String.

```
1 public static IEnumerable<TResult> Select<TSource, TResult>(
2     this IEnumerable<TSource> source, Func<TSource, TResult> selector)
3
4     //Func<TSource, TResult> selector
5     // فى الفن سى الى ع الشمال (السورس) هوا الانبوت الى هوا الى بينادى الفنكشن لان الفنكشن معمولها
6     //this IEnumerable<TSource> source
7     // الى على اليمين (الريزلت) ده الى هيرجع من الفنكشن على حسب ما هترجع
8     اكستنشن
```

- استخدمنا ToUpper() عشان نحول الحالة.
- ملحوظة مهمة: الـ C# Compiler بيعامل الـ Array معاملة خاصة وببيسمح نطبق عليها LINQ كأنها IEnumerable.
- لو الـ Expression في سطر واحد (One Line)، مش محتاج تكتب return ولا أقواس {}. لكن لو هتعمل Block، لازم تكتب return.

لاحظ هنا الاتنين واحد

```

1 var result01 = words.Select((x) => { return x; });
2 var result = words.Select((x) => x);

```

طريقة الـ Query Syntax لنفس المثال:

C#

```

1 var result = from word in words
2             select word.ToUpper();

```

الاثنين (Query Syntax و Method Syntax) بيطلعوا نفس النتيجة.

المثال الثاني: عمليات حسابية (Mathematical Operations)

لو عندنا لسته أرقام وعايزين نربع كل رقم (نضربه في نفسه).

الكود:

C#

```

1 List<int> numbers = new List<int> { 2, 3, 5, 7 };
2
3 // Method Syntax
4 var result = numbers.Select((x) => x * x);
5
6 // Query Syntax
7 var resultQuery = from n in numbers
8                   select n * n;

```

النتيجة هتكون مربع كل رقم في القائمة.

المثال الثالث: التحويل لنوع جديد (Transformation to DTO)

ده المثال الأهم واللي بيحاكي الواقع. عندنا Employee وعايزين نرجعه للعميل بس مش بكل بياناته، هنرجعه كـ EmployeeDto.

1. إنشاء كلاس الـ DTO:

C#

```

1 public class EmployeeDto
2 {
3     public string Name { get; set; } // الاسم الكامل

```

```

4         public int TotalSkills { get; set; } // عدد المهارات فقط
5
6         public override string ToString()
7         {
8             return $"Name:{Name} Skilles Count:{Skilles}";
9         }
10    }

```

- **ليه سميناه DTO؟** اختصار لـ Data Transfer Object. ده الـ Convention (العرف) عشان نعرف إن الكلاس ده وظيفته بس نقل البيانات من التطبيق للعميل بأمان.

2. كود التحويل (Projection):

C#

```

1 // تحميل الموظفين
2 var employees = Repository.LoadEmployees();
3
4 // والتحويل Select عملية الـ
5 var result = employees.Select(emp => new EmployeeDto
6 {
7     // دمج الاسم الأول والأخير
8     Name = $"{emp.FirstName} {emp.LastName}",
9
10    // الموجودة جوه الموظف List حساب عدد المهارات من الـ
11    TotalSkills = emp.Skills.Count
12 });
13
14 // (لو موجود أو طباعة الخصائص DTO في الـ override toString باستخدام) الطباعة
15 foreach(var emp in result)
16 {
17     Console.WriteLine($"{emp.Name} : {emp.TotalSkills}");
18 }

```

شرح اللي حصل:

- إحنا مبعتناش الـ Object الأصلي (Employee) إحنا خدنا منه الى عاوزينه عن طريق `Select`.
- أنشأنا `New EmployeeDto` جوه الـ `Select`.
- عملنا عملية دمج (Concatenation) للاسم.
- عملنا عملية حسابية (Count) للمهارات.
- النتيجة: بيانات مخصصة وجاهزة للعرض.

نهاية الجزء الأول: في الجزء ده غطينا المقدمة وعملية الـ `Select` بكل صورها (String, Math, DTO).

الجزء القادم (باقي المحاضرة): هشرحك بالتفصيل الـ **SelectMany** والفرق الجوهرى بينها وبين الـ **Select**، وكمان عملية الـ **Zip** واستخداماتها الذكية لدمج البيانات.

Projection (SelectMany & Zip)

2. شرح الـ SelectMany

الـ SelectMany بتعمل حاجتين ورا بعض:

1. الـ **Projecting**: بتعمل Select عادي.
2. الـ **Flattening**: بتفك النتائج كلها وتخليها في Sequence واحد مستوي (Flat).

إمتى نستخدمها؟ (الفرق بينها وبين Select)

لو كل عنصر في القائمة الأصلية بيحتوي جواه على "قائمة" ثانية، وأنت عايز تطلع كل العناصر الفرعية دي في لسته واحدة كبيرة، هنا تستخدم **SelectMany**.

- الـ **Select**: بترجع `<<List<List<Item` (قائمة جواها قوائم).
- الـ **SelectMany**: بترجع `<List<Item` (قائمة واحدة فيها كل العناصر).

المثال الأول: تحليل الجمل لكلمات (String Tokenization)

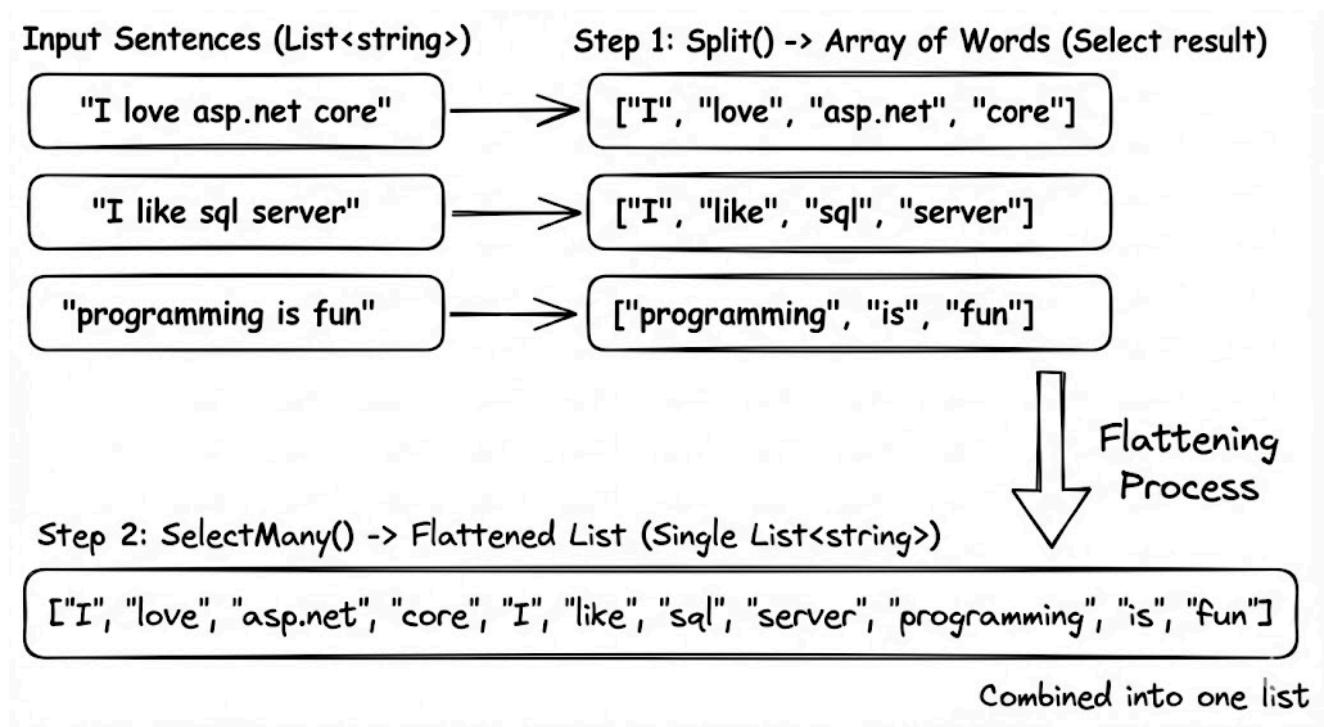
عندنا قائمة جمل، وعايزين نطلع كل الكلمات اللي في كل الجمل دي ونحطها في لسته واحدة.

الكود:

C#

```
1 // 1. قائمة جمل
2 List<string> sentences = new List<string>
3 {
4     "I love asp.net core",
5     "I like sql server",
6     "programming is fun"
7 };
8
9 // 2. SelectMany التحويل باستخدام
10 // لكل جملة Array of Strings بترجع دالة Split
11 // var = IEnumerable<string>
12 var words = sentences.SelectMany(x => x.Split(' '));
13
14 // 3. الطباعة
15 foreach (var word in words)
16 {
17     Console.WriteLine(word);
18 }
```

اللي حصل: دالة **Split** بترجع مصفوفة كلمات لكل جملة. الـ **SelectMany** أخذت المصفوفات دي كلها ودمجتها (Flatten) في مصفوفة واحدة كبيرة فيها كل الكلمات.



المثال الثاني: استخراج مهارات الموظفين (Flattening Lists)

عندنا لسته Employees ، وكل موظف عنده List<string> Skills . عايزين نطلع لسته بكل المهارات الموجودة في الشركة كلها.

الكود:

C#

```

1  var employees = Repository.LoadEmployees();
2
3  // للدخول للليسته الداخلية SelectMany استخدام
4  var allSkills = employees.SelectMany(emp => emp.Skills);
5
6  // skill asp.net مثلاً كذا موظف عندهم) عشان نشيل التكرار
7  var uniqueSkills = allSkills.Distinct();
8
9  foreach (var skill in uniqueSkills)
10 {
11     Console.WriteLine(skill);
12 }
  
```

- **ملاحظة:** الـ Select العادية هنا كانت هترجع <<List<List<string> >> وده مش اللي إحنا عايزينه، إحنا عايزين <List<string> واحدة.
- استخدمنا Distinct() عشان نمنع تكرار أسماء المهارات.
- كان ممكن تختصر السطر 4 و 7 فى سطر واحد عن طريق عمل **Chaining** (ربط)، يعني نادى على Distinct بعد ما SelectMany خلصت شغلها عشان "ينظف" النتيجة.


```
1 var allSkills = employees.SelectMany(emp => emp.Skills).Distinct();
```

🔥 Important

الـ **Distinct** مش تابعة للـ **Select** ولا للـ **SelectMany** كجزء منهم (مش بارامتر ليهم يعني)، هي دالة مستقلة بذاتها (**Standalone Extension Method**) في الـ LINQ.

طريقة Query Syntax للـ SelectMany

في الـ Query Syntax، الـ **SelectMany** بتتمثل بإنك تستخدم اثنين **from** ورا بعض.

الكود:

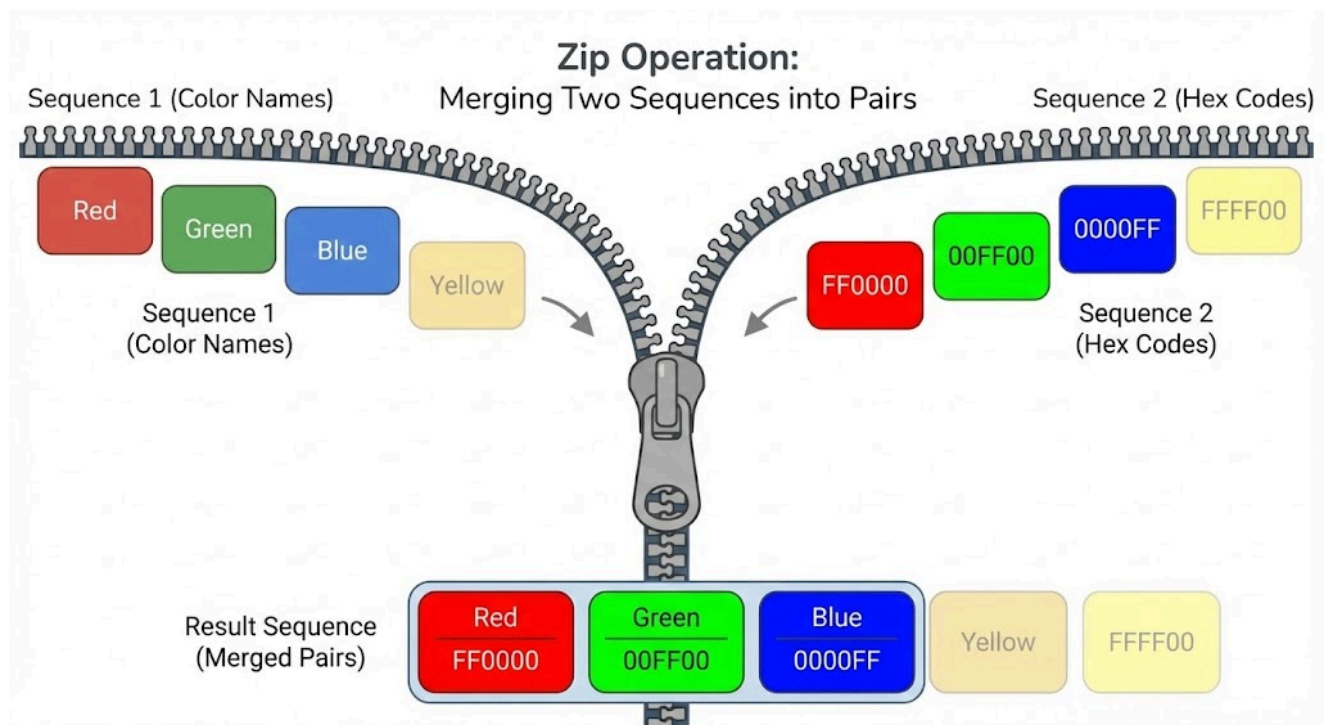
C#

```
1 var result = from emp in employees           // Loop 1: على الموظفين
2               from skill in emp.Skills       // Loop 2: على مهارات كل موظف
3               select skill;                  // النتيجة النهائية
4
5 // لازم تعمل أقواس حول الاستعلام كله في الـ Query Syntax عشان تعمل
6 var distinctResult = (from emp in employees
7                       from skill in emp.Skills
8                       select skill).Distinct();
```

الـ **Two from clauses** دول بيترجموا في الكومبايلر للـ **SelectMany**.

3. شرح الـ Zip

الـ **Zip Operation** فكرتها زي "السوستة" (**Zipper**). هي عملية دمج (**Merging**) بين 2 Sequences (قائمتين مختلفتين) عشان تطلع أزواج (**Pairs**) جديدة.



طريقة العمل

بتمشي على القائمة الأولى والقائمة الثانية في نفس الوقت، ويتخذ العنصر الأول من هنا مع الأول من هنا، والثاني مع الثاني، وهكذا.

سؤال مهم: لو القائمتين مش نفس الطول (واحدة أطول من الثانية)؟

- الإجابة: الـ Zip بتمشي لحد ما أقصر قائمة فيهم تخلص، وتتجاهل العناصر الزيادة في القائمة الأطول.

المثال الأول: دمج الألوان (Colors & Hex)

عندنا لسته أسماء ألوان، ولسته أكواد الـ Hex بتاعتها.

الكود:

C#

```
1 string[] colorNames = { "Red", "Green", "Blue" };
2 string[] hexCodes = { "FF0000", "00FF00", "0000FF", "FFFFFF" }; // لاحظ
   // العنصر الرابع زيادة
3
4 // استخدام Zip
5 // parameter يتخذ القائمة الثانية كـ
6 // (name, hex) ويتخذ دالة تحدد شكل النتيجة
7 var colors = colorNames.Zip(hexCodes, (name, hex) => $"{name} ({hex})");
8
9 foreach (var item in colors)
10 {
11     Console.WriteLine(item);
12 }
```

النتيجة: هيطبع Red , Green , Blue مع أكوادهم. وهيتجاهل الـ FFFFFFF لأن ملوش مقابل في قائمة الأسماء.

المثال الثاني: تكوين فرق عمل (Teams)

عايزين نكون فرق عمل، بحيث نأخذ أول 3 موظفين في القائمة، وندمجهم مع آخر 3 موظفين.

الكود:

C#

```
1 var employees = Repository.LoadEmployees();
2
3 // الجديدة في C# Indices والـ Ranges استخدام الـ
4 // أول 3 موظفين
5 var firstThree = employees[..3]; // أو employees.Take(3)
6
7 // آخر 3 موظفين
8 var lastThree = employees[^3..]; // أو employees.TakeLast(3)
9
10 // دمجهم مع بعض باستخدام Zip
11 var teams = firstThree.Zip(lastThree, (firstEmp, lastEmp) =>
12     $"{firstEmp.FirstName} with {lastEmp.FirstName}");
13
14 foreach (var team in teams)
15 {
16     Console.WriteLine(team);
17 }
```

هنا دمجنا كل موظف من القائمة الأولى مع الموظف المقابل له في القائمة الثانية.

هل يوجد Query Syntax للـ Zip؟

لا يوجد Keyword مباشرة للـ Zip في الـ Query Syntax. لو عايز تستخدمها، بتضطر تكتبها بـ Method Syntax عادي، أو ممكن تعمل خليط (Mix)، بس الأفضل استخدام الـ Method Syntax لأنها أوضح في الحالة دي.

خاتمة المحاضرة: دي كانت عمليات الـ Projection الثلاثة:

1. الـ **Select**: تحويل عنصر لعنصر (1-to-1).
2. الـ **SelectMany**: تحويل عنصر لقائمة ثم دمجهم (Flattening) (1-to-Many).
3. الـ **Zip**: دمج قائمتين مع بعض لإنتاج قائمة واحدة من الأزواج.

Sorting (OrderBy & IComparer)

مقدمة: ليه بنحتاج الـ Sorting؟

النهاردة حلقتنا رقم 5 وموضوعنا عن LINQ Sorting. مين فينا مش بيحتاج يرتب البيانات؟ سواء عشان تعرضها للـ End User بشكل منظم، أو عشان تطبق ترتيب مخصص (Custom Order). عشان كذا، الباشمهندس عمل Solution جديد ضاف فيه 4 مشاريع (Projects) بالإضافة لـ Shared Project عشان نغطي كل فكرة في مشروع لوحدها.

تعديلات الـ Data Setup

قبل ما نبدأ، حصل تعديل على كلاس Employee :

- الـ ID اتشال وبقي مكانه EmployeeNumber وده عبارة عن String مش رقم عادي.
- الـ EmployeeNumber واخد فورمات معين مقسم لـ 3 أجزاء بينهم شرطة (Hyphen -) :
 1. سنة التعيين (مثلاً 2017).
 2. القسم (مثلاً FA للمالية، HR للموارد البشرية).
 3. رقم تسلسلي (Sequence).
- مثال: FA-1234-2017 .

1. الترتيب الأساسي (Primary Sorting) - OrderBy

هنبدأ مع أول نوع وهو الترتيب باستخدام OrderBy . دي Extension Method بترجع IEnumerable وموجودة في System.Linq .

أ. الترتيب التصاعدي (Ascending)

الترتيب الافتراضي (Default) هو التصاعدي (من الصغير للكبير أو أبجدياً A-Z).

1. باستخدام Method Syntax

لو عندنا قائمة فواكه (String Array)، وعايزين نرتبها أبجدياً:

C#

```
1 string[] fruits =  
2     { "apricot", "orange", "banana", "mango", "apple", "grape",  
3       "strawberry" };  
3 // Fruits هي القائمة الأصلية  
4 var orderedFruits = fruits.OrderBy(f => f);
```

الـ f هنا بتمثل كل عنصر (String)، وهيرتب بناءً عليها أبجدياً

2. باستخدام Query Syntax

C#

```
1 var orderedFruits = from f in fruits // location
2                       orderby f // type of order
3                       select f; // return value using select
```

ملحوظة: لو كتبت `orderby` بس من غير ما تحدد، هو بيعتبرها `ascending` تلقائياً.

ب. الترتيب التنازلي (Descending)

لو عايز ترتب بالعكس (من الكبير للصغير (Z-A).

1. باستخدام Method Syntax

بنستخدم ميثود اسمها `OrderByDescending`:

C#

```
1 var orderedFruits = fruits.OrderByDescending(f => f);
```

2. باستخدام Query Syntax

بنكتب كلمة `descending` بعد اسم المتغير في جملة الـ `orderby`:

C#

```
1 var orderedFruits = from f in fruits
2                       orderby f descending // كلمة السر هنا
3                       select f;
```

ج. الترتيب بناءً على خاصية (Property)

ممكن نرتب مش بناءً على العنصر نفسه، لكن بناءً على خاصية فيه، زي "طول الكلمة" (`Length`).

مثال: ترتيب الفواكه حسب عدد الحروف (من الأقصر للأطول):

C#

```
1 // Method Syntax
2 var sortedByLength = fruits.OrderBy(f => f.Length);
3
```

```

4 // Query Syntax
5 var sortedByLength = from f in fruits
6                       orderby f.Length
7                       select f;

```

2. الترتيب الثانوي (Secondary Sorting) - ThenBy

إليه هو الترتيب الثانوي؟ نفترض أنك بترتب موظفين حسب الاسم، طب لو اتنين موظفين ليهم نفس الاسم؟ هنا بنحتاج معيار ثاني للترتيب (زي المرتب مثلاً).

الميثودز المستخدمة:

- الـ ThenBy : للترتيب الثانوي تصاعدياً.
- الـ ThenByDescending : للترتيب الثانوي تنازلياً.

مثال عملي:

عايزين نرتب الموظفين حسب الاسم (تصاعدي)، ولو تساوت الأسماء، نرتبهم حسب المرتب (تصاعدي).

C#

```

1 var sortedEmps = emps.OrderBy(x => x.Name) // الترتيب الأساسي
2                   .ThenBy(x => x.Salary); // الترتيب الثانوي في حالة تساوي الاسم

```

مثال ثاني: الاسم تصاعدي، لكن المرتب تنازلي (الأغنى الأول):

C#

```

1 var sortedEmps = emps.OrderBy(x => x.Name)
2                   .ThenByDescending(x => x.Salary);

```

نقطة فنية مهمة: الميثودز دي (OrderBy , ThenBy , etc) بترجع نوع اسمه `IOrderedEnumerable<T>`. النوع ده هو اللي بيخلينا نقدر نعمل Chaining (نربط الميثودز ببعض زي `.ThenBy().OrderBy()`).

3. الترتيب المخصص (Custom Sorting)

أحياناً الترتيب العادي مش بينفع. فاكرين الـ `EmployeeNumber` اللي كان فورمات (سنة-قسم-رقم)؟ لو رتبناه كـ `String` عادي، الكمبيوتر هيقارن حرف بحرف. يعني `FA-3-2017` ممكن تيجي قبل `FA-10-2017` لأن الـ 3 أكبر من الـ 1 في المقارنة النصية، وده غلط في الترتيب الرقمي.

الحل: نستخدم `IComparer` interface عشان نعمل منطق ترتيب خاص بيينا.

خطوات عمل Custom Comparer :

1. نعمل كلاس جديد ببيورث من `IComparer<Employee>`.
2. نعمل Implementation للميثود `Compare`.
3. نكتب اللوجيك بتاعنا جوه الميثود:
 - نقسم الـ String لأجزاء (Split) بناء على الـ Hyphen - .
 - نحول السنة والرقم التسلسلي لـ `Integers`.
 - نقارن السنة الأول، لو متساويين نقارن الرقم التسلسلي (Sequence).

كود الـ Comparer :

C#

```
1 internal class EmployeeComparer : IComparer<Employee>
2 {
3     public int Compare(Employee e1, Employee e2)
4     {
5         // "2017-FI-1343" => "2017", "FI", "1343"
6         var e1Year = Convert.ToInt32(e1.EmployeeNo.Split("-")[0]);
7         var e2Year = Convert.ToInt32(e2.EmployeeNo.Split("-")[0]);
8         var e1Seq = Convert.ToInt32(e1.EmployeeNo.Split("-")[2]);
9         var e2Seq = Convert.ToInt32(e2.EmployeeNo.Split("-")[2]);
10
11         // معناها رجع العنصر الى في اندكس صفر الى هوا هيكون السنة [0]
12         if (e1Year == e2Year)
13         {
14             return e1Seq.CompareTo(e2Seq);
15         }
16         else
17         {
18             return e1Year.CompareTo(e2Year);
19         }
20     }
21 }
22
```

طريقة الاستخدام:

بنبعت نسخة (Instance) من الـ Comparer ده كـ Parameter ثاني في دالة `OrderBy` :

C#

```
1 var sortedEmps = emps.OrderBy(x => x, new EmployeeComparer());
```

كده هو هيرتب بناءً على اللوجيك اللي إحنا كتبناه (السنة ثم الرقم التسلسلي) وهيتجاهل القسم اللي في النص.

تحذير: ال Custom Comparer ممكن يآثر على ال Performance لو البيانات كبيرة جداً، فلازم تكون حكيم في استخدامه أو تستخدم Pagination وفلتر في الداتا بيز الأول.

▶ 0:00 / 0:29

```
1 var WordsOfemp1 = e1.EmployeeNo.Split("-");
2
3 // how to use
4 WordsOfemp1[0];
5 WordsOfemp1[1];
6 WordsOfemp1[2];
```

هفكر ك تانى ليه هوا استخدم Convert.ToInt32

ال EmployeeNumber في الداتا بيز أو الكلاس نوعه String ، وشكله كذا: FA-10-2014 أو FA-3-2014 .

لو سبنا الكمبيوتر يقارنهم ك نصوص (Strings) من غير تحويل، الكمبيوتر بيقارن "حرف بحرف" من الشمال لليمين.

- تعال نقارن 3 و 10 ك نصوص:
- الكمبيوتر هيقارن الحرف الأول '3' مع الحرف الأول '1' .
- طبعا الواحد أصغر من الثلاثة.
- فالنتيجة هتكون إن ال 10 بتيجي قبل ال 3 في الترتيب الأبجدي!.

الهدف: الترتيب الرقمي (Numerical Sort)

إحنا عايزين نرتب بناءً على القيمة الرقمية:

- إحنا عارفين إن ال 3 أصغر من ال 10 ، فلازم ال 3 تيجي الأول.
- عشان الكمبيوتر يفهم ده، لازم نحول النص ده لرقم (Integer)، ساعتها هيقارن قيم (Value) مش حروف (Characters).

التطبيق في الكود

عشان كده المهندس عمل الخطوات دي جوه ال Comparer :

1. ال Split: قطع النص عشان ياخذ الجزء الخاص بالسنة والجزء الخاص بالسيكونس (المسلسل).
2. ال Convert: حول الأجزاء دي لأرقام باستخدام Convert.ToInt32 عشان لما ييجي يقارن، يقارن أرقام بجد.

4. العكس (Reverse)

آخر حاجة معناها هي Reverse . وظيفتها بسيطة جداً: هي بتشقلب القائمة زي ما هي، بتعكس ترتيب العناصر (الأول يبقى في الآخر والآخر في الأول).


```
1 var reversedFruits = fruits.Reverse();
```

ملاحظة خطيرة: الـ Reverse بترجع IEnumerable (بتشتغل بنظام Deferred Execution في اللينك). يعني لو ناديتها كذا `fruits.Reverse()` ; من غير ما تخزن الناتج في متغير، مش هيحصل حاجة للقائمة الأصلية ومش هتشوف أي تغيير. لازم تسند النتيجة لمتغير جديد عشان لما تعمل Loop عليه تلاقي الترتيب انعكس فعلاً.

Data Partitioning (Skip, Take, Chunk, Pagination)

❶ مقدمة الحلقة دي رقم 6 من سلسلة الـ LINQ التابعة لكورس الـ Full Stack Developer. موضوعنا النهاردة حيوي جداً ومهم لأي حد بيشتغل ويب أو ديسكتوب، وهو **Data Partitioning** أو تقسيم البيانات. ببساطة، لما بتدخل على فيسبوك أو أي موقع وتلاقيه بيعرض لك أول 10 بوستات، وبعدين تعمل Scroll يجيبك الـ 10 اللي بعدهم، العملية دي هي الـ **Data Partitioning**.

1. تجهيز بيئة العمل (Setup)

- احنا شغالين على Solution اسمه LINQ Tutorial 6.
- في عندنا Shared Project شرحناه في الحلقات اللي فاتت (خاصة الحلقة الأولى والثانية)، وكل المشاريع النهاردة بتعتمد عليه.
- الداتا اللي هنشتغل عليها هي قائمة من الـ Employees (موظفين)، واعتبرها جاية من Database، بس هي فعلياً Fake Data في الكود.
- تعديل بسيط في الموديل: ضفت خاصية Index لكل موظف عشان نقدر نميز ترتيبهم بسهولة وإحنا بنجرب (الأول، الثاني، وهكذا).

2. عائلة التجاهل (Skip Methods)

الفكرة الأساسية هنا إننا عايزين نتجاهل عدد معين من العناصر ونبدأ نأخذ الباقي.

أ- دالة Skip

- وظيفتها: تجاهل عدد محدد من العناصر من بداية القائمة.
- مثال: لو عايز أتجاهل أول 10 موظفين وأعرض الباقي.
- الكود:

C#

```
1 using System.Linq;
2
3 // emps هي القائمة الكاملة للموظفين
4 var query1 = emps.Skip(10);
5
6 // (مجهزة مسبقاً Extension Method) طباعة النتيجة
7 query1.Print("Skip First 10 Employees");
```

- النتيجة: هيعرض الموظفين من رقم 11 لحد 50 (لأننا عندنا 50 موظف وتجاهلنا أول 10).

ب- دالة SkipWhile

- **وظيفتها:** تفضل تتجاهل العناصر طالما (While) الشرط اللي إنت حاطه متحقق. أول ما تلاقي عنصر يكسر الشرط، توقف تجاهل وتأخذ العنصر ده وكل اللي بعده.
- **المدخلات:** بتأخذ Lambda Expression (Anonymous Function) بيمثل الشرط.
- **مثال:** تجاهل الموظفين طالما الراتب بتاعهم مش بيساوي 21400 .
- **الكود:**

C#

```
1 // تجاهل أي موظف طالما راتبه لا يساوي 21400
2 // أول ما تلاقي واحد راتبه 21400 ابدأ اعرض من عنده وهات الباقي كله
3 var query2 = emps.SkipWhile(x => x.Salary != 21400);
4
5 query2.Print("Skip While Salary != 21400");
```

- **النتيجة:** السيستم هيبداً يتجاهل العناصر من الأول لحد ما يوصل للموظف اللي راتبه 21400 (كان ترتيبه 13 تقريباً في المثال)، فيبدأ يعرض من عنده لحد آخر القائمة.

ج- دالة SkipLast

- **وظيفتها:** تجاهل عدد محدد من العناصر بس من آخر القائمة.
- **مثال:** تجاهل آخر 10 موظفين.
- **الكود:**

C#

```
1 var query3 = emps.SkipLast(10);
2
3 query3.Print("Skip Last 10 Elements");
```

- **النتيجة:** هيعرض الموظفين من 1 لحد 40، والـ 10 اللي في الآخر (من 41 لـ 50) مش هيبجوا.

3. عائلة الأخذ (Take Methods)

دي عكس الـ Skip تماماً، بدل ما أتجاهل، أنا "بأخذ" عناصر معينة.

- الـ **Take(10)** : خذ أول 10 عناصر بس واتجاهل الباقي.
- الـ **TakeWhile(...)** : خذ العناصر طالما الشرط متحقق، وأول ما الشرط يتكسر وقف وما تاخدش حاجة تاني.
- الـ **TakeLast(10)** : خذ آخر 10 عناصر في القائمة بس.

الفرق الجوهرى

- في **Skip**: بتفوت الأول وتأخذ الباقي. في **Take**: بتأخذ الأول وتفوت الباقي.

4. تقسيم المجموعات (Chunk Method)

دي ميزة جديدة نزلت مع .NET 6 (لازم تخلي البروجكت بتاعك .NET 6 عشان تشتغل، لو .NET 5 مش هتلاقيها).

- **وظيفتها:** بتقسم القائمة الكبيرة لمجموعات (Groups) صغيرة، كل مجموعة ليها حجم ثابت (Size).
- **تشبيه:** زي المدرس لما يدخل فصل فيه 40 طالب ويقسمهم مجموعات، كل مجموعة 10 طلاب.
- **الاستخدام:** مفيدة جداً لو عندك عملية معالجة ثقيلة (Heavy Operation) وعاليز تقسمها على مراحل (Batches)، وده مشهور جداً في شغل الـ Machine Learning والـ Python.

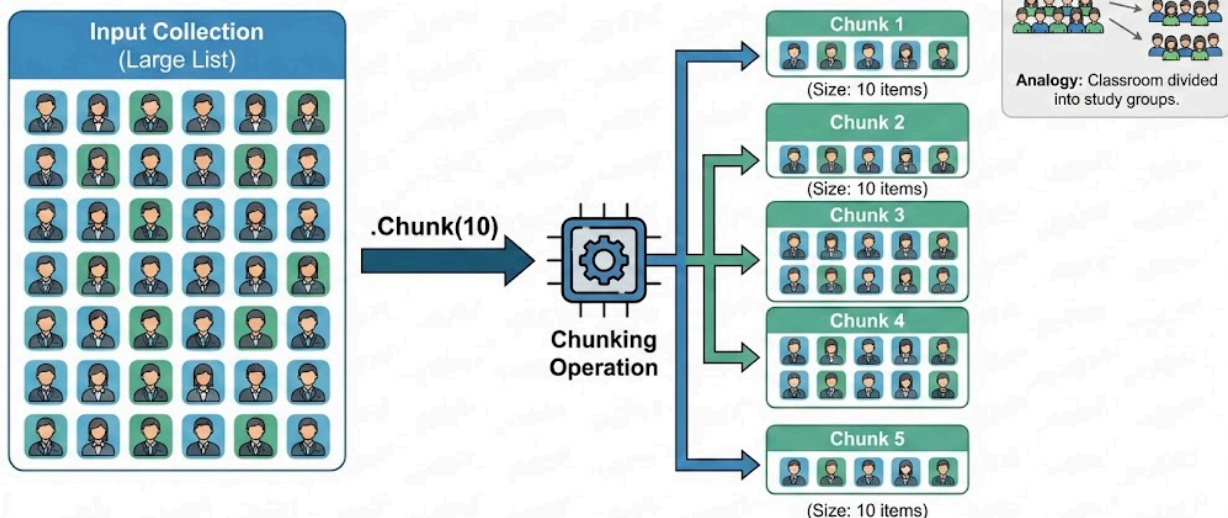
كود الـ Chunking:

C#

```
1 // قسم الموظفين لمجموعات، كل مجموعة فيها 10 موظفين
2 var chunks = emps.Chunk(10).ToList();
3
4 // عشان نعرضهم محتاجين Loop
5 for (int i = 0; i < chunks.Count; i++)
6 {
7     // chunks[i] دي عبارة عن مصفوفة فيها 10 موظفين
8     chunks[i].Print($"Chunk {i + 1}");
9 }
```

- **النتيجة:** هيقسم الـ 50 موظف لـ 5 مجموعات، كل مجموعة فيها 10.

Data Chunking in .NET 6



Result: The large collection is divided into smaller, manageable groups of a fixed size.

توضيح ليه استخدم **ToList** ومسبهاش **IEnumerable** ؟

عشان يقدر يستخدم الـ **Indexer** (الوصول بالعنصر)

لو تلاحظ في الكود، هو استخدم **For Loop** عادية مش **Foreach**:

```

1  for (int i = 0; i < chunks.Count; i++) //
2  {
3      var group = chunks[i]; // Index الوصول للعنصر عن طريق الـ
4  }

```

- الناتج اللي بيرجع من دالة `Chunk()` هو نوعه `IEnumerable`. النوع ده (`IEnumerable`) في السي شارب مابيد عمش الوصول عن طريق الاندكس (يعني مينفعش تقول `variable[0]` أو `variable[i]`).
- عشان يقدر يكتب `chunks[i]` جوه اللوب، كان لازم يحول الناتج لـ `List` لأن الليست هي اللي بتسمحك تدخل على العنصر برقمه المباشر.

عشان خاصية الـ Count

- في شرط اللوب هو كتب: `i < chunks.Count`.
 - الـ `IEnumerable` مفيش فيه خاصية (`Property`) اسمها `Count`. هو فيه دالة (`Method`) اسمها `Count()` ودي عشان تشتغل بتضطر تعد العناصر واحد واحد كل مرة.
 - الـ `List` فيها خاصية `Count`. جاهزة وسريعة، فكان لازم يحولها ليست عشان الكود ده يشتغل.
- الخاصية (`Count Property`): دي قيمة مُخزنة مسبقاً (`Pre-calculated`) جوه الـ `List` في الميموري، فالوصول ليها بيكون لحظي بتعقيد زمني ($O(1)$) لأنها مجرد قراءة لرقم جاهز.
- الدالة (`Count() Method`): دي عملية تنفيذية (`Operation`) بتضطر تعمل `Iteration` (تكرار) وتمر على كل عنصر في الـ `IEnumerable` عشان تعددهم، فبتستهلك وقت أطول بتعقيد زمني ($O(N)$) وكان في الفور لوب هيروح يستدعي الـ `Method` دى ويعد من اول وجديد على عكس الـ `Property` ببيسترجع رقم متخزن اصلا.

هل كان ممكن مايعملش `ToList()` ؟

أبوة، كان ممكن يستغنى عنها لو غير طريقة اللوب واستخدم `foreach` بدل `for`، الكود كان هيبقى كدا:

```

1  // هنا مش محتاجين ToList
2  var chunks = emps.Chunk(10);
3
4  int counter = 1;
5  foreach(var group in chunks)
6  {
7      group.Print($"Chunk {counter}");
8      counter++;
9  }

```

بس هو في الشرح فضل يستخدم الـ `for` عشان يطبع رقم المجموعة (`i + 1`) باستخدام العداد `i`، فعشان كذا اضطر يعمل `ToList()`.

ملحوظة عن التنفيذ (Execution) المهندس ذكر في الشرح جملة: "وخلينا نعمل توليست عشان ابش نعمل الاكسيوشن". ده لأن الـ LINQ بطبيعته Lazy (كسول)، مش بينفذ الأمر غير لما تطلب منه (Deferred Execution). لما بتكتب .ToList() أنت بتجبره ينفذ التقسيم فوراً ويحط النتيجة في الميموري، وده بيسهل التعامل معاها لو هتستخدم الاندكس زي ما هو عمل.

5. تطبيق عملي: نظام التصفح (Pagination)

ده أهم جزء في الحلقة، وهو تطبيق حقيقي بنستخدمه في الشغل (Real World Scenario).

الهدف

عايزين نعمل دالة، اليوزر يديها:

1. رقم الصفحة (Page Number).
2. حجم الصفحة (Page Size) "يعني عايز كام عنصر في الصفحة الواحدة". والدالة ترجع له البيانات المطلوبة بس.

Previous

1

2

3

4

5

6

Next

Rows: 10

Page: 1

Total: 58

Sr#	Country Name	ID	Country Code	Currency Code	Capital
1	ALGERIA	62	DZ	DZD	ALGIERS
2	ANGOLA	8	AO	AOA	LUANDA
3	BENIN	25	BJ	XOF	PORTO-NOVO
4	BOTSWANA	35	BW	BWP	GABORONE
5	BURKINA FASO	21	BF	XOF	OUAGADOUGOU
6	BURUNDI	24	BI	BIF	BUJUMBURA

الخطوة 1: عمل Extension Method

هنعمل Static Class ونسميه Extensions ونعمل جواه دالة Paginate تكون Generic عشان تشتغل على أي نوع بيانات مش بس الموظفين.

C#

```
1 public static class Extensions
2 {
3     // T لازم يكون Class (Constraint)
4     public static IEnumerable<T> Paginate<T>(this IEnumerable<T> source,
5     int page=1, int size=10) where T : class
6     {
7         // 1. Validation (تصحيح القيم الخطأ)
8         if (page <= 0) page = 1; // مفيش صفحة رقم 0 أو سالب
9         if (size <= 0) size = 10; // قيمة افتراضية لو الساييز مش منطقي
10
11        // 2. حساب عدد العناصر الكلي
12        var total = source.Count();
```

```

12
13 // 3. حساب عدد الصفحات الكلي (للعلم بالشيء)
14 // (Ceiling) ينقسم العدد الكلي على حجم الصفحة ونقرب للرقم الأكبر
15 // عشان القسمة تطلع بكسور الأول decimal حولنا لـ
16 var totalPages = (int)Math.Ceiling((decimal)total / size);
17
18 // 4. (المعادلة السحرية) LINQ تطبيق اللوجيك بالـ
19 // Skip: مضروب في حجم الصفحة (page - 1) بنفوت عدد صفحات
20 // ؟ لأن لو أنا في صفحة 1، مش عايز أفوت حاجة (0=1-1) ليه (page - 1)
21 // لو في صفحة 2، عايز أفوت حجم صفحة واحدة (1=2-1)
22 var result = source.Skip((page - 1) * size)
23                     .Take(size); // وبعدين بنأخذ عدد العناصر المطلوب للعرض
24
25 return result;
26 }
27 }

```

الخطوة 2: استخدام الدالة في الـ Main

هنطلب من اليوزر يدخل البيانات ونعرض النتيجة.

C#

```

1 // (Validation) قراءة المدخلات من اليوزر مع التأكد من صحتها
2 int page = 1;
3 int size = 10;
4
5 Console.Write("Result Per Page: ");
6 if(int.TryParse(Console.ReadLine(), out int resultSize))
7 {
8     size = resultSize;
9 }
10
11 Console.Write("Page Number: ");
12 if(int.TryParse(Console.ReadLine(), out int resultPage))
13 {
14     page = resultPage;
15 }
16
17 // Pagination تحميل البيانات واستدعاء دالة الـ
18 var emps = Repository.LoadEmployees();
19 var result = emps.Paginate(page, size); //

```

الخطوة 3: تنظيم واجهة العرض (UI Logic)

عايزين نكتب لليوزر "إحنا بنعرض من العنصر رقم X للعنصر رقم Y". الحسبة دي فيها شوية تفكير.

1. بداية العرض (Start Record):

- المعادلة: $Size + 1 * (Page - 1)$.
- **ليه +1؟** لأن الـ Index يبدأ من 0، بس اليوزر بيعد من 1. لو أنا في صفحة 2 وحجم 10، يبقى فوت 10 عناصر، يبقى بدايتي العنصر رقم 11.

2. نهاية العرض (End Record):

- هنا في "تريك" بسيطة. لو أنا في آخر صفحة، ممكن العناصر تكون أقل من الـ Size المطلوب.
- الشرط: لو `result.Count() < size` ده معناه إني في آخر صفحة.
- لو آخر صفحة: `StartRecord + resultCount - 1`.
- لو مش آخر صفحة: `Page * Size`.

كود العرض النهائي:

C#

```
1 var resultCount = result.Count();
2 var startRecord = ((page - 1) * size) + 1;
3
4 // استخدام Ternary Operator
5 var endRecord = resultCount < size
6                 ? startRecord + resultCount - 1
7                 : size * page;
8
9 // الطباعة
10 result.Print($"Showing Employees {startRecord} - {endRecord}");
```

تجربة الكود (Testing)

• التجربة 1:

- الحجم: 20
- الصفحة: 2
- النتيجة: جاب الموظفين من 21 لـ 40 (لأن الصفحة الأولى خدت من 1-20).

• التجربة 2:

- الحجم: 20
- الصفحة: 3
- النتيجة: المفروض الصفحة الثالثة تبدأ من 41. وإحنا كلنا 50، يبقى هيعرض من 41 لـ 50 بس (آخر 10 عناصر).

في الخلاصة

- استخدم `Skip` و `Take` لعمليات الـ Pagination.
- استخدم `SkipWhile` و `TakeWhile` لو الفلترة معتمدة على شرط منطقي مش عدد.
- استخدم `Chunk` (في .NET 6+) لو عايز تقسم الداتا لمجموعات للمعالجة.

- دائماً راعي الـ Indexes (0-based) لما تتعامل مع الـ User Input (1-based).

Quantifiers (Any, All, Contains)

مقدمة: يعني إيه Quantifiers؟

النهاردة موضوعنا عن الـ **Quantifiers**، وده موضوع مهم جداً في الـ LINQ لأنه بيتعلق بالإجابة بـ "أه" أو "لا" (Yes or No).

- تخيل دكتور بيدخل قاعة محاضرات ويبسأل: "هل فلان موجود؟"، الإجابة بتكون يا "نعم" يا "لا".
- ببساطة، الـ Quantifiers بترجع **Boolean Value (True/False)** بناءً على شرط (Condition) معين تحقق على عنصر أو ما تحققش.
- الدوال اللي هنتكلم عنها النهاردة بالتفصيل هي:
 1. الـ Any
 2. الـ All
 3. الـ Contains

التجهيز (Setup)

عشان نطبق عملي، إحنا شغالين على Console Application مبني على .NET Core، وعاملين Import لبروجكت اسمه Shared فيه كلاس Employee .

- بيانات الموظف (**Employee**): (Number, Name, Email, Salary, Skills).
- الداتا: عندنا Repository بتدينا مجموعة موظفين بشكل عشوائي عشان نجرب عليهم.

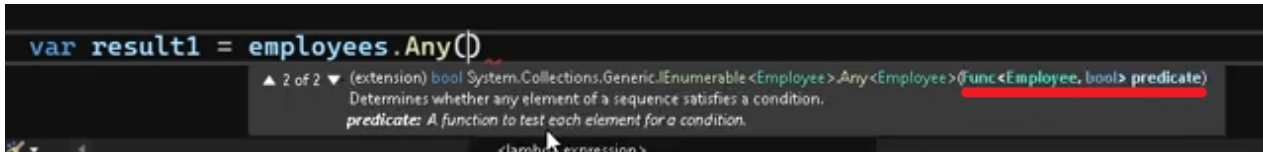
1. دالة Any

الـ Any ببساطة بتلّف على العناصر عنصر عنصر، وبتشوف هل الشرط تحقق ولا لأ. أول ما الشرط يتحقق، بترجع **True** وبتوقف تدوير (Short-circuit).

source ⇒ 1 2 3 4 5 6 7 8 - extension method
- O(n)
source.Any(6) true
source.Any(9) false

- الـ **Complexity**: في أسوأ الحالات (Worst Case) بتكون $O(n)$ ، وده لو العنصر اللي بندور عليه كان في آخر القائمة، فبالتالي هتضطر تزور كل العناصر.
- هي عبارة عن Extension Method موجودة في **IEnumerable**.

```
public static bool Any<TSource>(this IEnumerable<TSource> source);  
... public static bool Any<TSource>(this IEnumerable<TSource> source, Func<TSource, bool> predicate);  
... public static IEnumerable<TSource> Append<TSource>(this IEnumerable<TSource> source, TSource element);  
... public static IEnumerable<TSource> AsEnumerable<TSource>(this IEnumerable<TSource> source);  
... public static double Average<TSource>(this IEnumerable<TSource> source, Func<TSource, int> selector);  
... public static double Average<TSource>(this IEnumerable<TSource> source, Func<TSource, long> selector);  
... public static decimal? Average<TSource>(this IEnumerable<TSource> source, Func<TSource, decimal?> selector);
```



مثال 1: البحث عن اسم يبدأ بـ "Jack"

لو طلب منك تشوف هل فيه أي موظف اسمه ببدا بـ "Jack"؟

C#

```
1 // الكود لاستخدام Any
2 var result1 = employees.Any(emp => emp.Name.StartsWith("Jack",
  StringComparison.InvariantCultureIgnoreCase));
3
4 Console.WriteLine($"Any Jack? {result1}");
```

• شرح الكود:

- استخدمنا Any وبعتنا ليها الـ Predicate (الشرط).
- الشرط: emp.Name.StartsWith("Jack") .
- تركاية مهمة: لو الاسم في الداتا "jack" (سمول) وأنا ببحث عن "Jack" (كابيتال)، مش هيجيبه لأن #C حساسة للحروف (Case Sensitive). عشان كدة زدونا StringComparison.InvariantCultureIgnoreCase عشان يتجاهل حالة الأحرف (Capital or Small).
- النتيجة: True لأن فعلاً عندنا موظف اسمه Jack.

مثال 2: البحث عن راتب أقل من 10,000

لو السؤال اتغير: "هات لي هل فيه موظفين رواتبهم أقل من 10,000؟".

C#

```
1 var result2 = employees.Any(emp => emp.Salary < 10_000);
2 Console.WriteLine($"Any salary < 10000? {result2}");
```

- غيرنا الشرط لـ Salary < 10000 ، والنتيجة هتكون True لو فيه موظف واحد على الأقل بيحقق الشرط.
- لاحظ 10_000 هي بالضبط 10000 . الكومبايلر بيتجاهل الـ _ تماماً وقت التشغيل، يعني دي حاجة "شكلية" بس عشان المبرمج عينه ترتاح وهو بيقرا الكود.

مثال 3: البحث داخل لسته فرعياً (Skills)

لو عاوزين نعرف: "هل فيه أي موظف عنده مهارة واحدة (Skill) فقط؟".

C#

```

1 var result3 = employees.Any(emp => emp.Skills.Count == 1);
2 Console.WriteLine($"Any employee with 1 skill? {result3}");

```

- هنا دخلنا جوه لسته الـ Skills لكل موظف وشيكنا على العدد.

2. دالة All

دالة All بتشوف هل كل العناصر بتحقق الشرط ولا لأ.

- لازم تلف عليهم كلهم عشان تتأكد، ولو لقت عنصر واحد بس مش بيحقق الشرط، بترجع False فوراً.
- الـ **Complexity**: أكيد $O(n)$ لأنها غالباً محتاجة تزور الكل.

مثال 1: التأكد من وجود إيميل لكل الموظفين

السؤال: "هل كل الموظفين عندهم إيميل (مش فاضي)؟".

C#

```

1 var resultAll = employees.All(x => !string.IsNullOrEmpty(x.Email));
2
3 Console.WriteLine($"All employees have email? {resultAll}");

```

- استخدمنا `string.IsNullOrEmpty` عشان نتأكد إن الإيميل مش null ومش مسافة فاضية.
- لو فيه موظف واحد الإيميل بتاعه فاضي، النتيجة هتكون False.

مثال 2: هل كل الموظفين عندهم مهارة C#؟

عاوز أشيك هل كل الموظفين اللي عندي في الشركة يعرفوا C#؟.

C#

```

1 // عشان ندور جوه لسته المهارات Any وجواها All بنستخدم
2 var allHasCSharp = employees.All(emp => emp.Skills.Any(s => s == "C#"));
3
4 Console.WriteLine($"All have C#? {allHasCSharp}");

```

- هنا بنقول: لكل موظف (All)، ادخل جوه مهاراته وشوف هل فيه أي مهارة (Any) تساوي "C#".
- النتيجة هتكون False لو فيه موظف واحد ماعندوش المهارة دي.

3. الدمج مع Query Syntax

إحنا شوفنا الـ Method Syntax، تعالوا نشوف إزاي نستخدم الـ Quantifiers مع الـ Query Syntax.

مثال: موظفين عندهم مهارة ++C

عاوزين نجيب كل الموظفين اللي عندهم مهارة ++C.

C#

```
1 var query1 = from emp in employees
2               where emp.Skills.Any(s => s == "C++")
3               select emp;
4
5 // دالة طباعة موجودة مسبقاً في المشروع
6 query1.Print("Employees with C++ skill");
```

- هنا استخدمنا Any جوه الـ where عشان نفلتر الموظفين.

مثال متقدم مع All

عاوز أجيب الموظفين اللي كل مهاراتهم (Skills) عدد حروفها أكبر من أو يساوي 328.

- يعني لو واحد عنده مهارة "#C" (حرفين) مش هيجي معانا، لازم كل مهاراته تكون طويلة (زي (SQL, etc, OOP).

C#

```
1 var query2 = from emp in employees
2               where emp.Skills.All(s => s.Length >= 3)
3               select emp;
4
5 query2.Print("Employees with all skills length >= 3");
```

- لاحظ إن الموظفين اللي عندهم "#C" ما ظهورش في النتيجة لأنها حرفين بس.

4. دالة Contains (والمشاكل بتاعتها)

الـ Contains دالة خاصة شوية (Special)، وليها حالتين:

1. الـ **Instance Method**: زي اللي في الـ String أو HashSet ، دي ممكن تكون سريعة جداً $O(1)$ في الـ HashMaps.
2. الـ **Extension Method**: زي اللي في الـ List ، دي بتمشي $O(n)$ في أسوأ الظروف.

source ⇒ 1 2 3 4 5 6 7 8

- instance method
- $O(n)$ List
- $O(1)$ HashSet

source.Contains(6) true
source.Contains(9) false

```
... public bool Contains(char value);
... public bool Contains(String value);
... public void CopyTo(int sourceIndex, char[] destination, int destinationIndex, int count);
... public bool EndsWith(char value);
... public bool EndsWith(String value);
... public bool EndsWith(String value, bool ignoreCase, CultureInfo? culture);
... public bool EndsWith(String value, StringComparison comparisonType);
```

مثال بسيط: سترينج (String)

C#

```
1 // وسريعة Instance Method دي
2 bool hasDoubleE = employee.Name.Contains("ee");
```

المشكلة الكبيرة: البحث عن Object (Reference Type)

لو جيت دورت على موظف كامل جوه الليسته، هتقابل مشكلة الـ **Reference Equality**.

تخيل عملنا موظف جديد بنفس بيانات موظف موجود في الليسته (نفس الإيميل والبيانات) وسألنا الليسته: "هل الموظف ده عندك؟"

C#

```
1 var empToCheck = new Employee { Email = "jack@example.com" }; // نفس بيانات جاك
2 var resultContains = employees.Contains(empToCheck);
3
4 Console.WriteLine($"Contains Jack? {resultContains}"); // False !! هتطلع
```

ليه النتيجة False؟

- لأن Employee ده **Reference Type**.
- الكمبيوتر بيقارن مكانهم في الـ **References**، مش القيم اللي جواهرم عن طريق فنكشن Equals الى بيورثها أى كلاس من كلاس Object.
- حتى لو كل البيانات متطابقة، هما كائنات مختلفين في الذاكرة.

الحل: Override Equals & GetHashCode

عشان نحل المشكلة دي ونخلي المقارنة تتم بناءً على "الإيميل" مثلاً، لازم نعمل **Override** لدالة Equals جوه كلاس Employee.

خطوات الحل:

1. نروح لكلاس Employee.
2. نعمل Override لـ Equals.

3. نعمل Override لـ GetHashCode (مهم جداً بشأن الـ Hash-based collections).

الكود داخل كلاس Employee:

C#

```
1 public override bool Equals(object obj)
2 {
3     // false أو التايب مختلف رجع Employee لو الأوبجكت مش من نوع 1.
4     if (obj == null || this.GetType() != obj.GetType())
5         return false;
6
7     // Employee حول الأوبجكت لـ 2.
8     Employee other = (Employee)obj;
9
10    // قارن بناءً على الإيميل (أو أي مفتاح مميز) 3.
11    return this.Email == other.Email;
12 }
13
14 public override int GetHashCode()
15 {
16     // GetHashCode معادلة بسيطة بشأن نضمن تفرد الـ
17     // ID بتاع الإيميل ورقم عشوائي أو الـ GetHashCode بنستخدم الـ
18     return this.Email.GetHashCode() * 12345;
19 }
```

النتيجة بعد التعديل:

لو شغلت نفس كود Contains تاني، النتيجة هتكون True، لأن المقارنة بقت بتتم على قيمة الإيميل مش عنوان الميموري.

ملخص سريع:

- الـ **Any**: بترجع True لو عنصر واحد على الأقل حقق الشرط (Lazy & Short-circuit).
- الـ **All**: بترجع True لو كل العناصر حققت الشرط.
- الـ **Contains**: بتدور على عنصر بعينه، ولازم تاخد بالك من الـ Reference Types وتعمل Override لـ Equals لو بتقارن Objects خاصة بيك.

Grouping Data (GroupBy, ToLookup)

مقدمة: يعني إيه Grouping؟

في البداية، خلونا نفهم يعني إيه **Grouping**. العملية دي ببساطة إنني باخد البيانات وأصنفها جوه مجموعات (Groups)، وكل مجموعة بتحتوي على عدة عناصر.

- **مثال عملي:** لو عندي داتا للمشتريات، وجيت قلتلك "جمع لي المشتريات دي بناءً على كل شهر". يعني مشتريات شهر فبراير لوحدها، ومشتريات شهر مارس لوحدها، وهكذا.
- **الأهمية:** الموضوع ده بيضيف "Business Value" كبيرة لليوزر ومهم جداً في عرض التقارير.

في الحلقة دي، هنشرح Methods في الـ LINQ بيعملوا القصة دي:

1. الـ GroupBy

2. الـ ToLookup

وهنعرف الفرق الجوهرى بينهم اللي بيتسأل كثير في الإنترنت فيوهات.

تجهيز المشروع (Project Setup)

قبل ما نبدأ كود، المشروع بتاعنا عبارة عن Console Application مبني على .NET 5، وفيه الملفات الأساسية دي:

- الـ **Employee Class**: فيه بيانات الموظفين (الاسم، القسم، الخ..).
- الـ **Repository**: كلاس فيه داتا وهمية (Dummy Data) لعدد من الموظفين.
- الـ **Extension Methods**: ميثودز جاهزة عشان نطبع البيانات ونعرضها بشكل List عشان نركز في الشرح على طول.

أولاً: استخدام Group By (Method Syntax)

دلوقتي عندنا لسته موظفين Employees، وكل موظف تابع لقسم (Department) معين (زي HR, IT, Finance). أنا معرفش عدد الموظفين في كل قسم قد إيه، وعمايز أجمعهم بناءً على القسم اللي بيشتغلوا فيه.

1. كتابة الكود

عشان نعمل ده بـ Method Syntax، بنستخدم GroupBy:

C#

```
1 // 1. تحميل الموظفين من الـ Repository
2 var employees = repository.LoadEmployees();
3
4 // 2. بناءً على الـ Grouping عمل
```



```

5 // Department (اللي هو) اللي هنجمع على أساسه Key عشان نحدد الـ Lambda Expression بنستخدم
6 var groups = employees.GroupBy(emp => emp.Department);

```

2. تحليل الـ Return Type

لو وقفت على GroupBy ودوست F12 عشان تشوف الـ Definition بتاعها، هتلاقيها بترجع IEnumerable :

```

1 IEnumerable <IGrouping> <TKey, TSource>

```

يعني إيه الكلام ده؟

- الناتج عبارة عن IEnumerable (يعني نقدر نعمل عليه Loop).
- كل عنصر جوه الـ IEnumerable ده هو عبارة عن IGrouping .

الـ IGrouping تعريفه انه هوا Interface وارث من IEnumerable وبيكون من حاجتين:

```

1 public interface IGrouping<TKey, TElement> : IEnumerable<TElement>
2 {
3     // الخاصية الوحيدة الزيادة
4     TKey Key { get; }
5 }

```

- الـ TKey : ده نوع المفتاح (في مثالنا كان String اللي هو اسم القسم).
- الـ TElement : ده نوع العنصر اللي جوه القائمة (في مثالنا كان Employee).

بس لازم تبقى فاهم انه **Represents a collection of objects that have a common key** لانه وارث من IEnumerable فبيتعامل انه Collection وتلف عليه بـ Foreach عادى.

ملخص سريع

الـ IGrouping = قيمة (Key) + قائمة (List) مدمجين في كائن واحد.

- لما تعوز تعرف اسم الجروب -> استخدم .Key
- لما تعوز تجيب العناصر اللي جوه الجروب -> استخدم المتغير نفسه كأنه List.

3. طباعة النتيجة (Iteration)

عشان نعرض البيانات، هنعمل foreach على المجموعات دي.

- كل group جواه خاصية اسمها .Key (اسم القسم).
- وكل group نفسه بيعتبر List من الموظفين، فنقدر نبعته للـ Extension Method اللي بتطبع، أو نعمل عليه Loop تاني داخلي.

C#

```

1 عشان تبقى فاهم الـ إيتيريشن بيمر على إيه var انا مستخدمتش //

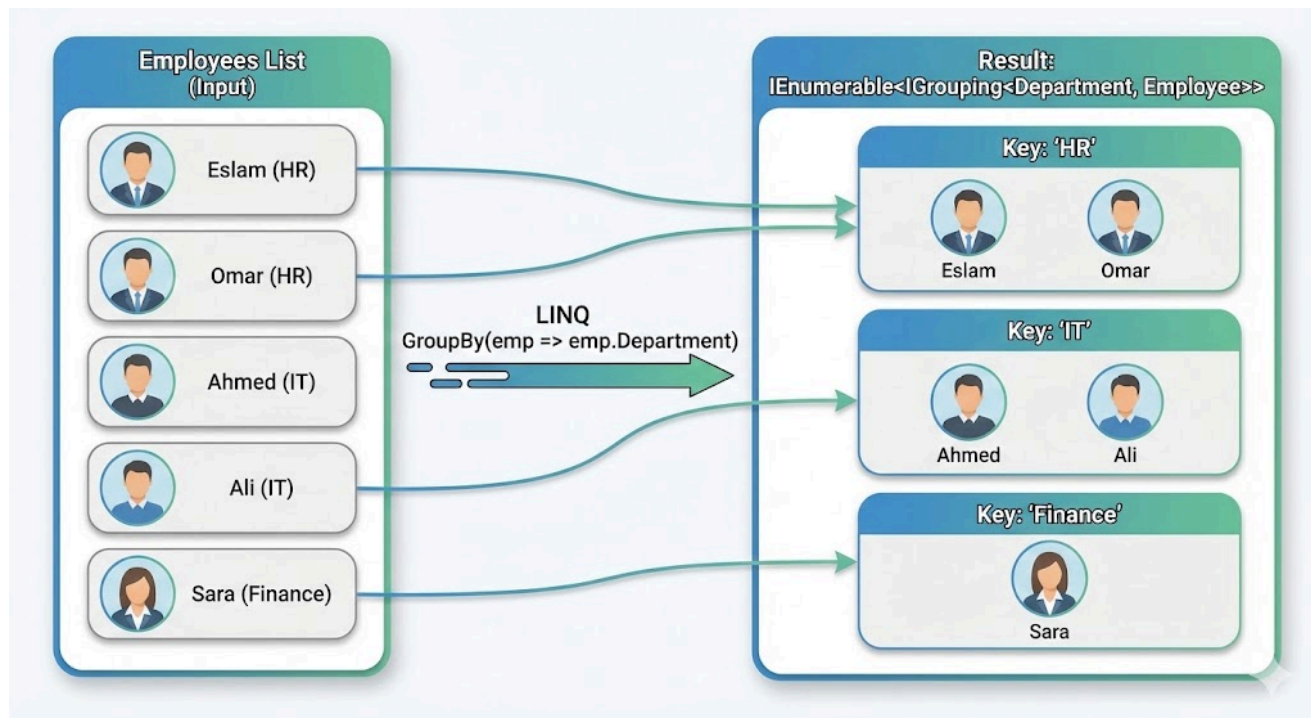
```

```

2  foreach (IGrouping<string,Employee> group in groups)
3  {
4      // طباعة اسم القسم (المفتاح) وعنوان للطباعة
5      Console.WriteLine($"Department: {group.Key}");
6
7      // طباعة الموظفين اللي جوه الجروب ده
8      // List of Employees نفسه بيشتغل كـ IGrouping الـ
9      group.Print();
10 }

```

النتيجة: هيطبعلك كل قسم وتحتة أسماء الموظفين الشغالين فيه (Finance كلهم مع بعض، IT كلهم مع بعض.. وهكذا).



ثانياً: استخدام Group By (Query Syntax)

لو أنت بتفضل طريقة الـ Query Syntax (شبه الـ SQL)، تقدر تعمل نفس النتيجة بالطبط12.

[تذكير](#)

لو مش عارف الفرق بين Query Syntax و Method Syntax، ارجع لحلقة سابقة شرحنا فيها الفرق بالتفصيل.

الكود بـ Query Syntax:

C#

```

1  var groups = from emp in employees
2              group emp by emp.Department;

```

وبعد كده بتعمل الـ `foreach` والطباعة بنفس الطريقة العادية جداً، وهيديك نفس النتيجة بالطبط.

ثالثاً: مفهوم Deferred Execution (التنفيذ المؤجل)

نقطة مهمة جداً لازم تاخذ بالك منها في `GroupBy`:

الـ `GroupBy` بترجع `IEnumerable`، وده معناه إنها بتستخدم `Deferred Execution`.

يعني إيه؟

يعني سطر الكود اللي كتبنا فيه `GroupBy` ده مجرد "وصف" أو "اتفاق" (`Contract`) للي هيحصل، لكن التنفيذ الفعلي مش بيحصل في السطر ده.

```
1 var groups = employees.GroupBy(emp => emp.Department);
```

التنفيذ بيحصل فقط لما تبدأ تعمل `Iteration` (لما اللوب تشتغل).

⚠ خلي بالك

لو عندك 100 سطر كود بعد جملة الـ `GroupBy` وقبل الـ `Foreach`، عملية التجميع مش هنتم غير لما توصل للـ `Foreach`.

رابعاً: استخدام ToLookup

نيجي بقى للطريقة الثانية، وهي `ToLookup`.

استخدامها في الكود سهل جداً، مجرد بنشيل `GroupBy` ونحط `ToLookup`.

C#

```
1 // نفس الكود بس غيرنا اسم الميثود
2 var groups = employees.ToLookup(emp => emp.Department);
3
4 // الطباعة بنفس الطريقة مفيش أي اختلاف
5 foreach (var group in groups)
6 {
7     Console.WriteLine($"Department: {group.Key}");
8     group.Print();
9 }
```

خامساً: الفرق الجوهرى (سؤال إنترفيو) 💡

طب طالما الاتنين بيطلعوا نفس النتيجة، إيه الفرق؟ ولية عندي 2 Methods بيعملوا نفس الحاجة؟

ده سؤال مشهور جداً في المقابلات (Interview Question) لأنه بيقيس فهمك لـ 3 حاجات:

1. مهارة الـ **Deferred Execution vs Immediate Execution**.

2. مهارة فهم الـ **Grouping of Data**.

3. إزاي تعمل **Processing** للداتا بأحسن طريقة (Performance).

الجدول ده يلخص الفرق اللي المهندس عصام شرحه:

ToLookup	GroupBy	وجه المقارنة
Immediate Execution (فوري)	Deferred Execution (مؤجل)	نوع التنفيذ
بيخزن النتيجة في الذاكرة (Buffer in Memory).	لا يتم التخزين، بيعيد التنفيذ مع كل Loop.	التخزين في الذاكرة
<code>ILookup<TKey, TSource></code>	<code>IEnumerable<IGrouping<...>></code>	نوع الإرجاع (Return Type)
قائمة (List) جواها مجموعات (نفس الشكل النهائي).	قائمة (List) جواها مجموعات.	الشكل الناتج
لو هتعمل أكثر من عملية (Multiple Processes) على نفس الداتا المجمعة.	لو هتعمل Loop مرة واحدة بس.	سيناريو الاستخدام الأمثل

تفصيل الفرق:

1. الـ **GroupBy**:

- بما إنها Deferred، لو عملت `foreach` على النتيجة مرتين، الـ LINQ هيروح يعمل عملية الـ Grouping من أول وجديد في المرة الثانية!.

2. الـ **ToLookup**:

- بمجرد ما السطر ده بيتكتب، الكود بيتنفذ فوراً والنتيجة بتتخزن في الميموري (Caching/Buffering).
- ده مفيد جداً لو عندك "Multiple Processes" (مثلاً هتعمل Cycle 1 وتعديل حاجة، وبعدين Cycle 2 تعمل حاجة ثانية على نفس التجميع). هنا مش منطقي إنه يروح يجمع الداتا كل مرة، فالأحسن نستخدم `ToLookup` عشان نعملها مرة واحدة ونحطها في الميموري.

الخلاصة: أستخدم مين وإمتى؟

الجواب بيعتمد على "أنت عايز تعمل إيه بالداتا؟":

- استخدم **GroupBy**  :
 - لو هتعمل عملية واحدة (Single Process) زي إنك تطبع الليسته وخلص.
 - مش محتاج تعمل Loops كتير على نفس التجميعية.
- استخدم **ToLookup**  :
 - لو هتعمل معالجات متعددة (Multiple Processes) على الداتا.
 - عايز تعمل Buffer للداتا في الميموري عشان توفر وقت المعالجة بدل ما تعيد التجميع كل شوية.

Join Operations(Join, Group Join)

1. مقدمة: يعني إيه Join؟

في الحلقة دي بنتكلم عن موضوع مهم جداً وهو الـ **Data Joining**.

- **الفكرة:** عملية الـ Join هي ربط بين مصدرين مختلفين للبيانات (Two different data sources) أو أكثر، موجودين في أماكن منفصلة.
- **الهدف:** إننا نعرض لليوزر (Client) بيانات مفهومة ومفيدة (Informative) بدلاً من عرض أرقام مبهمه.
- **طريقة الربط:** الربط بيتتم عن طريق **Common Attribute** (عامل مشترك) موجود في المصدرين.

في الـ LINQ، عندنا دالتين (Methods) أساسيين للعملية دي:

1. الـ Join

2. الـ GroupJoin هنتعرف عليهم بالتفصيل.

2. تجهيز البيانات (Data Setup)

قبل ما نبدأ، المهندس عمل تعديل بسيط على الـ Project القديم عشان يخدم فكرة الـ Join:

- **تعديل كلاس الـ Employee:** زمان كان القسم (Department) عبارة عن string ، دلوقتي حولناه لرقم DepartmentId .
- **إضافة كلاس الـ Department:** بقى عندنا كلاس منفصل للأقسام فيه Id و Name .
- **الـ Repository:** بقى فيه دالتين:
 - الـ LoadDepartments () : بترجع لسته فيها الأقسام (مثلاً: 2=IT, 1=Finance).
 - الـ LoadEmployees () : بترجع الموظفين، وكل موظف مربوط برقم القسم بتاعه.

الهدف: عايزين نربط الموظفين بالأقسام بناءً على العامل المشترك اللي هو Id .

3. أولاً: الـ Join العادية (Inner Join)

هنا إحنا عايزين نربط الموظف باسم القسم بتاعه. عندنا طريقتين للكتابة: **Method Syntax** و **Query Syntax**.

أ) طريقة الـ Method Syntax

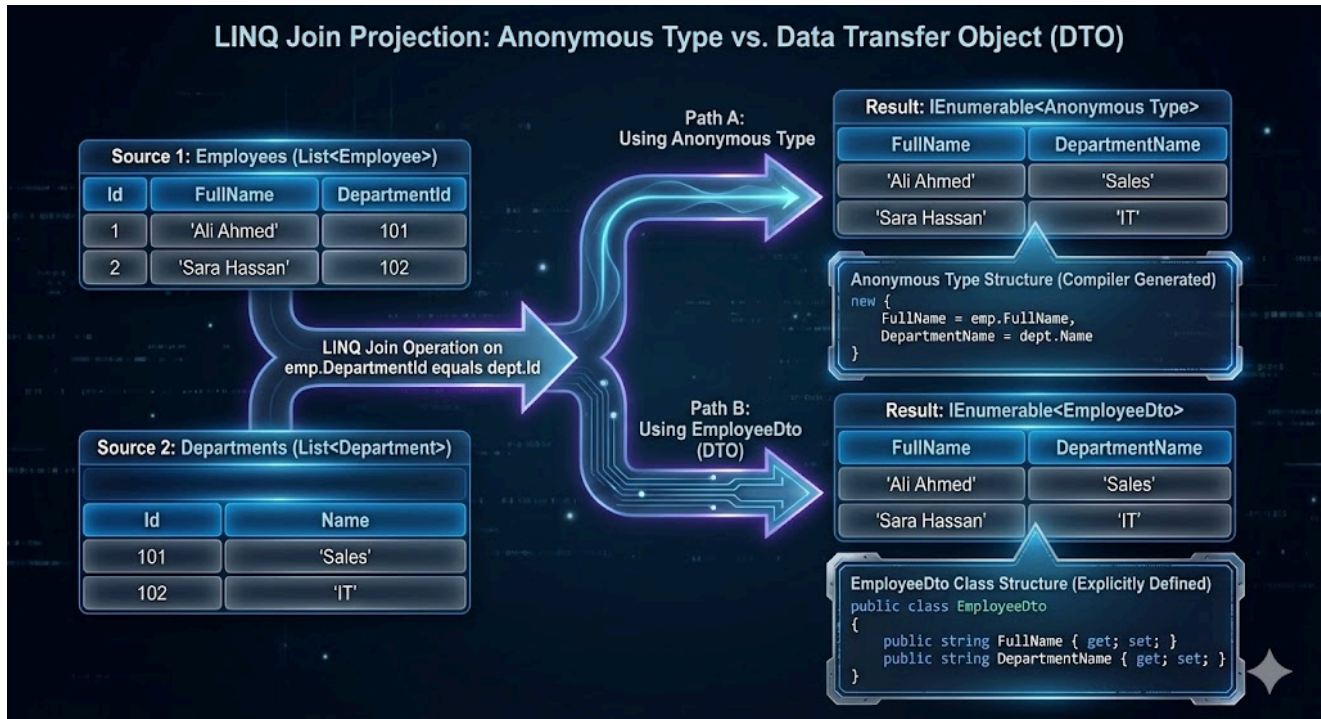
دي الطريقة اللي بنستخدم فيها الـ Extension Methods. الـ Join هنا بتاخد 4 بارامترات (Parameters) ولازم نفهمهم كويس.

```
...public static IEnumerable<TResult> Join<TOuter, TInner, TKey, TResult>(this IEnumerable<TOuter> outer, IEnumerable<TInner> inner, Func<TOuter, TInner, TKey> keySelector, Func<TOuter, TInner, TResult> resultSelector)  
...public static IEnumerable<TResult> Join<TOuter, TInner, TKey, TResult>(this IEnumerable<TOuter> outer, IEnumerable<TInner> inner, Func<TOuter, TInner, TKey> keySelector, Func<TOuter, TInner, TResult> resultSelector)  
...public static TSource Last<TSource>(this IEnumerable<TSource> source, Func<TSource, bool> predicate);  
...public static TSource Last<TSource>(this IEnumerable<TSource> source);
```

الـ 4 بارامترات هما:

1. الـ **Outer Sequence**: المصدر الأول (الأساسي) للبيانات (مثلاً الموظفين employees).
2. الـ **Inner Sequence**: المصدر الثاني الذي يربط معاه (الأقسام departments).
3. الـ **Outer Key**: المفتاح المشترك من المصدر الأول (emp.DepartmentId).
4. الـ **Inner Key**: المفتاح المشترك من المصدر الثاني (dept.Id).
5. الـ **Result Selector**: شكل النتيجة النهائية التي هترجع (دمجناهم في كلاس جديد أو Anonymous Object).

كلاس للمساعدة (DTO): عشان النتيجة تكون منظمة، عملنا كلاس بسيط اسمه EmployeeDto عشان نخزن فيه النتيجة بدل ما نستخدم Anonymous Object.



C#

```
1 public class EmployeeDto
2 {
3     public string FullName { get; set; }
4     public string DepartmentName { get; set; }
5 }
```

كود الـ Join بـ **Method Syntax**:

C#

```
1 // 1. Load Data Sources
2 var employees = Repository.LoadEmployees();
3 [cite_start]var departments = Repository.LoadDepartments(); [cite: 4, 5]
4
5 // 2. Perform Join
6 var result = employees.Join(
7     departments, // Inner Sequence (اللي يربط معاه)
8     emp => emp.DepartmentId, // Outer Key Selector (المفتاح من الموظفين)
```

```

9      dept => dept.Id, // Inner Key Selector (المفتاح من الأقسام)
10     (emp, dept) => new EmployeeDto // Result Selector (شكل النتيجة)
11     {
12         FullName = emp.FullName,
13         DepartmentName = dept.Name
14     }
15 };
16
17 // 3. Print Result
18 foreach (var item in result)
19 {
20     Console.WriteLine($"{item.FullName} - {item.DepartmentName}");
21 }

```

ملاحظة: الترتيب هنا بدأ بالموظف (Outer) وربطنا معه القسم (Inner).

ب) طريقة الـ Query Syntax (الأسهل)

في عمليات الـ Join، الـ Query Syntax ستكون أسهل بكثير وأكثر قراءة من الـ Method Syntax.

الشكل العام:

```

from [outer_element] in [outer_source]
join [inner_element] in [inner_source]
on [outer_key] equals [inner_key]
select [result]

```

كود الـ Join بـ Query Syntax:

C#

```

1  var queryResult = from emp in employees
2                      join dept in departments
3                      on emp.DepartmentId equals dept.Id // الربط هنا
4                      select new EmployeeDto
5                      {
6                          FullName = emp.FullName,
7                          DepartmentName = dept.Name
8                      };
9
10 // النتيجة ستكون هي هي بالظبط زي الطريقة اللي فاتت

```

خد بالك: كلمة `equals` هي المستخدمة في الربط مش `==`.

4. ثانياً: الـ Group Join

النوع ده بيعمل خطوة زيادة عن الـ Join العادية. هو بيربط المصدرين ببعض، بس يجمع النتائج في مجموعات (Groups). يعني مثلاً: عايز أجيب القسم، وجواه لسته بكل الموظفين اللي شغالين فيه.

أ) طريقة الـ Method Syntax

هنا هنعكس الآلية، هنبدأ بـ الأقسام كـ Outer Source عشان نجمع جواها الموظفين.

البارامترات:

1. الـ **Outer**: الأقسام (departments).
 2. الـ **Inner**: الموظفين (employees).
 3. الـ **Outer Key**: dept.Id.
 4. الـ **Inner Key**: emp.DepartmentId.
 5. الـ **Result Selector**: هنا النتيجة هتكون (القسم الواحد، ومجموعة الموظفين المرتبطين بيه).
- كلاس للمساعدة (DepartmentGroup): هنعمل كلاس عشان نعرض فيه شكل الجروب النهائي.

C#

```
1 public class DepartmentGroup
2 {
3     public string Department { get; set; }
4     public List<string> Employees { get; set; } // لسته أسماء الموظفين
5 }
```

كود الـ GroupJoin بـ Method Syntax:

C#

```
1 var groupJoinResult = departments.GroupJoin(
2     employees, // Inner Sequence
3     dept => dept.Id, // Outer Key (Department ID)
4     emp => emp.DepartmentId, // Inner Key (Employee's Dept ID)
5     (dept, emps) => new DepartmentGroup // Result Selector
6     {
7         Department = dept.Name,
8         // عشان ناخذ الأسماء بس Select هنا عبارة عن لسته موظفين، هنعمل emps
9         Employees = emps.Select(e => e.FullName).ToList()
10    }
11 );
```

طباعة النتيجة (Nested Loops): بما إن النتيجة عبارة عن مجموعات، هنحتاج 2 loop عشان نطبع.

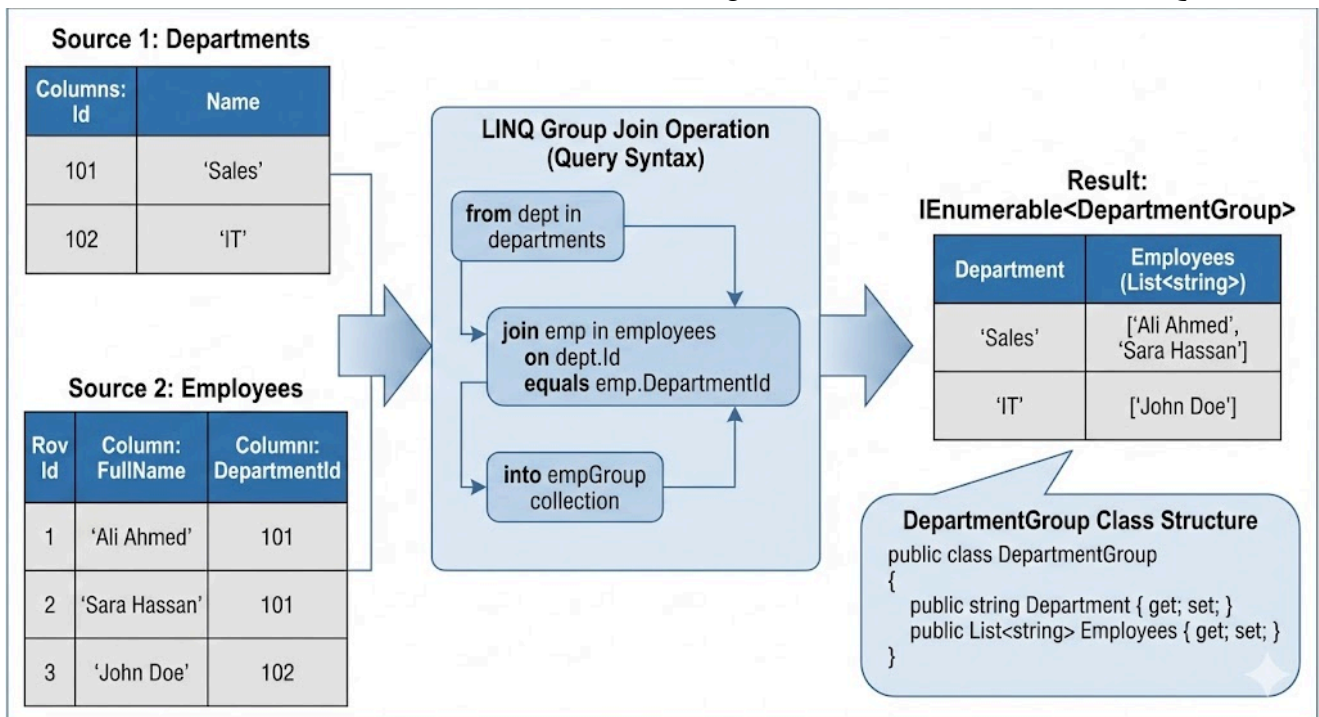
C#

```

1  foreach (var group in groupJoinResult)
2  {
3      // طباعة اسم القسم (الجروب)
4      Console.WriteLine($"Group: {group.Department}");
5
6      // طباعة الموظفين جوه القسم ده
7      foreach (var name in group.Employees)
8      {
9          Console.WriteLine($"    - {name}");
10     }
11 }

```

النتيجة: هيطلع "Sales" وتحتها كل الموظفين بتوعها، "Accounting" وتحتها موظفيها، وهكذا.



Group Join الـ Query Syntax طريقة الـ

دي كمان أسهل شوية، والسر كله في كلمة **into**.

الشكل العام: بعد ما تعمل **join** عادي، بتزود كلمة **into** وتدي اسم للمجموعة (مثلاً **empGroup**). وبكده **empGroup** دي بقت شايلة لسته الموظفين اللي تبع القسم ده.

كود الـ GroupJoin بـ Query Syntax:

C#

```

1  static void Main(string[] args)
2  {
3
4      var Employees = Repository.LoadEmployees();

```

```

5     var Departments = Repository.LoadDepartment();
6
7     var JoinGroupResult = from dep in Departments
8                           join emp in Employees
9                           on dep.Id equals emp.DepartmentId
10                          into GroupResult
11                          select GroupResult;
12
13 }
14
15 // اللي فوق foreach الطباعة بنفس طريقة الـ

```

في الحالة دي إحنا ما عملناش Object معقد، إحنا قولنا له اعمل Grouping والنتيجة رجعت كمجموعة من المجموعات.

طيب لو حبيت تعمل كلاس DTO وتأخذ فيه الـ Result هتعمل كده

```

1  static void Main(string[] args)
2  {
3
4      var Employees = Repository.LoadEmployees();
5      var Departments = Repository.LoadDepartment();
6
7      var JoinGroupResult = from dep in Departments
8                            join emp in Employees
9                            on dep.Id equals emp.DepartmentId
10                           into GroupResult
11                           select new empGroupDto()
12                           {
13                               DepartmentName = dep.Name,
14                               Emps = GroupResult.Select(x =>
15                                   x.FullName).ToList();
16
17                           };
18
19      foreach (var SingleGroup in JoinGroupResult)
20      {
21          Console.WriteLine(SingleGroup.DepartmentName);
22
23          foreach (var EmployeeNames in SingleGroup.Emps)
24          {
25              Console.WriteLine(EmployeeNames);
26          }
27      }
28  }

```

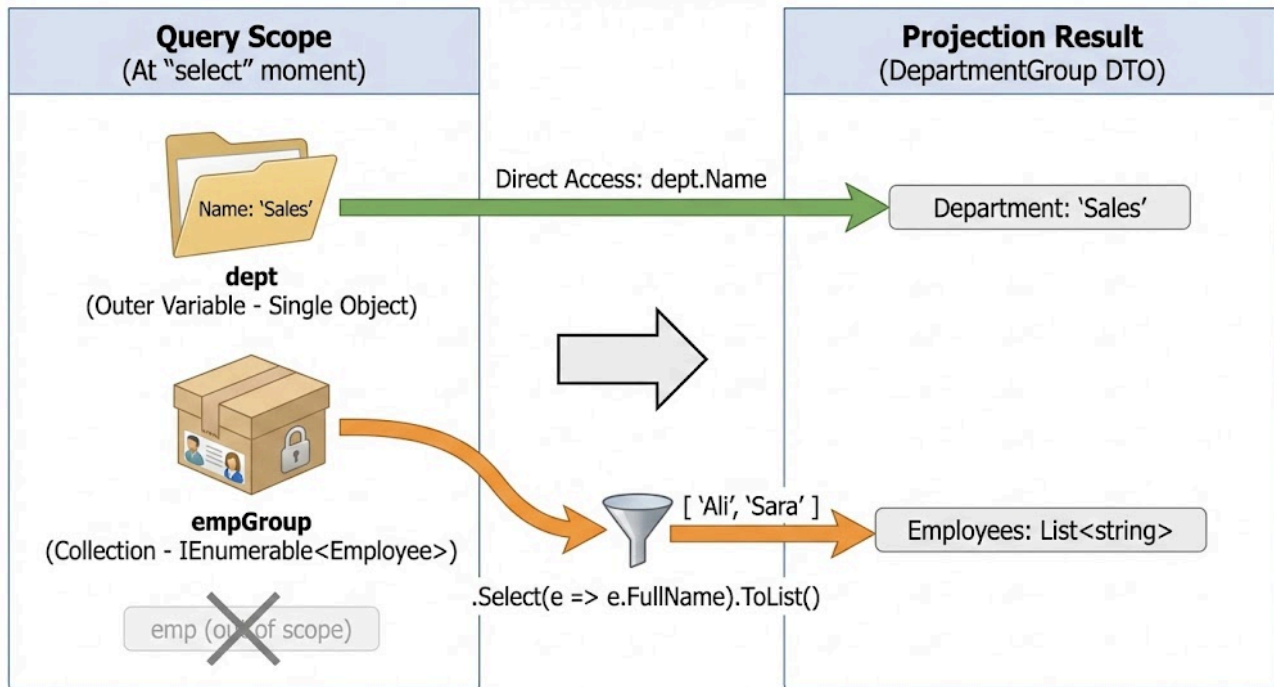
توضيح للسطر ده:

```

1  DepartmentName = dep.Name,

```

2 Emps = empGroup.Select(e => e.FullName).ToList()



الـ Technical Analysis: dept vs empGroup

- الـ **Iteration Scope (نطاق التكرار):**
 - الـ **dept** : يمثل **Outer Iteration Variable**. يظل متاحاً (In Scope) لأن الاستعلام يدور حوله، مما يسمح بالوصول المباشر لخصائصه.
 - الـ **emp** : يخرج من النطاق (**Out of Scope**) فور استخدام كلمة **into**. يتم استبداله كلياً بالكائن المجموع **empGroup**.
- الـ **Data Structure (هيكلية البيانات):**
 - الـ **dept** : كائن مفرد (**Single Entity**). يمتلك خصائص مباشرة مثل **Name**.
 - الـ **empGroup** : حاوية بيانات (**Collection / IEnumerable**). لا تمتلك خصائص الموظف، بل تحتوي على كائنات الموظفين بداخلها.
- الـ **Projection Mechanism (آلية الإسقاط):**
 - لِلوصول لاسم القسم: نستخدم (**dept.Name**) **Direct Access**.
 - لِلوصول لأسماء الموظفين: يجب استخدام **Transformation/Projection** عبر دالة **Select** لاستخراج البيانات من داخل الـ **Collection** وتحويلها لقائمة.

5. الخلاصة (Summary)

- الـ **Join**: يستخدم لربط مصدرين بيانات (**Two Data Sources**) بناءً على علاقة بينهم (**Matching Key**) لعرض بيانات مفيدة.
- الـ **Method Syntax**: قوية بس الـ **Syntax** بتاعها في الـ **Join** ممكن يكون معقد شوية وطويل.
- الـ **Query Syntax**: أسهل بكثير في القراءة والكتابة خصوصاً مع الـ **Joins**.

- **الـ Group Join:** بتنفيذ لما تكون عايز تربط وتجمع البيانات في نفس الوقت (Master-Detail relationship) زي قسم وتحتة موظفين.

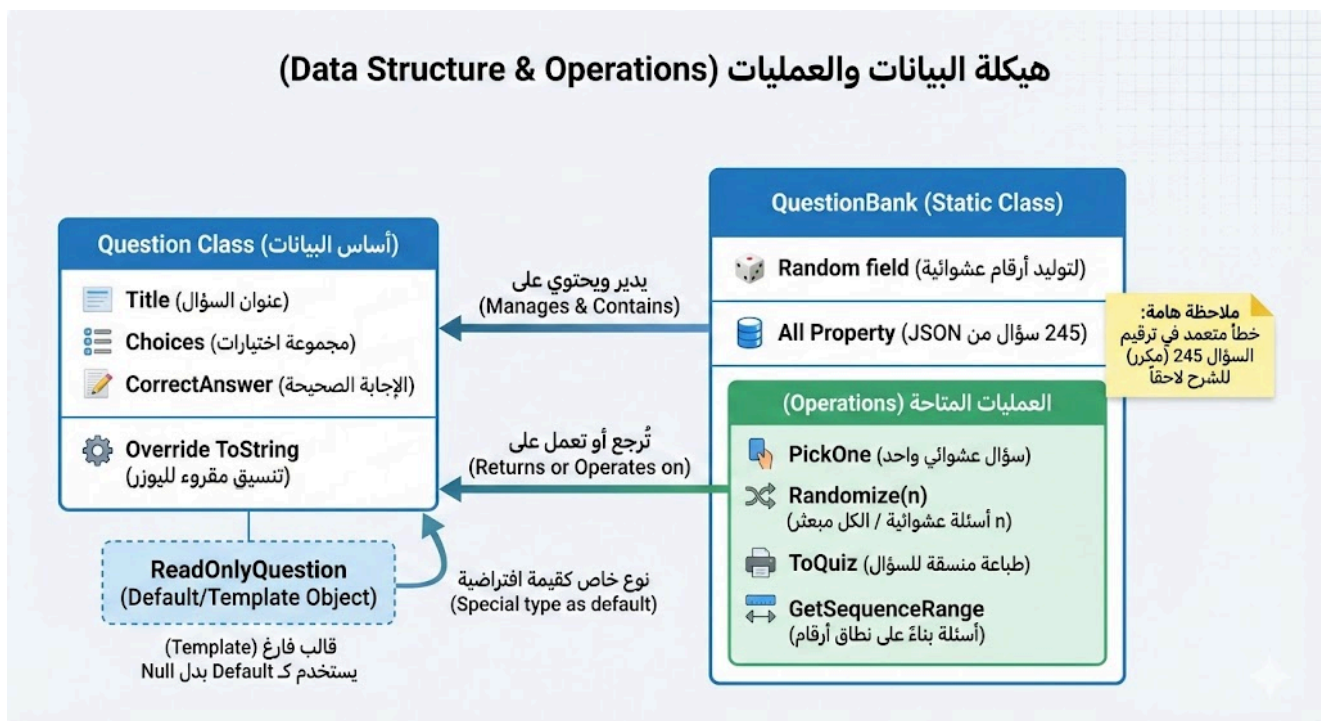
Generation Operations(Empty, DefaultIfEmpty, Range, Repeat)

1. مقدمة وتجهيز المشروع (Project Setup)

قبل ما ندخل في الكود، لازم نفهم الـ Setup اللي هنشتغل عليه، لأننا بنبني على كود موجود مسبقاً في Shared Project .

هيكلية البيانات (Data Structure)

- الـ **Question Class**: كل سؤال عندنا له عنوان (Title)، ومجموعة اختيارات (Choices)، وإجابة صحيحة (CorrectAnswer).
- الـ **Default/Template Object**: عندنا حاجة اسمها `ReadOnlyQuestion`، وده عبارة عن "قالب" (Template) للسؤال وليس سؤال حقيقي، بنستخدمه كـ Default Object بدل ما نرجع Null، ويحتوي على أماكن فاضية للإجابات.
- الـ **Override ToString**: تم عمل `Override` عشان نطبع السؤال بشكل مقروء لليوزر.
- الـ **QuestionBank (Static Class)**: ده بنك الأسئلة اللي بنعمل عليه العمليات:
 - بيحتوي على `Random field` عشان نولد أرقام عشوائية.
 - الـ **All Property**: بترجع 245 سؤال (موجودين كـ JSON في الأصل).
 - ملاحظة هامة**: المهندس تعمد وجود خطأ في ترقيم الأسئلة، السؤال رقم 245 متكرر مرتين (كسلسلة نصية) عشان هيتحتاج الخطأ ده قدام في الشرح.



العمليات المتاحة في الـ QuestionBank

- الـ `PickOne`: بتجيب سؤال عشوائي واحد (من 0 لـ 245).
- الـ `Randomize(int n)`: بتجيب عدد n من الأسئلة بشكل عشوائي. لو طلبت عدد أكبر من الموجود، بترجع لك كل الأسئلة بس "مبعثرة" (Shuffled).
- الـ `ToQuiz`: بتطبع السؤال بالشكل المنسق.

- الـ `GetSequenceRange` : تتبع لها `Range` معين (أراي فيها أرقام)، وهي تجيب الأسئلة اللي بتمثل الأرقام دي.

2. الـ (`Enumerable.Empty`) Empty Generation

أول موضوع معنا هو إزاي نكريت (Create) لسته فاضية، وإيه الفرق بين الطرق التقليدية وطريقة LINQ.

الطريقة التقليدية (The Bad Way)

لو عملنا لسته جديدة بالشكل ده:

C#

```
1 var questions = new List<Question>(); // Immediate Execution
2 // ... 1000 lines of code ...
3 foreach(var q in questions) { ... }
```

المشكلة هنا:

- اللسته تم خلقها في الميموري (Allocated) في السطر الأول.
- لو قعدنا 1000 سطر كود مش بنستخدمها، هي لسه حازرة مكان في الميموري على الفاضي.

الطريقة الأفضل (`Enumerable.Empty`)

C#

```
1 var questions = Enumerable.Empty<Question>();
```

المميزات:

1. الـ **Deferred Execution**: اللسته بتنخلق "كعقد" (Contract) أو وعد (Promise). التنفيذ الفعلي مش بيحصل غير لما تطلب الداتا (مثلاً لما تعمل Loop).
2. الـ **Performance & Readability**: أفضل في الأداء وأنظف في القراءة، الكود بيقول بوضوح "أنا عايز لسته فاضية".
3. الـ **Caching Mechanism**: الـ Method دي بتعمل Cache للـ Empty Sequence. ده معناه لو استدعتها 1000 مرة لنفس الـ Type، هي مش بتحجز مكان جديد في الميموري كل مرة، وده سر كفاءتها العالية مقارنة بـ `new List<T>()` اللي بتحجز مكان في الـ Heap مع كل استدعاء.
4. الـ **Handling Unions**: مفيدة جداً لو عايز تعمل اتحاد (Union) بين لستتين، ومحتاج لسته "محايدة" (Neutral) ما تضربش Null Exception، لو إحنا بنجمع لستات (Lists) جوه لسته واحدة كبيرة، أفضل قيمة نبدأ بيها هي `Enumerable.Empty<T>()` بدل `null` وبدل لسته فاضية محجوزة بـ `new`.

ملحوظة: لازم تستخدم `Enumerable.Empty` طالما أنت محتاج لسته فاضية تبدأ بيها شغلك.

3. التعامل مع القيم الافتراضية (DefaultIfEmpty)

إيه اللي يحصل لو اللسته فاضية وأنا عايز أرجع قيمة؟ هنا بتيجي فايده DefaultIfEmpty .

فهم الـ Default Values

كل Data Type في السي شارب له قيمة افتراضية:

- `int` -> `0`
- `DateTime` -> `01/01/0001`
- `Object` (Reference Type) -> `null`

C#

```
1 // مثال لمعرفة الـ Default
2 Console.WriteLine(default(int)); // Prints 0
3 Console.WriteLine(default(object)); // Prints nothing (null)
```

استخدام DefaultIfEmpty

الـ Method دي لو اللسته فاضية، بترجعك الـ Default Value الخاصة بالنوع ده.

مثال 1: بدون تحديد قيمة (يرجع Null)

C#

```
1 var questions = Enumerable.Empty<Question>(); // لسته فاضية
2 var q2 = questions.DefaultIfEmpty(); // هترجع Null لأن ده الـ Default
   Question Class
3
4 // Check لو حاولنا نطبع ونعمل
5 // Demo أو فاضي زي ما شقنا في الـ "Null" هيطبع
```

مثال 2: تحديد قيمة افتراضية مخصصة (Custom Default) نقدر نبعت للـ Method دي القيمة اللي عايزينها تظهر لو اللسته فاضية.

C#

```
1 // اللي عملناه في الكلاس Default Template هنا بنقوله لو فاضية، رجع الـ
2 var q3 = questions.DefaultIfEmpty(Question.Default);
3
4 Null أو يرجع Error هيطبع شكل السؤال التمبلت بدل ما يضرب // q3.ToQuiz();
```

تطبيق عملي هام: الـ `DefaultIfEmpty` مهمة جداً ومستخدمة بكثرة في حالات **Left Outer Join** (زي الداتا بيز)، لما يكون العنصر موجود في الشمال ومش موجود في اليمين، بنحط الـ `Default` بدل ما الدنيا تضرب.


```
<<<< QUESTION TITLE GOES HERE >>>>
1) <<<< CHOICE #1 GOES HERE >>>>
2) <<<< CHOICE #2 GOES HERE >>>>
3) <<<< CHOICE #3 GOES HERE >>>>
4) <<<< CHOICE #4 GOES HERE >>>>
```

لاحظ Question.Default ده اوبجكت استاتيكي عملته initialization داخل كلاس Question بحيث استدعيه للتعبير عن ان السؤال فاضي او بدل NULL يعنى (مفیش سؤال)

```
1 public readonly static Question Default = new Question
2 {
3     Title = "<<<< QUESTION TITLE GOES HERE >>>>",
4     Choices = new List<Choice>
5     {
6         new Choice { Order = 1, Description = "<<<< CHOICE #1 GOES HERE
7 >>>>" },
8         new Choice { Order = 2, Description = "<<<< CHOICE #2 GOES HERE
9 >>>>" },
10        new Choice { Order = 3, Description = "<<<< CHOICE #3 GOES HERE
11 >>>>" },
12        new Choice { Order = 4, Description = "<<<< CHOICE #4 GOES HERE
13 >>>>" }
14    },
15    CorrectAnswer = 0
16 };
```

4. توليد نطاق من الأرقام (Range)

الـ Range بتولد سلسلة من الأرقام الصحيحة (Integral Numbers).

الطريقة القديمة vs الطريقة الجديدة

زمان (Traditional Loop): كنا عشان نعمل أرقام من 1 لـ 10، نكتب كود طويل، نعرف Array ونعمل Loop نملاً فيها البيانات.

باستخدام (Range) LINQ:

C#

```
1 // هات لي 10 أرقام بداية من رقم 1
2 var range = Enumerable.Range(1, 10);
```

- الميزة: كود مقروء جداً، وكمان Deferred Execution (مش بينفذ غير لما تطلب الأرقام).

استخدامات الـ Range

- أي حاجة فيها **Games** أو **Randomness**.
- توليد أرقام عشوان نستخدمها كـ **Index** لجلب بيانات.

مثال عملي من المحاضرة: عابزين نجيب أسئلة عشوائية بناءً على **Range** معين:

C#

```
1 var range = Enumerable.Range(1, 10); // 10 لـ 1
2 // استخدمنا الأرقام دي عشوان نجيب الأسئلة المقابلة ليها من البنك
3 var questions = QuestionBank.GetQuestionRange(range);
4 questions.ToQuiz();
```

هنا هو جاب الأسئلة من 2 لـ 11 (لأن الـ **Index** بيبدأ من 0 وهو بدأ الـ رينج من 1).

5. التكرار (Repeat)

الـ **Repeat** بتستخدم لتكرار عنصر معين عدد محدد من المرات.

السلوك (Behavior)

C#

```
1 var q = QuestionBank.PickOne(); // اخترنا سؤال واحد
2 var repeatedQuestions = Enumerable.Repeat(q, 10); // كرر السؤال ده 10 مرات
```

نقطة جوهرية (Reference Equality)

هل الـ 10 أسئلة دول هما 10 أسئلة مختلفين؟ لأ. الـ **Repeat** بتكرر نفس الـ **Reference** لنفس الأوبجكت في الميموري.

إثبات بالكود:

1. لو عملنا **Loop** عادية وضيفنا **new Question** كل مرة، الـ **ReferenceEquals** هترجع **False** (لأن كل واحد أوبجكت مستقل).
2. لكن مع **Repeat** :

C#

```
1 // Repeat لو قارنا العنصر الأول بالعنصر الثاني في لسته معموله بـ
2 bool areSame = ReferenceEquals(repeatedQuestions[0],
3 repeatedQuestions[1]);
// النتيجة: True
```

استخدامات Repeat

ممکن حد یسأل طب إیه فایدتها لو هو نفس العنصر؟

1. الـ **Multi-processing**: لو عایز تعمل عملیات مختلفة على نفس الأوبجیکت في نفس الوقت (توزع الشغل على Cores مختلفة) وده لان احنا بنتعامل مع الـ **Address in memory** (Address in memory).
2. الـ **Scientific Simulations**: زي محاكاة الكيمياء (مثال صديق المهندس)، لما نحتاج نكرر نفس العنصر عشان نشوف تأثيرات مختلفة عليه أو مخرجات مختلفة.

الخلاصة (Summary)

- الـ **Generation Operations** بتشمل: `Empty` , `DefaultIfEmpty` , `Range` , `Repeat` .
 - كلهم بيعتمدوا على **Deferred Execution**، يعني التنفيذ مؤجل لحد وقت الحاجة، وده بيوفر في الأداء (Performance) والموارد.
 - الـ **Methods** دي بتمكنك من كتابة كود **Functional** نظيف ومقروء.
- نصيحة المهندس:** "الوقت اللي بتعرف فيه الـ **Variable** والوقت اللي بتستخدمه فيه، لو مفيش بينهم استهلاك للموارد (بسبب الـ **Deferred Execution**)، ده قمة الكفاءة."

Element Operations(ElementAt, First&Last, Single)

🤖 يعني إيه Element Operations ؟

ببساطة، الـ **Element Operations** هي العمليات اللي بتديك القدرة إنك ترجع عنصر واحد (Element) معين بـ Index معين من الـ Collection، أو بمكان معين، أو حسب شرط معين. دي عمليات مستخدمة كتير جداً في الشغل الحقيقي.

المهندس قسم الموضوع لـ 3 مجموعات رئيسية هنشرحهم بالتفصيل:

1. الوصول بالترتيب (Index).
2. الوصول لأول أو آخر عنصر (First/Last).
3. الوصول لعنصر وحيد (Single).

1 الوصول عن طريق الـ (ElementAt) Index

لو إنت عايز تجيب عنصر موجود في خانة (Index) محددة.

الـ ElementAt

بتجيب العنصر اللي في الـ index اللي إنت بتحدده.

- مثال: لو عايز تجيب السؤال اللي في المكان رقم 10 (خلى بالك فنكشن All دي داخل كلاس البنك مش تبع LINQ):

C#

```
1 var questionAt10 = QuestionBank.All.ElementAt(10);
2 Console.WriteLine(questionAt10.Title);
3 // النتيجة: هيجيب السؤال رقم 11 (لأن العد بيبدا من 0)
```

⚠️ المشاكل المحتملة (Exceptions):

1. الـ **ArgumentOutOfRangeException**: لو طلبت Index مش موجود (مثلاً الليست فيها 245 سؤال وإنت طلبت رقم 300)، البرنامج هيعطيك Exception ويقولك إنك خرجت بره الحدود.
2. الـ **ArgumentNullException**: لو الـ List نفسها (الـ Source) كانت null، البرنامج هيعطيك لأنك بتحاول تدور في ولا حاجة.

الـ ElementAtOrDefault

دي الحل للمشكلة الأولى. دي بتقول للبرنامج: "حاول تجيب العنصر اللي في الـ Index ده، ولو ملقتهوش، هاتلي القيمة الافتراضية (Default) للنوع ده".

- بما إننا شغالين مع Class (Reference Type)، فالـ Default هو null.

- الميزة: البرنامج مش هيصرب (مش هيحصل Crash)، وهيرجعك null وتقدر تعمل check عليه.

مثال للكود:

C#

```
1 var question = QuestionBank.All.ElementAtOrDefault(300);
2
3 if (question == null)
4 {
5     Console.WriteLine("Index out of range, returned null");
6 }
7 else
8 {
9     Console.WriteLine(question.Title);
10 }
```

2 الوصول للأول والآخر (First & Last)

هنا إحنا بنجيب أول أو آخر عنصر في السلسلة، سواء بشكل عام أو بناءً على شرط.

الـ First

بترجع أول عنصر في الـ Sequence بيققق شرط معين.

1. بدون شرط (No Parameters): بتجيب أول عنصر في الليسته وخلاص.

C#

```
1 var firstQ = QuestionBank.All.First();
```

2. بشرط (With Predicate): بتجيب أول عنصر بيققق الشرط ده.

C#

```
1 // هاتلي أول سؤال طول العنوان بتاعه بيساوي صفر (فاضي)
2 var q = QuestionBank.All.First(x => x.Title.Length == 0);
```

المشكلة (InvalidOperationException):

لو الشرط اللي إنت حاطه متحققش في ولا عنصر (مغيش ولا ماتش)، أو الليسته فاضية، الميثود دي هتضرب Exception اسمه InvalidOperationException وتقولك "Sequence contains no matching element".

الـ FirstOrDefault

دي الحل الآمن. نفس فكرة `ElementAtOrDefault`. لو ملقاش عنصر بيحقق الشرط، بيرجع `null` بدل ما يضرب `Exception`.

مثال:

C#

```
1 var q = QuestionBank.All.FirstOrDefault(x => x.Title.Length == 0);
2
3 if (q == null)
4 {
5     Console.WriteLine("Question is null (No match found)");
6 }
```

◆ الـ Last و LastOrDefault

نفس فكرة `First` بالظبط، بس بدل ما يجيب من أول القائمة، يجيب من آخر القائمة.

- الـ **Last**: بتجيب آخر عنصر بيحقق الشرط. لو ملقاش -> تضرب `Exception`.
- الـ **LastOrDefault**: بتجيب آخر عنصر بيحقق الشرط. لو ملقاش -> ترجع `null`.

3 العنصر الوحيد المتفرد (Single)

دي أهم واحدة ومفيدة جداً لما تكون بتدور على حاجة المفروض تكون فريدة (Unique) زي الـ Primary Key أو الـ ID.

◆ الـ Single

بتقول للبرنامج: "أنا متوقع يرجعلي عنصر واحد فقط بيحقق الشرط ده".

شروطها الصارمة (Constraints):

1. لو ملقاش ولا عنصر: بتضرب `InvalidOperationException`.
2. لو لقي أكثر من عنصر (**Duplicates**): بتضرب `InvalidOperationException` برضه! وده الفرق الجوهرى بينها وبين `First`.

ملحوظة مهمة: الـ `Exception` اللي بيطلع لما يكون في تكرار ده "حقه فلوس" على رأي البشمةهندس، لأنه بينبهدك إن عندك مشكلة في الداتا (تكرار في حاجة المفروض تكون Unique) قبل ما السيستم يبيوط.

مثال (غلطة متعددة): لو دورنا على أسئلة فيها الرقم 245 ، وكان الرقم ده موجود في سؤالين، الـ `Single` هتضرب `Exception` فوراً.

C#

```

1 // لو فيه أكثر من سؤال بيحتوي على 245 Crash هتعمل
2 var uniqueQ = QuestionBank.All.Single(x => x.Title.Contains("245"));

```

الـ SingleOrDefault

دي بتعالج مشكلة واحدة بس من المشكلتين بتوع Single .

- بتعالج إيه؟ لو ملفاش ولا عنصر (Empty Result)، بترجع null عادي ومبتضربش Exception.
- مبتعالجش إيه؟ لو لقي أكثر من عنصر (Duplicates)، لسه هتضرب Exception. لأن شرط الـ "Single" (التفرد) لسه مكسور.

الاستخدام الأمثل:

لما تكون بتدور على ID وممكن يكون موجود وممكن لأ، بس لو موجود لازم يكون واحد بس.

C#

```

1 var q = QuestionBank.All.SingleOrDefault(x => x.Title.Length == 0);
2 if (q == null)
3 {
4     Console.WriteLine("Question is null/empty"); //
5 }

```

ملخص الفروقات (Cheatsheet)

الميثود	الوظيفة	لو ملفاش حاجة (Not Found)	لو لقي أكثر من واحد (Duplicates)
ElementAt	عنصر بـ Index	Exception (Out of Range)	
ElementAtOrDefault	عنصر بـ Index	يرجع null	
First	أول عنصر	Exception	يرجع الأول
FirstOrDefault	أول عنصر	يرجع null	يرجع الأول
Last	آخر عنصر	Exception	يرجع الأخير
LastOrDefault	آخر عنصر	يرجع null	يرجع الأخير
Single	عنصر وحيد	Exception	Exception 🚫
SingleOrDefault	عنصر وحيد	يرجع null	Exception 🚫

💡 نصائح أخيرة من المحاضرة

- كل واحدة من دول ليها "Use Case" خاصة بيها ومقدرش أستغنى عنها.
- خد بالك إن لو الـ Source نفسه null ، كل دول هيبضربوا `ArgumentNullException`.
- استخدم `Single` لما تكون عايز تتأكد إن الداتا مفيهاش تكرار (Primary Key check).

Equality Operations (SequenceEqual)

مقدمة وتمهيد

في الحلقة دي هنتكلم عن **Equality** أو المساواة، وتحديدًا إزاي نحدد لو سلسلتين (Two Sequences) متساويين ولا لأ.

في متطلبات سابقة (Prerequisites) عشان تفهم الموضوع ده صح، لازم تكون مراجع حلقتين فاتوا:

1. الفرق بين **Value Type** و **Reference Type**.

2. حلقة الـ **Equality** (المساواة) اللي اتشرحت قبل كده.

تجهيز المشروع (Project Setup)

إحنا شغالين على **Console App** وفيه **Shared Project** من الحلقات اللي فاتت اللي هو (Question Bank). التعديلات اللي تمت عشان نجهز للحلقة دي:

1. الـ **Question Class**: نصفناه من أي زيادات (Default questions)، وخلينا بس الـ **Attributes** والـ **Override ToString**.
2. الـ **Choice Class**: عملنا فيه نفس الكلام.
3. الـ **QuestionBank**: لسه زي ما هو بيغذيها بالأسئلة.

المفهوم الأساسي: SequenceEqual

دالة **SequenceEqual** الموجودة في **System.Linq** بتعمل وظيفة محددة وهي التأكد من تطابق سلسلتين. وعشان ترجع **True** لازم تحقق شرطين أساسيين بالترتيب:

1. **العدد (Count)**: لازم عدد العناصر في القائمة الأولى يساوي عدد العناصر في القائمة الثانية.
2. **تطابق العناصر (Elements Match)**: كل عنصر في القائمة الأولى لازم يساوي العنصر المقابل ليه في القائمة الثانية بنفس الترتيب.

التجارب العملية (Demos)

التجربة الأولى: مقارنة قوائم بنفس الـ References (True Case)

هنا عملنا 3 أسئلة (Objects) وحطيناهم في List اسمها **Quiz1**، ورجعنا حطينا نفس الـ Objects دي في List تانية اسمها **Quiz2**.

الكود:

```

1 // Method 1 Logic
2 // بنجيب 3 أسئلة ونحطهم في متغيرات
3 var q1 = QuestionBank.PickOne();
4 var q2 = QuestionBank.PickOne();
5 var q3 = QuestionBank.PickOne();
6
7 // فيها الأسئلة دي List 1 بنعمل
8 var quiz1 = new List<Question> { q1, q2, q3 };
9
10 // References (نفس الـ) فيها نفس الأسئلة List 2 بنعمل
11 var quiz2 = new List<Question> { q1, q2, q3 };
12
13 // SequenceEqual المقارنة باستخدام
14 bool isEqual = quiz1.SequenceEqual(quiz2);
15
16 Console.WriteLine($"Quiz 1 and Quiz 2 are {(isEqual ? "Equal" : "Not
    Equal")}");

```

النتيجة:

Quiz 1 and Quiz 2 are Equal .

التفسير: ليه طلخوا متساويين؟

1. العدد واحد (3 هنا و3 هنا).

2. العناصر هي هياها (نفس الـ Reference في الميموري)، يعني q1 في الليسته الأولى ببشاور على نفس مكان q1 في الليسته الثانية.

التجربة الثانية: مقارنة Value Types (الأرقام)

هنا استخدمنا Enumerable.Range عشان نجيب أرقام من 1 لـ 4 ونقارنهم.

الكود:

```

1 // Method 2 Logic
2 // بنجيب أرقام من 1 لـ 4
3 var quiz1 = Enumerable.Range(1, 4);
4 var quiz2 = Enumerable.Range(1, 4);
5
6 bool isEqual = quiz1.SequenceEqual(quiz2);
7 Console.WriteLine($"Quiz 1 and Quiz 2 are {(isEqual ? "Equal" : "Not
    Equal")}");

```

Quiz 1 and Quiz 2 are Equal .

التفسير: لأن دي (Integers (Value Types، فالمقارنة يتم على "القيمة" مش "المكان في الذاكرة". بما إن الأرقام زي بعض بالظبط، النتيجة True .

التجربة الثالثة: المشكلة الحقيقية (Different References)

هنا بقى السيناريو الأهم. لو عملنا List فيها أسئلة، و List ثانية فيها أسئلة جديدة بس شايلة نفس البيانات بالظبط (نفس العنوان، نفس الاختيارات).

الكود:

C#

```
1 // Method 3 Logic
2 // (جديدة في الذاكرة لكل نداء Objects هنا بيتم إنشاء) بنجيب مجموعة أسئلة
3 var quiz1 = QuestionBank.GetQuestionRange(1, 4);
4 var quiz2 = QuestionBank.GetQuestionRange(1, 4);
5
6 // بنقارن بينهم
7 bool isEqual = quiz1.SequenceEqual(quiz2);
8 Console.WriteLine($"Quiz 1 and Quiz 2 are {(isEqual ? "Equal" : "Not
    Equal")}");
```

النتيجة:

Quiz 1 and Quiz 2 are Not Equal .

التفسير (ليه مش متساويين؟): من وجهة نظر البرمجة (Default Behavior)، دول Reference Types . يعني كل سؤال في quiz1 محطوط في عنوان في الذاكرة مختلف عن السؤال اللي شبيهه في quiz2 . الـ Compiler شافهم حاجات مختلفة، رغم إن ليهم نفس الشكل والبيانات.

الحل: Custom Equality (Overriding Equals)

من وجهة نظر الـ Business، لو السؤال ليه نفس العنوان ونفس الإجابات يبقى هو نفس السؤال. عشان نحقق ده، لازم نعمل Override لـ Equals و GetHashCode في الكلاسات بتاعتنا.

ملحوظة مهمة إحنا عندنا Nested Objects (كلاس Question جواه List من Choice). فلانم نطبق الـ Equality في الاثنين.

1. تعديل كلاس Choice

خطوات التنفيذ:

1. الـ **Reference Check**: نتأكد إن الـ Object مش null وإنهم مش نفس المكان في الميموري أصلاً.
2. الـ **Type Check**: نتأكد إنهم نفس النوع (Class).
3. الـ **Casting**: نحول الـ Object لـ Choice.
4. الـ **Field Comparison**: نقارن القيم (Order, Description).
5. الـ **GetHashCode**: نستخدم أرقام أولية (Prime Numbers) عشان نطلع Hash مميز ونقلل التصادم (Collision).

كود Choice Class

C#

```

1  public override bool Equals(object obj)
2  {
3      // 1. Check for null
4      if (ReferenceEquals(null, obj)) return false;
5      // 2. Check if it's the exact same reference
6      if (ReferenceEquals(this, obj)) return true;
7      // 3. Check type
8      if (obj.GetType() != this.GetType()) return false;
9
10     // 4. Compare Properties
11     var other = (Choice)obj;
12     return Order == other.Order && Description == other.Description;
13 }
14
15 public override int GetHashCode()
16 {
17     // Collision عشان نقلل الـ Prime بنستخدم أرقام
18     unchecked
19     {
20         int hash = 17;
21         hash = hash * 23 + Order.GetHashCode();
22         hash = hash * 23 + (Description != null ?
Description.GetHashCode() : 0);
23         return hash;
24     }
25 }

```

2. تعديل كلاس Question

نفس الخطوات، بس هنا في تركة مهمة جداً وهي مقارنة الـ List of Choices.

- هنقارن Title و CorrectAnswer .
- عشان نقارن الـ Choices (اللي هي List) جوه الـ Equals، هنستخدم SequenceEqual (بتاعة LINQ).
- ليه؟ لأننا خلاص ضبطنا الـ Equality جوه Choice ، فدلوقتي SequenceEqual هتقدر تقارنهم ببعض صح.

كود Question Class:

C#

```

1 public override bool Equals(object obj)
2 {
3     if (ReferenceEquals(null, obj)) return false;
4     if (ReferenceEquals(this, obj)) return true;
5     if (obj.GetType() != this.GetType()) return false;
6
7     var other = (Question)obj;
8
9     // لاحظ هنا بتكلم على التيتل والكوريكشن انسر
10    // لاحظ هنا مش هتفرق انت مستخدم == او ايكوالز لان انت بتقارن فاليو
11    return Title.Equals(other.Title)
12           && CorrectAnswer.Equals(other.CorrectAnswer)
13           // مقارنة الخصائص العادية + القائمة باستخدام SequenceEqual
14           && Choices.SequenceEqual(other.Choices); // هنا السر كله
15 }
16
17 public override int GetHashCode()
18 {
19     unchecked
20     {
21         int hash = 17;
22         hash = hash * 23 + (Title != null ? Title.GetHashCode() : 0);
23         hash = hash * 23 + (CorrectAnswer != null ?
24           CorrectAnswer.GetHashCode() : 0);
25         // Exceptions عشان نتفادى الـ Null Check يفضل تعمل GetHashCode في الـ
26         // للهاش كود لو حابب Choices وممكن تضيف الـ
27         return hash;
28     }
29 }

```

النتيجة النهائية (Final Result)

بعد ما عملنا Override في الكلاسين (Question و Choice)، لو رجعنا شغلنا التجربة الثالثة تاني:

الكود:

```

1 // Method 3 again
2 var quiz1 = QuestionBank.GetQuestionRange(1, 4);
3 var quiz2 = QuestionBank.GetQuestionRange(1, 4);
4
5 bool isEqual = quiz1.SequenceEqual(quiz2); // اللي إحنا Equals دلوقتي بتنادي الـ
    كتبناها

```

النتيجة:

Quiz 1 and Quiz 2 are Equal .

دلوقتي السيستم فهم إن طالما البيانات واحدة، يبقى الأسئلة متساوية حتى لو هما كائنات مختلفة في الميموري.

ملخص وملاحظات ختامية

- الـ **SequenceEqual**: دي Extension Method تابعة لـ LINQ (System.Linq)، وظيفتها تقارن تسلسلات ببعضها.
- الـ **ReferenceEquals**: دي ميثود تابعة لـ Object Class ، وظيفتها تشوف هل المتغيرين بيشاروا على نفس المكان في الميموري ولا لأ.
- الـ **Best Practice**: لما تعمل Custom Type وبتستخدمه في مقارنات أو كـ Key في Dictionary، لازم تعمل Override لـ Equals و GetHashCode عشان تضمن السلوك الصحيح

Concatenation

المحاضر: المهندس عصام عبد النبي الموضوع: Concatenation (الدمج) - وهي عملية إضافة محتويات قائمة (List) أو سلسلة (Sequence) لمحتويات سلسلة أخرى لتكوين سلسلة جديدة.

1. مقدمة وتجهيز المشروع

في البداية، الموضوع ببساطة هو إن النتيجة اللي بتطلع معاك هي حاصل دمج سلسلتين مع بعض.

- الطرق المتاحة: نقدر نعمل العملية دي بأكثر من طريقة، لكن الـ LINQ وفرت لنا Method جاهزة اسمها Concat .
- إعداد المشروع (Setup): نفس المشروع المستخدم في الحلقات اللي فاتت، التعديل الوحيد إننا شلنا الـ Override الخاص بـ Equals لأننا مش هحتاجه في الحلقة دي.

2. المثال الأول: الدمج المباشر (Basic Concatenation)

هنا هتشفوف إزاي ندمج قيمتين من نوع IEnumerable بشكل مباشر.

الكود والخطوات:

1. هنعمل Method اسمها RunMethod1 .
2. هنكربت مجموعتين من الأسئلة (quiz1 و quiz2) بشكل عشوائي.
3. نستخدم Concat لدمجهم في quiz3 .

C#

```
1 static void RunMethod1()
2 {
3     // تجهيز البيانات
4     var quiz1 = QuestionBank.Randomize(3); // أسئلة 3
5     var quiz2 = QuestionBank.Randomize(2); // سؤالين
6
7     // عملية الدمج باستخدام Concat
8     // النتيجة هنا هتأخذ نفس صفة العناصر المدمجة (IEnumerable of Questions)
9     var quiz3 = quiz1.Concat(quiz2);
10
11     // الطباعة للتأكد
12     // بتطبع الأسئلة بشكل منظم ToQuiz دالة
13     quiz3.ToQuiz();
14 }
```

النتيجة:

- هتلاحظ إن quiz3 أصبح IEnumerable ويحتوي على الأسئلة من القائمة الأولى والقائمة الثانية بدون أي تعديلات.
- لما نعمل Run ، هيطبعك الـ 3 أسئلة بتوع الكويز الأول، وتحتهم السؤالين بتوع الكويز الثاني (الإجمالي 5 أسئلة).

3. المثال الثاني: دمج أنواع محددة (Projection with Concat)

افترض إننا مش عايزين نطلع الأسئلة كاملة، إحنا محتاجين بس "عناوين الأسئلة" (Titles) ونعملها دمج.

الكود والخطوات:

هنستخدم Select عشان نختار الـ Title بس، وبعدين نعمل Concat .

C#

```
1 // بنختار عنوان السؤال بس من القائمة الأولى
2 var questionTitles = quiz1.Select(q => q.Title)
3 // وبندمجه مع عناوين الأسئلة من القائمة الثانية //
4 .Concat(quiz2.Select(q => q.Title));
5
6 // طباعة العناوين
7 foreach(var title in questionTitles)
8 {
9     Console.WriteLine(title);
10 }
```

💡 ملاحظة مهمة جداً عن الـ Scope (نطاق المتغيرات):

جاني سؤال على الفيسبوك بخصوص تسمية المتغيرات داخل الـ Lambda Expression.

- في الكود اللي فوق، استخدمنا المتغير q في الجزء اللي على الشمال، واستخدمنا q برضه في الجزء اللي على اليمين.
- هل دول ليهم علاقة ببعض؟ لأ، نهائياً.
- الـ q اللي جوه Select الأولى ده Local Variable نطاقه (Scope) بيتنتهي بانتهاء القوسين بتوعها. والـ q الثانية دي حاجة تانية خالص، مجرد تشابه أسماء.

النتيجة: لما تشغل RunMethod2 ، هيعرضلك عناوين الأسئلة بس كنصوص، لأننا نقينا (Selected) الشيء اللي عايزينه بس.

4. المثال الثالث: دمج أكثر من قائمة (Chaining)

قوة Concat إنها Extension Method ، يعني تقدر تناديها ورا بعض أكثر من مرة وتدمج أي عدد من القوائم.

الكود والخطوات:

هنفترض إن عندنا quiz1 و quiz2 وهنجيب مجموعة تالته quiz3 باستخدام GetQuestionRange (مثلاً من السؤال 11 لـ 14).

C#

```
1 // الطريقة: دمج متسلسل
2 var allQuizzes = quiz1.Concat(quiz2)
3                          .Concat(quiz3); // دمجت القائمة التالته كمان
4
5 // تقدر تكتبها بشكل مقروء أكثر لو حابب:
6 /*
7 var allQuizzes = quiz1
8                  .Concat(quiz2)
9                  .Concat(quiz3);
10 */
```

⚠ شرط مهم جداً (Type Safety):

- لازم اللي بتدمجه يكون من نفس النوع.
- يعني مينفعش تدمج قائمة من String (زي العناوين) مع قائمة من Int (زي الإجابة الصحيحة CorrectAnswer). الكومبايلر هيديك Error فوراً لأن الأنواع مش متطابقة.

النتيجة: هيطلعلك الـ 2 سؤال بتوع الكويز الأول + الـ 3 أسئلة بتوع الثاني + الـ 14 سؤال الجداد، الإجمالي هيكون 19 سؤال في قائمة واحدة.

5. المثال الرابع: الدمج باستخدام Collection_INITIALIZER و SelectMany

دي طريقة تانية للدمج بس "لفة" شوية، بتعتمد إننا نحط القوائم جوه Array أو List وبعدين نفكها.

الفكرة:

بدل ما أستخدم Concat، أنا هعمل new List أو Array وأحط جواها quiz1 و quiz2 كعناصر.

C#

```
1 عملنا مصفوفة بتحتوي على الكويز الأول والكويز الثاني //
2 var combinedQuizzes = new [] { quiz1, quiz2 };
```

المشكلة هنا:

- المتغير combinedQuizzes نوعه إيه دلوقتي؟
- هو مش IEnumerable<Question> .. لأ، هو IEnumerable<IEnumerable<Question>> (يعني مصفوفة جواها مصفوفتين).
- لو حاولت تنادي دالة ToQuiz() مش هتشتغل لأنها مستتية Questions مش "قوائم أسئلة".

الحل (SelectMany):

عشان نحول "مصفوفة المصفوفات" دي لمصفوفة واحدة مسطحة (Flatten)، بنستخدم `SelectMany` (اللي شرحناها قبل كده).

C#

```
1 // عشان "تفك" القوائم دي وتدمجهم في قائمة واحدة SelectMany بنستخدم
2 var finalQuiz = combinedQuizzes.SelectMany(q => q);
3
4 // دلوقتي نقدر نطبع عادي
5 finalQuiz.ToQuiz();
```

النتيجة: كأننا عملنا دمج عادي، وهيطلعنا الأسئلة كلها ورا بعض (مثلاً $5 = 3 + 2$ أسئلة).

ملخص الحلقة (Recap)

إحنا اتعلمنا النهاردة 4 طرق أو أشكال للتعامل مع الدمج:

1. **الدمج المباشر (Direct Concat):** دمج قائمتين من نفس النوع بشكل مباشر `list1.Concat(list2)`.
2. **الدمج مع التخصيص (With Projection):** استخدام `Select` قبل الدمج عشان ندمج خصائص معينة (زي العناوين بس)، مع مراعاة إن النوع لازم يكون واحد.
3. **الدمج المتعدد (Chaining):** دمج أكثر من قائمة ورا بعض `list1.Concat(list2).Concat(list3)` مفيش قيود على العدد.
4. **الدمج عن طريق التسطيح (Via SelectMany):** إننا نجمع القوائم جوه `Collection` واحد (زي مصفوفة) وبعدين نستخدم `SelectMany` عشان نفكهم ونخليهم قائمة واحدة.

خطوتك الجاية: جرب بإيدك تعمل `List` فيها أرقام و `List` ثانية فيها أرقام، واعملهم `Concat`. وبعدين جرب تعمل `Concat` لليسته `Strings` مع `Integers` وشوف الـ `Error` اللي هيطلعك عشان تثبت المعلومة.

Aggregate Operations

في الحلقة دي، هنتكلم عن الـ **Aggregate Operations**، وهي باختصار: "أديني مجموعة عناصر (Sequence) وأنا أطلعك منهم عنصر واحد بس نتيجة عملية معينة عليهم".

① ملحوظة مهمة قبل البدء المشروع ده اتعمل على **NET 6**. ليه؟ عشان في بعض الـ **Functionalities** والميثودز الجديدة (زي **MaxBy** و **MinBy**) مش موجودة في **NET 5**، فلازم تكون شغال على **NET 6**. عشان الكود يشتغل معاك كله.

1. دالة Aggregate (التجميع اليدوي)

الـ **Aggregate** ميثود موجودة في **System.Linq**، ووظيفتها إنها بتعمل تجميع للقيم جوه الـ **Collection** بس بالطريقة اللي أنت بتحددها (**Custom Function**)، مش طريقة مفروضة عليك، خلى بالك هي ليها أكثر من **Overload** وانواعهم كالتالي:

1. التجميع البسيط (Simple Overload)

بتأخذ **Func** ببيستقبل قيمتين (القديمة والجديدة) من نفس نوع الليسته، وبيرجع قيمة من نفس النوع.

```
1 public static TSource Aggregate<TSource>(this IEnumerable<TSource>  
    source,  
2     Func<TSource, TSource, TSource> func  
3 );
```

2. التجميع بقيمة ابتدائية (With Seed)

بتأخذ **seed** (قيمة البداية)، والـ **Func** هنا ببيستقبل الـ **seed** (أو التراكمي) والعنصر الحالي، وبيرجع التراكمي الجديد

```
1 public static TAccumulate Aggregate<TSource, TAccumulate>(  
2     this IEnumerable<TSource> source,  
3     TAccumulate seed,  
4     Func<TAccumulate, TSource, TAccumulate> func  
5 );
```

3. التجميع مع اختيار النتيجة النهائية (With Result Selector)

seed: القيمة اللي هنبداً بيها. 2. **func**: اللوجيك اللي هيتكرر مع كل عنصر. 3. **resultSelector**: دالة بتتنادى مرة واحدة في الآخر عشان تحدد إيه اللي يرجع (ممكن تحول النتيجة لنوع تالت خالص).

```
1 public static TResult Aggregate<TSource, TAccumulate, TResult>(  
2     this IEnumerable<TSource> source,  
3     TAccumulate seed,  
4     Func<TAccumulate, TSource, TAccumulate> func,  
5     Func<TAccumulate, TResult> resultSelector  
6 );
```

◆ مثال 1: دمج النصوص (Simple Overload)

لو عندنا مصفوفة أسماء وعاليزين نطلع نص واحد فيه كل الأسماء وبينهم فواصل (Comma).

البيانات:

C#

```
1 var names = new[] { "Ali", "Salem", "Abeer", "Jalal" };
```

الطريقة القديمة (قبل LINQ):

كنا بنعمل foreach ونجمع في متغير، وبعدين نشيل آخر فاصلة بـ TrimEnd ، أو نستخدم String.Join .

طريقة LINQ Aggregate:

بنستخدم دالة Aggregate اللي بتاخذ (Func (Delegate) .

C#

```
1 var commaSeperatedNames = names.Aggregate((a, b) => a + ", " + b);
2 Console.WriteLine(commaSeperatedNames);
3 // Output: Ali, Salem, Abeer, Jalal
```

شرح الـ Logic (إيه قصة a و b ؟):

- الـ (Accumulator) a : دي القيمة القديمة أو "التراكمية" اللي شايلة النتيجة لحد دلوقت.
- الـ (Current/Next) b : دي القيمة الجديدة اللي عليها الدور في الـ Array.
- العملية: بياخد a يدمجها مع b ، والنتيجة تبقى هي الـ a في اللفة اللي بعدها.

محاكاة اللي بيحصل جوه الذاكرة:

1. أول لفة: a = "Ali".
2. ثاني لفة: a بقت "Ali, Salem".
3. ثالث لفة: a بقت "Ali, Salem, Abeer" ... وهكذا.

◆ مثال 2: الجمع مع قيمة ابتدائية (With Seed)

لو عندنا أرقام وعاليزين نجمعهم، بس نبدأ الجمع من رقم معين مش من الصفر.

البيانات:

C#

```
1 var numbers = new[] { 1, 2, 3, 4, 5 };
```

الكود باستخدام Seed:

هنا هنستخدم Overload ثاني للدالة بياخد **Seed** (قيمة ابتدائية).

C#

```
1 // نقطة البداية) Seed هي الـ 0
2 var result = numbers.Aggregate(0, (a, b) => a + b);
3 Console.WriteLine(result);
4 // Output: 15 (1+2+3+4+5)
```

لو غيرنا الـ **Seed**: لو خلينا الـ Seed بـ 2 مثلاً:

C#

```
1 var result = numbers.Aggregate(2, (a, b) => a + b);
2 // Output: 17
```

لأنه بدأ بـ 2 وجمع عليها الـ 15 بتوع المصفوفة.

◆ مثال 3: أطول سؤال (With Result Selector)

عايزين نطلع أطول سؤال (كتابةً) من بنك الأسئلة.

- المنطق: هنفترض إن أول عنصر هو "الأطول" (Seed)، ونقارنه باللي بعده، لو الجديد أطول ناخده، لو لا نخلينا مع القديم.

C#

```
1
2 var quiz = QuestionBank.All;
3 var longestQuestionTitle = quiz[0];
4 //-----
5
6 longestQuestionTitle =
7     quiz.Aggregate(longestQuestionTitle,
8         (longest, next) => longest.Title.Length < next.Title.Length ?
9         next : longest,
10        x => x);
```

تفصيل الكود:

1. الـ **Seed**: Quiz.Questions[0] (افتترضنا أول واحد هو الأطول).

2. المقارنة: هل next أطول من longest ؟

- آه: رجع next (هو بقى الأطول الجديد).
 - لأ: رجع longest زي ما هو.
3. في الآخر الدالة هترجع كائن (Object) السؤال صاحب أطول عنوان.

🔗 النتيجة الكود طلع إن السؤال رقم 124 هو صاحب أطول نص، مش أول سؤال زي ما افترضنا في البداية.

ف حالة لو انت بتتعامل مع String فيه تركايه بسيطه:
لو عايزها تشتغل StringBuilder وتكون سريعة جداً، بتكتبها كدة:

C#

```
1 names.Aggregate(  
2     new StringBuilder(),           // البداية بيلدر فاضي  
3     (sb, name) => sb.Append(name).Append(", "), // دالة التجميع  
4     sb => sb.ToString()           // النتيجة النهائية  
5 );
```

2. دوال التجميع القياسية (Standard Aggregate Methods)

دي دوال جاهزة بتعمل عمليات حسابية شائعة، سهلة وسريعة.

1. الكاونت (Count & LongCount)

بتعد العناصر الموجودة في الـ Sequence.

- الـ Count العادية: #C

```
1 var count = Quiz.Questions.Count(); // بترجع int  
2 // Output: 200
```

- الـ Count بشرط (Predicate): ممكن نعد بس الأسئلة اللي علامتها بـ 1. #C

```
1 var countOneMark = Quiz.Questions.Count(x => x.Mark == 1);  
2 // Output: 21
```

- ممكن برضه تتعمل بـ Where وبعدين Count ، والاتنين بيطلعوا نفس النتيجة.
- الـ LongCount: لو بتتعامل مع داتا ضخمة جداً والعدد ممكن يعدي حدود الـ integer ، استخدم LongCount لأنها بترجع long .

2. الماكس (Max & MaxBy)

- الـ Max: بتجيب أكبر قيمة (كرقم) لخاصية معينة. #C

```
1 var maxMark = Quiz.Questions.Max(x => x.Mark);  
2 // Output: 9 (أعلى درجة سؤال هي 9)
```

- الـ **MaxBy** (ميزة .NET 6): دي بقى بتجيب العنصر نفسه (**Object**) اللي صاحب أكبر قيمة، مش بس القيمة. #C

```
1 var questionWithMaxMark = Quiz.Questions.MaxBy(x => x.Mark);
2 Console.WriteLine(questionWithMaxMark.Title);
3 // هيطبع عنوان السؤال اللي درجته 9
```

مع دمج الفلتر مع الـ **Max** ممكن نفلتر الأول وبعدين نجيب الماكس. مثلاً: هات أعلى درجة للأسئلة اللي عدد اختياراتها أقل من 3.

C#

```
var max = Quiz.Questions.Where(x => x.Choices.Count < 3).Max(x => x.Mark)
```

3. المِن (Min & MinBy)

نفس فكرة الـ **Max** بالظبط بس بتجيب الأقل.

- الـ **Min**: ترجع أقل قيمة (مثلاً أقل درجة 1).
- الـ **MinBy**: ترجع السؤال اللي واخد أقل درجة.

4. المجموع (Sum)

بتجمع قيم لخاصية معينة.

C#

```
1 // جمع كل درجات الأسئلة في الامتحان
2 var totalMarks = Quiz.Questions.Sum(x => x.Mark);
3 // Output: مجموع الدرجات كلها
```

5. المتوسط (Average)

بتحسب المتوسط الحسابي للقيم.

C#

```
1 var avgMark = Quiz.Questions.Average(x => x.Mark);
2 Console.WriteLine(avgMark.ToString("0.00"));
3 // Output: 5.09 (مثلاً)
```

ملخص سريع (Summary)

- الـ **Aggregate**: إنت اللي بتكتب اللوجيك بتاع التجميع (Custom Logic) زي دمج النصوص أو مقارنات معقدة.
- الـ **Standard Methods**: دوال جاهزة وسريعة (Count , Sum , Average , Max , Min).
- الأداء: الدوال دي Very Handy وأداؤها عالي (Performant) خصوصاً مع قواعد البيانات.

جدول الفروقات (The "Zatoona" Table)

الميثود (Method)	الوظيفة (Functionality)	النتيجة (Returns)	ملاحظات هامة (Key Notes)
Aggregate	تجميع "يدوي" (Custom) بمنطق أنت اللي بتكتبه	TSource أو أي نوع تحدده	بتستخدم Accumulator (قيمة قديمة) و Next (قيمة جديدة) لعمل عملية تراكمية.
Count	بتعد العناصر في الليسته	int	ممكن تاخذ شرط (Predicate) تعد بس اللي بيحققوه.
LongCount	بتعد العناصر برضه	long	استخدمها لو الداتا ضخمة جداً والعدد ممكن يعدي حجم الـ int .
Sum	بتجمع قيم رقمية	int , decimal , etc.	بتجمع خاصية معينة (مثلاً مجموع الدرجات).
Average	بتحسب المتوسط الحسابي	double	مفيدة جداً في الإحصائيات (مثلاً متوسط درجات الطلاب).
Max / Min	بتجيب قيمة أكبر أو أصغر عنصر	int , double , etc.	بترجع القيمة نفسها (مثلاً الرقم 9)، مش السؤال اللي جايب 9.
MaxBy / MinBy	بتجيب الكائن (Object) صاحب أكبر/أصغر قيمة	TSource (الكائن نفسه)	مهم: دي ميزة نزلت في .NET 6.

Set Operations(Except, Intersect, Union)

مقدمة (Introduction)

النهاردة موضوعنا عن **Set Operations** أو العمليات اللي بنعملها على المجموعات. ده موضوع حيوي جداً خصوصاً لما تكون بتتعامل مع بيانات موجودة في أكثر من قائمة (List/Sequence) وعاليز تشوف:

- إيه المتشابه بينهم؟
- إيه المختلف؟
- إيه القيم المنفردة (Unique) من غير تكرار؟

مع ملاحظة مهمة جداً المشروع ده مبني على **NET 6** مش **NET 5**. ليه؟ لأن في بعض الـ **Operations** والميثودز اللي هنتكلم عنها (زي **By methods**) مش موجودة غير في **NET 6**، فلازم تتأكد إنك شغال على الإصدار ده.

تجهيز المشروع (Project Setup)

عشان نطبق عملي، عملنا Setup بسيط عبارة عن 2 Classes:

1. كلاس الاجتماع (Meeting)

بيمثل محضر اجتماع بين موظفين.

- الـ **Date**: استخدمنا فيه **DateOnly** (نوع جديد في **NET 6**) بيخزن التاريخ بس (سنة، شهر، يوم) من غير وقت.
- الـ **Time**: استخدمنا **TimeOnly** للبداية والنهاية (ساعة، دقيقة، ثانية).
- الـ **Participants**: قائمة بالموظفين المشاركين.
- الـ **Overrides**: عملنا **Override** للـ **ToString** عشان يطبع بيانات الاجتماع والموظفين بشكل شجري (Hierarchical).

2. كلاس الموظف (Employee)

- له **Id** و **Name**.
- الـ **Overrides**:
- الـ **ToString**: عشان يطبع الاسم والرقم بين أقواس [].
- الـ **Equals & GetHashCode**: عشان نحدد مبدأ المساواة. الموظف ده بيساوي الموظف ده لو (الاسم بيساوي الاسم) و (الرقم بيساوي الرقم).

أولاً: إزالة التكرار (Distinct & DistinctBy)

المشكلة: التكرار مع **Concat**

لو عندنا اجتماعين (Meeting1 , Meeting2) وعايزين ندمج المشاركين فيهم مع بعض، بنستخدم Concat .

- النتيجة: هيدمج القائمتين بس هيكسر الأسماء لو حد حضر الاجتماعين.

الحل 1: Distinct

دي الميثود الحرفية للقيم المنفردة، بتشيل التكرار من الـ Collection.

الكود:

C#

```
1 var distinctParticipants = meeting1.Participants
2   .Concat(meeting2.Participants)
3   .Distinct();
```

إزاي بتشتغل؟

1. بتشوف هل في Default Equality Comparer ؟
2. لو إنت عامل Override للـ Equals في كلاس الـ Employee : هتقارن القيم (الاسم والرقم) وهتشيل التكرار فعلاً.
3. لو مش عامل Override: هيعتبرهم Reference Type ، يعني لو نفس الاسم بس في Object مختلف في الذاكرة (New Reference)، هيشوفهم مختلفين ومش هيشيل التكرار.

الحل 2: DistinctBy (جديدة في .NET 6)

طب لو أنا مش عايز أو مش متاح ليا أعمل Override للـ Equals ؟ أو عايز أقارن بناء على خاصية معينة بس (زي رقم الموظف)؟ هنا بنتيجي فائدة DistinctBy . بتقوله "شيل التكرار بناء على المفتاح ده".

الكود:

C#

```
1 // شيل التكرار بناء على رقم الموظف فقط
2 var distinctByParticipants = meeting1.Participants
3   .Concat(meeting2.Participants)
4   .DistinctBy(x => x.EmployeeNumber);
```

الميزة: حتى لو الـ References مختلفة، طالما رقم الموظف واحد، هيعتبرهم واحد ويشيل التكرار.

ثانياً: الاستثناء (Except & ExceptBy)

دي بتجاوب على سؤال: "مين موجود في القائمة الأولى ومش موجود في القائمة الثانية؟" (A - B).

1. الـ Except

بتجيب العناصر الموجودة في الـ Sequence الأول وتم استثناءها من الـ Sequence الثاني.

C#

```

1  // هات لي المشاركين في الاجتماع الأول بس ما يكونوش حضروا الاجتماع الثاني
2  var set3 = meeting1.Participants.Except(meeting2.Participants);

```

المشكلة: نفس فكرة Distinct ، لازم تكون عامل Override للـ Equals عشان يعرف إن "سواء" اللي هنا هي "سواء" اللي هناك ويستثنيها. لو مش عامل Override، مش هيعرف إنهم نفس الشخص وهيرجلك "سواء" عادي كأنها مش موجودة في القائمة الثانية.

2. الـ ExceptBy

لو معندناش مساواة (Equals Override)، نستخدم ExceptBy .

التكوين (Signature): بتأخذ منك:

1. القائمة اللي بنستثني منها (Source).
2. قائمة المفاتيح اللي هنستثنيها (Items to exclude keys).
3. الـ Key Selector (الخاصية اللي هنقارن بيها).

الكود:

C#

```

1  // استثني الناس اللي أرقامهم الوظيفية موجودة في القائمة الثانية
2  var set4 = meeting1.Participants
3      .ExceptBy(meeting2.Participants.Select(x => x.EmployeeNumber), // القائمة
4              الثانية (كمفاتيح)
                    x => x.EmployeeNumber); // المفتاح من القائمة الأولى

```

ملاحظة: هنا بنقارن بالـ EmployeeNumber ، فلو "سواء" موجودة في الاجتماع الثاني بنفس الرقم، هيتم استبعادها من النتيجة حتى لو الـ Objects مختلفة.

ثالثاً: التقاطع (Intersect & IntersectBy)

دي بتجاوب على سؤال: "مين العناصر المشتركة الموجودة في القائمتين؟" ($A \cap B$).

1. الـ Intersect

بتجيب التقاطع بين المجموعتين.

- لو مفعّل الـ Equals : هتجيب المشترك (مثلاً "سواء" لأنها في الاجتماعين).
- لو لاغيين الـ Equals (شغالين بالـ Reference): النتيجة هتكون **صفر** (مفيش تقاطع) لأن الـ Objects مختلفة في الذاكرة.

C#

```
1 var commonParticipants =
  meeting1.Participants.Intersect(meeting2.Participants);
```

2. الـ IntersectBy

بتحل مشكلة الـ Reference عن طريق تحديد مفتاح للتقاطع.

الكود:

C#

```
1 var commonByParticipants = meeting1.Participants
2   .IntersectBy(meeting2.Participants.Select(x => x.EmployeeNumber),
3               x => x.EmployeeNumber);
```

هنا حتى لو الـ References مختلفة، طالما الأرقام واحدة، هيجيب التقاطع بينهم.

رابعاً: الاتحاد (Union & UnionBy)

في ناس بتتلخبط بين Union و Concat .

- الـ Concat: دمج فقط (بيسمح بالتكرار).
- الـ Union: اتحاد مجموعات (بيجيب العناصر الفريدة بدون تكرار).

1. الـ Union

بتعمل دمج وتلقائياً بتعمل Distinct .

- لو في Equals : "سواء" هتظهر مرة واحدة.
- لو مفيش Equals : "سواء" هتظهر مرتين (زي الـ Concat).

الكود:

C#

```
1 var unionSet = meeting1.Participants.Union(meeting2.Participants);
```

2. الـ UnionBy

دي بقى اللي فيها اللغز والاختلاف في الـ Parameters. هنا بنقوله اعمل اتحاد وشيل التكرار بناء على EmployeeNumber .

C#

```

1 var unionBySet = meeting1.Participants
2   .UnionBy(meeting2.Participants, // لاحظ هنا بعطنا الليسته كاملة مش المفاتيح بس
3           x => x.EmployeeNumber);

```

النتيجة: هيجيب كل الناس، و"سواء" هتيجي مرة واحدة لأننا وحدنا المعيار على رقم الموظف.

سؤال الحلقة (The Big Question)

في الـ `UnionBy`، إحنا بعطنا الـ `List` الثانية (`set2`) كاملة زي ما هي: `UnionBy(set2, x => x.Id)`

بينما في `IntersectBy` و `ExceptBy`، اضطررنا نعمل `Select` للمفاتيح من الـ `List` الثانية:

```
IntersectBy(set2.Select(x => x.Id), x => x.Id)
```

السؤال ليكم:

ليه في `UnionBy` ما طلبش أحدد الـ `Property` اللي هقارن على أساسها من الليسته الثانية وبعته الليسته كاملة؟ بينما في `IntersectBy` كان لازم أعمل `Project (Select)` للشيء اللي بتقاطع معاه؟

حالة `UnionBy` (الاتحاد)

الهدف: دمج القائمتين.

في الاتحاد، النتيجة النهائية بتحتوي على عناصر من القائمة الأولى و عناصر من القائمة الثانية (لو كانت مش مكررة).

- بما إن الـ `LINQ` محتمل ياخذ عناصر من القائمة الثانية ويحطها في النتيجة النهائية، فهو محتاج الـ `Object` الكامل بتاع القائمة الثانية، مش بس المفتاح بتاعه.
- عشان كده لازم تبعتله `IEnumerable<TSource>` (الليسته كاملة)، وهو بياخذ منها الـ `Key` بنفسه عشان يقارن.

مثال: لو بتعمل `Union` بين (موظفين القاهرة) و (موظفين الإسكندرية). لو في موظف في إسكندرية مش موجود في القاهرة، لازم أضيف "بيانات الموظف ده كاملة" للنتيجة. فلازم يكون معايا الليسته كاملة.

حالة `IntersectBy` و `ExceptBy` (التقاطع والاستثناء)

الهدف: فترة القائمة الأولى بناءً على القائمة الثانية.

في العمليتين دول، النتيجة النهائية بتيجي حصرياً من القائمة الأولى (1 Source).

- القائمة الثانية دورها هنا مجرد "مرجع" أو "لسته مفاتيح" بنبص فيها عشان نقرر (ناخذ العنصر من الأولى ولا لا).
- إحنا مستحيل نرجع أي `Object` من القائمة الثانية في النتيجة، إحنا بس محتاجين الـ `Key` بتاعه عشان نتشيك عليه (`Lookup`).
- عشان كده الـ `LINQ`، عشان يوفر في الأداء والميموري، طلب منك تبعتله المفاتيح بس (`IEnumerable<TKey>`)، لأنه مش محتاج الـ `Object` الكامل في أي حاجة.

مثال: لو عايز (الموظفين اللي في القاهرة) وموجودين كمان في (الإسكندرية) `Intersect` . النتيجة هتكون "موظفين من لسته القاهرة". لسته إسكندرية أنا بس بصيت فيها على "أرقام الموظفين" عشان أشوف مين مشترك، مش محتاج بياناتهم الكاملة.

ملخص سريع:

- الـ `UnionBy` : محتاج يرجع عناصر من الليسته الثانية ← لازم ياخذ الليسته كاملة.
- الـ `IntersectBy` : بيرجع عناصر من الليسته الأولى بس ← محتاج المفاتيح من الثانية بس عشان المقارنة.

الخلاصة:

1. الـ `Distinct`: إزالة تكرار (بيعتمد على `Equals` أو `Reference`).
2. الـ `Except`: استثناء (الموجود في الأول ومش في الثاني).
3. الـ `Intersect`: تقاطع (المشترك فقط).
4. الـ `Union`: اتحاد (الكل بدون تكرار).
5. الـ `(DistinctBy , ExceptBy ...)` **By Methods**: ظهرت في .NET 6 عشان تسهل المقارنة بمفتاح محدد (Key Selector) بدلاً من الاعتماد الكلي على `Equals` Override .

Expression Trees

1. مقدمة: يعني إيه Expression Trees؟ ولية مهمة؟

في البداية، الموضوع ده بيعتبر من أهم المواضيع خصوصاً لما مستواك يعلى شوية في التراك وتدخل في Entity Framework Core (EF Core) و ORMs.

- الفرق الجوهرى:
 - الـ IEnumerable : بيتغذى على Func (Delegates) و بيرن الكود في الميموري.
 - الـ IQueryable : (وده النوع الجديد اللي هنتعامل معاه) بيتغذى على Expressions مش مجرد كود بيتنفذ.
- الفكرة الأساسية (Code as Data):
 - ببساطة، الـ Expression Tree هي طريقة لتمثيل الكود بتاعك باستخدام Data Structure.
 - بدل ما الكود يكون Static وتكتب الفانكشن وتعملها Compile مرة واحدة، لآ.. إحنا هنا بنخلق الفانكشن بشكل ديناميكي (On the fly).
 - تقدر تخزن الكود في متغير، تعدل عليه، تشيل أجزاء وتحط أجزاء، كأن الكود "Data" في إيدك.

ليه بنستخدم الـ Tree تحديداً؟

- ليه مش Array أو Linked List أو Stack؟
 - لأن الكود بطبيعته هرمي (Hierarchical).
 - تخيل عملية حسابية فيها أقواس وتداخلات، لازم تفك الأقواس الداخلية الأول وبعدين الخارجية. أفضل هيكل بيانات يمثل التسلسل ده هو الشجرة (Tree).

2. استرجاع سريع: Delegates vs Lambda Expressions

عشان تفهم الـ Expression Trees لازم تكون فاهم Delegates و Func كويس (مراجعة سريعة):

لو عندنا Delegate اسمه <Func<int, bool>، ده معناه فانكشن بتستقبل int وبترجع bool (زي دالة بتشوف الرقم زوجي ولا لا).

الكود العادي (Delegate):

C#

```
1 // Lambda Expression ببشاور على Func تعريف متغير من نوع
2 // لو باقي القسمة على 2 بيساوي صفر true ويرجع num بياخد
3 Func<int, bool> isEven = num => num % 2 == 0;
4
5 // الاستدعاء
6 bool result = isEven(10); // بترجع true
7 // أو
8 bool result2 = isEven.Invoke(10); // نفس النتيجة
```

ممکن برضو نعملها ک Local Function أو Static Method، بس ال Delegate هو الأكثر شيوعاً دلوقتى.

3. تحويل الكود لـ Expression Tree (Code as Data)

دلوقتى إزاي نحول الكود اللي فات ده لـ Expression؟ فى كلاس اسمه Expression موجود فى System.Linq.Expressions.

التحويل: بدل ما نقول <Func<int, bool>، بنحطها جوه "Wrapper" اسمه <...>Expression.

C#

```
1 using System.Linq.Expressions;
2
3 // لاحظ هنا: إحنا مش بنعمل فانكشن نتنفذ، إحنا بنعمل "شجرة" بتوصف الفانكشن دي
4 Expression<Func<int, bool>> isEvenExpression = num => num % 2 == 0;
```

الفرق بين ال Delegate وال Expression:

- ال Delegate: هو الكود نفسه جاهز للتنفيذ.
- ال Expression: هو "وصف" للكود (Data describing the code). المهندس عصام شبيهها بتشبيهه عبقري: ال Delegate هو "الكنز" نفسه.
- ال Expression: هو "خريطة الكنز".

إزاي أنفذ ال Expression؟

لو حاولت تستدعي isEvenExpression(10) الكومبايلر هيعطيك Error، لأن ده مش دالة، ده Data Structure. عشان تنفذه لازم تحوله لـ Delegate الأول باستخدام ميثود اسمها Compile().

C#

```
1 // 1. تحويل الخريطة (Expression) لكنز (Delegate)
2 Func<int, bool> isEvenFunc = isEvenExpression.Compile();
3
4 // 2. تنفيذ الكود
5 Console.WriteLine(isEvenFunc(10)); // True
```

4. تشريح ال Expression Tree (Architecture)

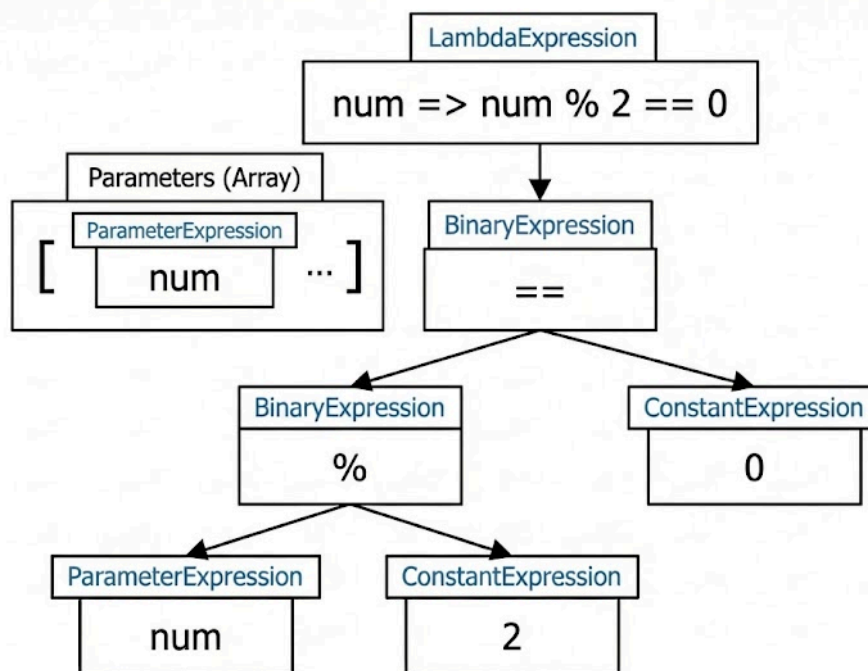
عشان تفهم إيه اللي بيحصل جوه، لازم نعرف الهيكلية:

1. ال Expression Class: ده الأب الكبير (Abstract Base Class).
2. بيورث منه أنواع كتير زي:

- الـ BinaryExpression : للعمليات الثنائية (جمع، طرح، أكبر من، يساوي...).
- الـ ConstantExpression : للثوابت (أرقام، نصوص).
- الـ ParameterExpression : للمتغيرات والباراميترز.
- الـ LambdaExpression : اللي شايك الليلة كلها.

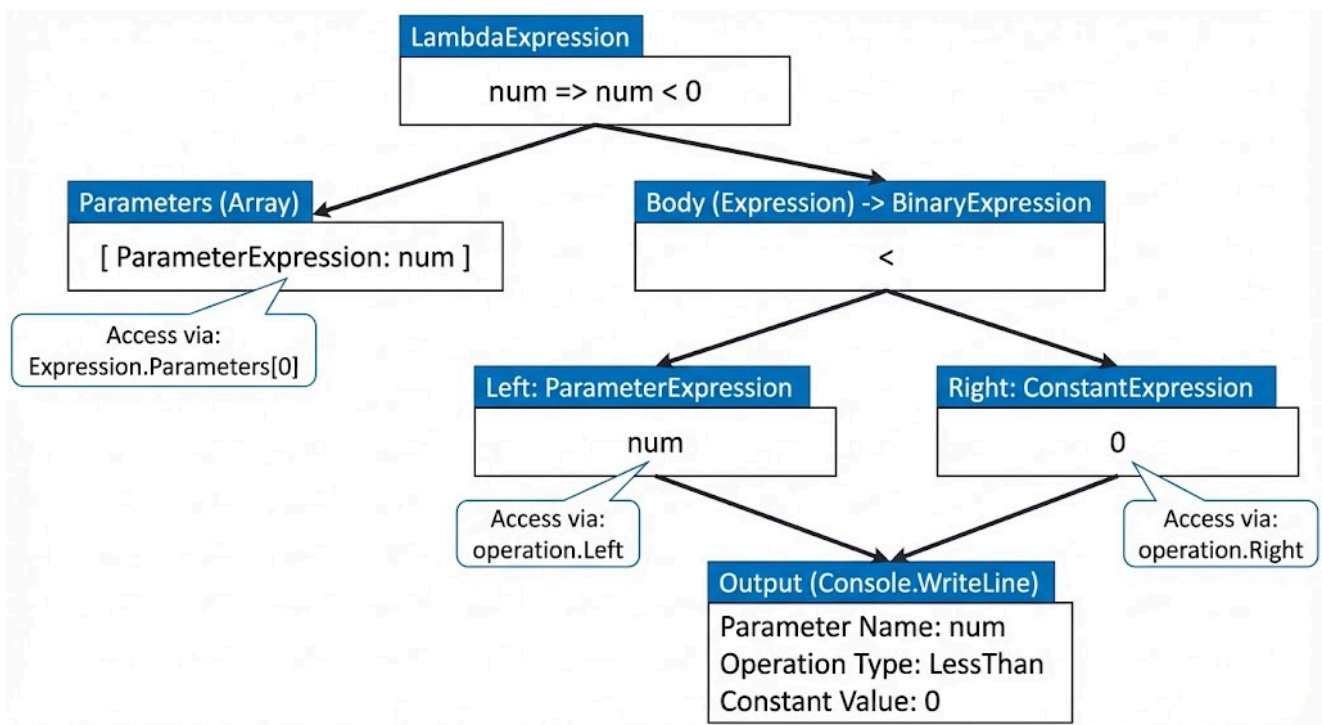
في مثالنا `num => num % 2 == 0`

- العملية كلها عبارة عن **LambdaExpression**.
- جواها **Body** (جسم الدالة) و **Parameters**.
- الـ Body عبارة عن مقارنة `==` (Binary Expression).
- الطرف اليمين `0` (Constant).
- الطرف الشمال `num % 2` (Binary Expression ثاني عبارة عن Modulo).



5. الـ Demo 1: تفكيك الـ Expression (Deocmpose/Parse)

تعالوا نشوف إزاي نقدر نقرأ الكود كـ Data ونفككه حته حته. هنعمل Expression بيشوف الرقم سالب ولا لا: `num < 0`.



C#

```
1 Expression<Func<int, bool>> isNegativeExpression = num => num < 0;
```

خطوات التفكيك:

1. استخراج الباراميتر (num):

C#

```
1 // Parameters عبارة عن Array واحد أكثر من واحد
2 ParameterExpression param = isNegativeExpression.Parameters[0];
3 // param.Name سيكون "num"
4 // param.Type سيكون int
```

2. استخراج العملية (Body): الـ Body يتألف من الـ Lambda هو `num < 0` ، وده نوعه `BinaryExpression` . بس الخاصية `Body` بترجع `Expression` عام، فلازم نعمل **Casting**.

C#

```
1 BinaryExpression operation =
  (BinaryExpression)isNegativeExpression.Body;
```

3. تفكيك العملية (Left & Right): أي عملية ثنائية ليها طرف يمين وطرف شمال.

- الطرف الشمال (Left): هو `num` (وده `Parameter`).
- الطرف اليمين (Right): هو `0` (وده `Constant`).

C#

```
1 ParameterExpression left = (ParameterExpression)operation.Left;
```

```
2 ConstantExpression right = (ConstantExpression)operation.Right;
```

4. طباعة التفكير:

C#

```
1 Console.WriteLine($"Parameter Name: {left.Name}"); // num
2 Console.WriteLine($"Operation Type: {operation.NodeType}"); //
  LessThan
3 Console.WriteLine($"Constant Value: {right.Value}"); // 0
```

ملحوظة: NodeType ده Enum فيه كل أنواع العمليات الممكنة (جمع، طرح، أكبر من، etc).

6. الـ Demo 2: بناء Expression من الصفر (Manual Construction)

دي أهم وأقوى جزئية. أحياناً ميكونش عندك الـ Lambda Expression جاهزة، إنت محتاج تبنيها بنفسك برمجياً (مثلاً بتعمل (Dynamic Query String Engine).

يعني إيه Dynamic Query String Engine؟

ببساطة، هو "محرك" أو كود ذكي بيقرر يترجم الكلام اللي اليوزر بيكتبه في البحث (أو في رابط الموقع URL) ويحوله لـ كود داتا بيز (LINQ Query) بشكل لحظي وديناميكي.

المشكلة فين أصلاً؟

في العادي، لما بتيجي تعمل فلتر (Search/Filter)، بتكتب الكود ثابت (Hardcoded). مثلاً بتقول: "هات لي الموظفين اللي سنهم أكبر من 20". `x => x.Age > 20` طيب لو اليوزر عاوز يفلتر بالاسم؟ أو بالمرتبة؟ أو بتاريخ التعيين؟ هل هتكتب `if condition` لكل عمود في الداتا بيز؟ طبعاً مستحيل!

الحل بالـ Expression Trees (زي ما اتشرح):

هنا بيجي دور الـ Engine اللي المهندس عمله. الفكرة إنك بتخلي اليوزر بيعت الفلتر كـ نص (String)، والإنجين ده يترجم النص لـ Expression Tree يفهمها الكود.

السيناريو بيمشي كالتالي:

1. مدخلات اليوزر (Query String): اليوزر بيبعت في الـ URL حاجة زي كده:

```
1 filter=Age gt 20 AND Name? eq 'Ahmed'
```

(يعني السن أكبر من 20 والاسم بيساوي أحمد). ده مجرد نص (String).

2. الـ Parsing (التحليل): الـ Engine بيقرأ النص ده ويفككه (زي ما عملنا Decompose في الشرح).

- بي فهم إن Age ده اسم العمود (Property).
- بي فهم إن gt يعني علامة أكبر من (Binary Expression).

- يفهم إن 20 دي القيمة (Constant).

3. بناء الـ **Expression Tree**: بدل ما نكون كاتيين الكود بايدنا، الـ Engine بيبنى الـ Tree في الرن تايم (Run-time) باستخدام `Expression.AndAlso` و `Expression.GreaterThan`.
4. التنفيذ: في الآخر، الـ Tree دي بتتحول لـ SQL تروح للداتا بيز تجيب البيانات المطلوبة بالطبط.

مثال (Demo2 (Manual Construction)

عايزين نبني الكود ده يدويًا: `num => (num % 2) == 0`

هنستخدم الـ **Factory Methods** الموجودة جوه كلاس `Expression` (طريقة البناء من تحت لفرق).

الخطوات بالتفصيل:

1. تعريف الباراميتر (`num`):

C#

```
1 // ونوعه "num" متغير اسمه
2 ParameterExpression numParam = Expression.Parameter(typeof(int),
  "num");
```

2. تعريف الثوابت (2 و 0):

C#

```
1 ConstantExpression twoConst = Expression.Constant(2, typeof(int));
2 ConstantExpression zeroConst = Expression.Constant(0, typeof(int));
```

3. بناء عملية باقي القسمة (`num % 2`): دي عملية `Modulo`، بتاخذ طرفين (الباراميتر، والاثنتين).

C#

```
1 // بيمثل BinaryExpression ده (num % 2)
2 BinaryExpression moduloExpr = Expression.Modulo(numParam, twoConst);
```

4. بناء عملية المساواة (`... == 0`): هنقارن ناتج الـ `Modulo` مع الصفر.

C#

```
1 // بيمثل BinaryExpression ده ((num % 2) == 0)
2 BinaryExpression equalExpr = Expression.Equal(moduloExpr, zeroConst);
```

5. تغليفهم في **Lambda Expression**: عشان يبقى قابل للتحويل لـ `Delegate`.

C#

```
1 // والباراميترز دي (equalExpr) ده Body بنقله خد الـ
```

```

2 Expression<Func<int, bool>> isEvenExpression =
3     Expression.Lambda<Func<int, bool>>(equalExpr, new
    ParameterExpression[] { numParam });

```

6. التنفيذ (Compile & Run):

C#

```

1 var isEvenFunc = isEvenExpression.Compile();
2 Console.WriteLine(isEvenFunc(10)); // True
3 Console.WriteLine(isEvenFunc(9));  // False

```

7. ملخص الفرق Construction vs Decomposition :

وجه المقارنة	التفكيك (Decomposition)	البناء (Construction)
الاتجاه	من كود (Lambda) -> إلى أجزاء (Nodes)	من أجزاء (Nodes) -> إلى كود (Lambda)
المعطيات	جاهز Expression	مجرد بيانات وأنواع (Types)
الأدوات المستخدمة	expr.Body , expr.Parameters , expr.Left	Expression.Parameter , Expression.Equal , Expression.Lambda
الاستخدام الأشهر	تحويل #C لـ SQL (زي في EF Core)	عمل فلاتر بحث ديناميكية (Dynamic Query Builder)
مثال المحاضرة	تحليل <code>num < 0</code>	بناء <code>num % 2 == 0</code>

الخلاصة:

- الـ **Decomposition**: عشان تقرأ وتفهم وتترجم الكود اللي اتكتب.
- الـ **Construction**: عشان تكتب كود ديناميكي بناءً على طلب اليوزر.

8 . الخاتمة والاستخدامات العملية

- الموضوع ممكن بيان صعب، بس هو أساسي في أي **Filtering** أو **Sorting** ديناميكي في المشاريع الكبيرة.
- كل الخوارزميات القوية بتعتمد عليه.
- مثال عملي: بناء محرك بحث بياخد الـ input من اليوزر ويعمل Query String ديناميكي بناءً على المدخلات.

Next Steps for You:

- ممكن تاخذ الكود بتاع "بناء الـ Expression يدوياً" وتجرب تضيف عليه شرط كمان (مثلاً: الرقم زوجي و أكبر من 5) باستخدام `Expression.AndAlso`.
- جرب تعمل Function عامة بتاخذ اسم الخاصية (Property Name) والقيمة، وترجع Expression ينفع يستخدم في `Where()` بتاع LINQ.

IQueryable vs IEnumerable

مقدمة (Introduction)

الموضوع النهاردة حساس ومهم جداً. كثير من المبرمجين بيستخدموا `IEnumerable` و `IQueryable` بس للأسف مش عارفين الفرق العميق بينهم (In-depth) وتفصيلهم اللي بتفرق في أداء التطبيق.

عشان نفهم الفرق ده عملياً، لازم نستخدم `Database` ونطبق بمثال حي باستخدام `Entity Framework Core (EF Core)`.

تجهيز بيئة العمل (Project Setup)

عشان التجربة تكون واضحة، المهندس عصام جهز Solution فيه 3 مشاريع:

1. الـ `Data Project`: فيه تعريف الداتابيز (Context) والـ `Entity`.
2. الـ `IEnumerable Demo`: عشان نجرب عليه.
3. الـ `IQueryable Demo`: عشان نجرب عليه.

شكل الداتابيز (Database Schema)

عندنا جدول بسيط للكتب `Books` فيه:

- الـ `ID`
- الـ `Title`
- الـ `Author`
- الـ `Price` الداتابيز اتملت بشوية بيانات (كتب) عشان نقدر نعمل عليها استعلامات (Queries).

ملحوظة: الكود المكتوب بيشرح لغة `#C`، لكن الداتابيز (SQL Server) مبنفهمش `#C`، هي بتفهم `SQL` بس. الكومبايلر بتاع `#C` هو الوحيد اللي بيشرح كود `Linq` اللي بنكتبه.

1 السيناريو الأول: استخدام IEnumerable

في السيناريو ده، هنشوف إيه اللي بيحصل لما نستخدم `IEnumerable` عشان نفلتر البيانات.

الكود المستخدم (The Code)

C#

```
1 // اتصال بالداتابيز
2 var context = new BookContext();
3
```

```

4 // 1. تعريف المتغير كـ IEnumerable
5 IEnumerable<Book> books = db.Books;
6
7 // 2. تطبيق الفلتر (Where)
8 var filteredBooks = books.Where(x => x.Price > 50);
9
10 // 3. طباعة النتيجة
11 foreach (var item in filteredBooks)
12 {
13     Console.WriteLine($"Item: {item.Title} - Price: {item.Price}");
14 }

```

التحليل: إيه اللي حصل في الخلفية؟ (Under the Hood)

لما شغلنا الكود ده وفتحنا الـ **SQL Profiler** (الأداة اللي بتورينا الأوامر اللي راحت للداتابيز)، لقينا الآتي:

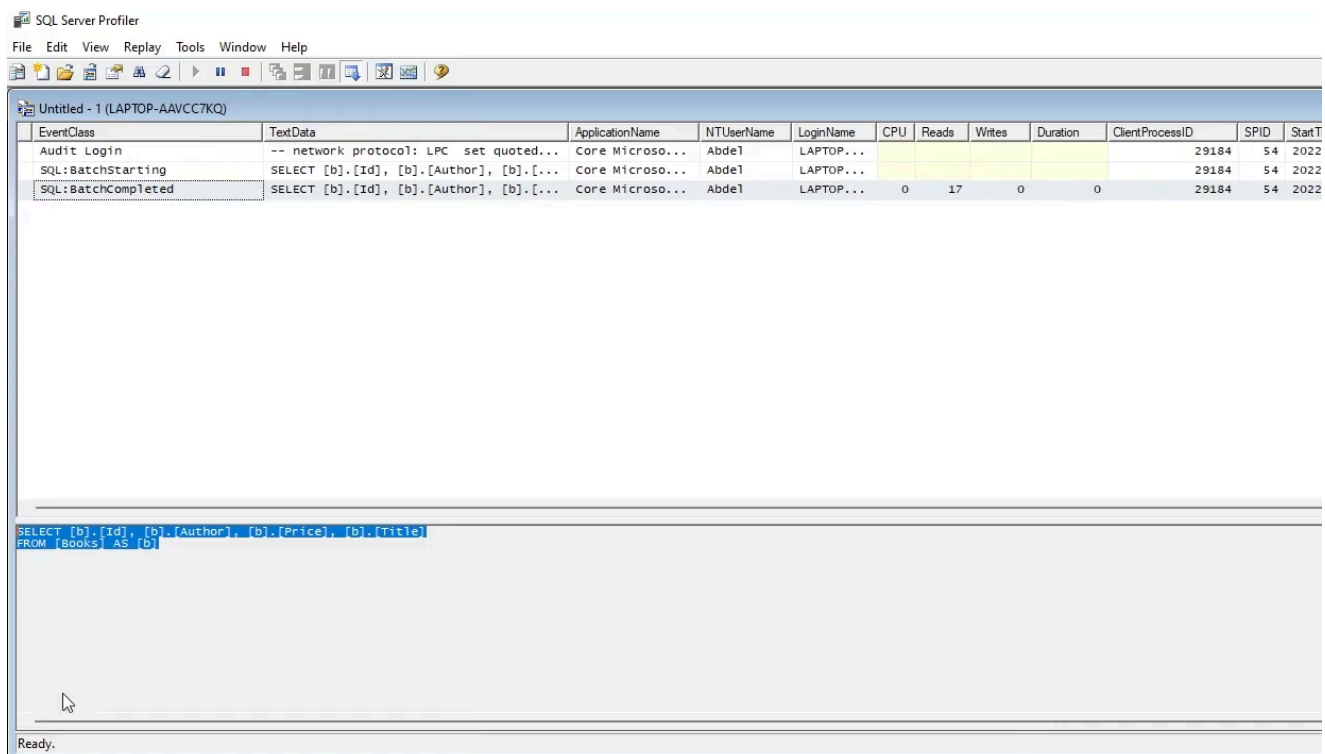
1. الجملة اللي اتنفذت في الـ SQL:

SQL

```

1 SELECT [b].[Id], [b].[Author], [b].[Price], [b].[Title]
2 FROM [Books] AS [b]

```



الملاحظة الخطيرة: الجملة دي رجعت كل الكتب من الداتابيز، مفيش **WHERE clause** راحت للداتابيز!

2. مكان المعالجة (Client-side Evaluation):

- الـ EF Core راح جاب كل البيانات من الجدول وحطها في الـ **Memory** (الذاكرة عند التطبيق بتاعك).
- بعد ما البيانات جت في الميموري، بدأ يطبق عليها الفلتر **Where(x => x.Price > 50)** واحد واحد.

3. ترتيب التنفيذ:

- الكود يستخدم **Deferred Execution** (التنفيذ المؤجل).
- لحظة الـ `foreach` ، هو راح جاب البيانات كلها، وبعدين بدأ يمشى عليهم بـ `Iterator` (عداد) يختبر شرط السعر داخل الميموري.

⚠ **تحذير:** لو بتستخدم `IEnumerable` مع داتابيز، أنت بتسحب كل الجدول عندك في الميموري وبعدين تفلتر. ده ممكن يكون كارثة لو البيانات حجمها كبير.

2 السيناريو الثاني: استخدام IQueryable

دلوقتي هنغير نوع المتغير لـ `IQueryable` ونشوف الفرق.

الكود المستخدم (The Code)

C#

```
1 // نفس الاتصال بالداتابيز
2 var context = new BookContext();
3
4 // 1. تعريف المتغير كـ IQueryable (موجود في System.Linq)
5 IQueryable<Book> books = db.Books;
6
7 // 2. تطبيق الفلتر (Where)
8 // مش مجرد دالة Expression Tree هنا الفلتر عبارة عن
9 var filteredBooks = books.Where(x => x.Price > 50);
10
11 // 3. طباعة النتيجة
12 foreach (var item in filteredBooks)
13 {
14     Console.WriteLine($"Item: {item.Title} - Price: {item.Price}");
15 }
```

التحليل: إيه اللي حصل في الخلفية؟

لما فتحنا الـ **SQL Profiler** المرة دي، النتيجة كانت مختلفة تماماً:

EventClass	TextData	ApplicationName	NTUserName	LoginName	CPU	Reads	Writes	Duration	ClientProcessID	SPID	StartTime
Audit Login	-- network protocol: LPC set quoted...	Core Microso...	Abdel	LAPTOP...					29624	54	2022-
SQL:BatchStarting	SELECT [b].[Id], [b].[Author], [b]....	Core Microso...	Abdel	LAPTOP...					29624	54	2022-
SQL:BatchCompleted	SELECT [b].[Id], [b].[Author], [b]....	Core Microso...	Abdel	LAPTOP...	0	19	0	1	29624	54	2022-


```
SELECT [b].[Id], [b].[Author], [b].[Price], [b].[Title]
FROM [Books] AS [b]
WHERE [b].[Price] > 50.0
```

Ready.

1. الجملة اللي اتنفذت في الـ SQL:

```
1 SELECT [b].[Id], [b].[Author], [b].[Price], [b].[Title]
2 FROM [Books] AS [b]
3 WHERE [b].[Price] > 50 -- شاييف الفرق !
```

الملاحظة: الفلترة تمت جوه الداتابيز (Server-side Evaluation).

2. إزاي ده حصل؟ (The Magic of Provider):

- الـ **IQueryable** مش بس شاييل داتا، ده شاييل حاجة اسمها **Expression Tree**.
- الكود بتاعك `x => x.Price > 50` ماتنفذش ككود **C#** مباشر، لكن اتحول لشجرة تعبيرات (وصف للكود).
- حاجة اسمها **Query Provider** (في حالتنا هو **EF Core**) أخذ الشجرة دي وترجمها للغة اللي بتفهمها الداتابيز (SQL).
- بعث الجملة المترجمة للـ **SQL Server**، والسرقر رجع بس البيانات اللي بتحقق الشرط.

100 %				
Results Messages				
	Id	Author	Price	Title
1	10	Herbert Schildt	52.12	Java: The Complete Reference
2	11	Joan Casteel	167.53	Oracle 11g
3	13	C++ Programming Language	105.18	C++ Programming Language
4	15	David Flanagan	53.32	JavaScript: The Definitive
5	19	Fullstack Vue	51.07	Fullstack Vue
6	22	Jeremy Wilken	59.99	Angular in Action

السؤال الجوهرى

لما الـ **IQueryable** بتجيب داتا مفلتر (Filtered Data) وتدخل الميموري، هل بتعمل **Lazy Loading** بالمعنى الحرفي؟ ولا بتشتغل إزاي جوه الـ **foreach** ؟

التصور الخاطئ (The Myth)

إن الداتا بنتجي من الداتابيز، تتخزن كلها في ليستة في الميموري، وبعدين الـ Loop تشتغل عليها.

الكلام ده بيحصل بس لو عملت `()ToList` صراحة قبل اللوب.

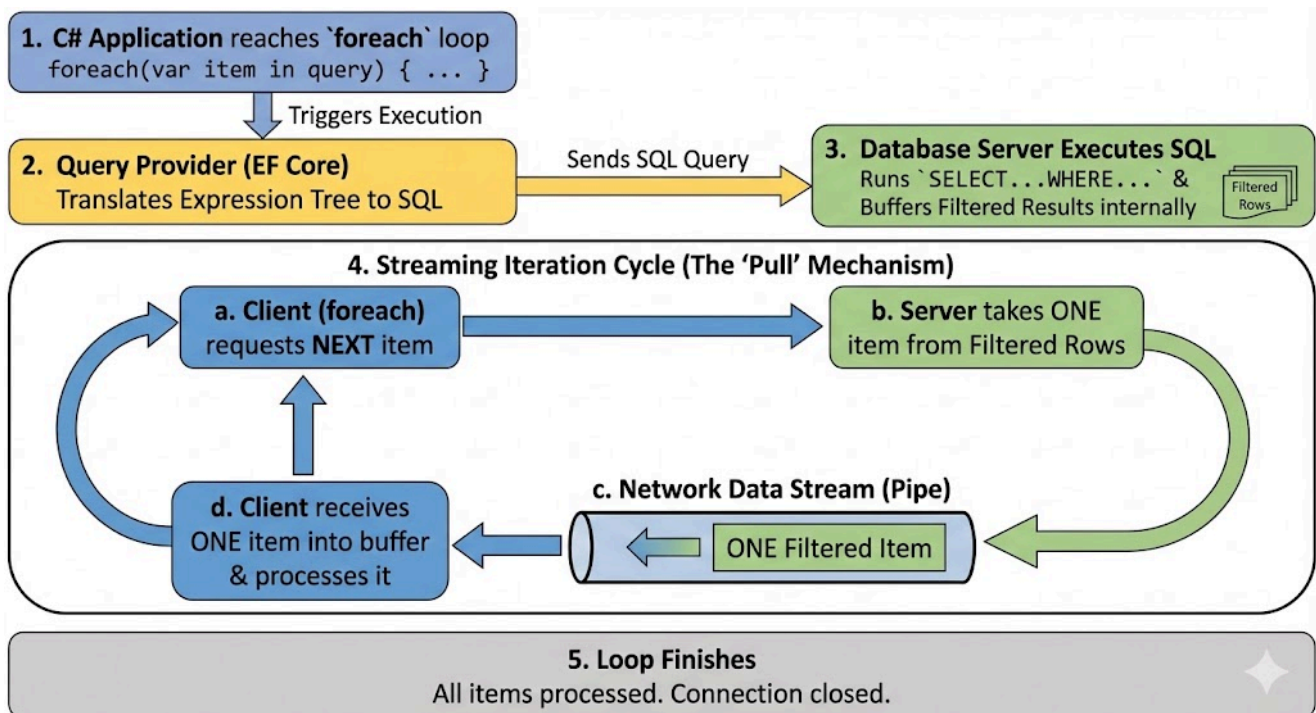
✓ الحقيقة: الـ Streaming (التدفق)

في حالة `IQueryable`، العملية مش تحميل كامل (Buffering) ولا هي Lazy Loading بتاع الـ Navigation Properties. هي حاجة تالفة اسمها **Streaming**.

إيه اللي بيحصل "تحت الغطاء" جوه الـ foreach ؟

1. لحظة الصفر: أول ما الكود يلمس الـ `foreach`، الـ Query بتتحول لـ SQL وتتبع للسيرفر.
2. فتح الماسورة (Open DataReader): الداتابيز بتنفذ الفلتره عندها، وبتبدأ تجهز النتائج. التطبيق بيفتح "ماسورة" (Connection) مع الداتابيز.
3. نظام بالقطعة (Iteration):
 - الـ Loop تطلب أول لفة ← الـ App يسحب أول صف بس من الماسورة للميموري، في كل لفة من لغات الـ Loop (لما الكومبيلر ينادي `Enumerator.MoveNext()`)، هو فعلياً بينادي `DataReader.Read()` اللي بتجيب الصف التالي من الـ Buffer بتاع الشبكة.
 - تخلص اللفة وتطلب الثاني ← يسحب ثاني صف من الماسورة المفتوحة.
 - وهكذا لحد ما النتائج تخلص أو إنت تعمل `break`.

الزئونة: الداتا بتوصل الميموري وهي مفلتره جاهزة، وبتوصل واحد ورا واحد، مش دفعة واحدة.



IQueryable	IEnumerable	وجه المقارنة
Server-side: الداتابيز بتفلتر وتبعت الخلاصة.	Client-side: بيسحب الجدول "خام" ويفلتره في الميموري.	مكان الفلتره

وجه المقارنة	IEnumerable	IQueryable
استهلاك الميموري	عالي: يبحر مكان للداتا كلها قبل الفلتر.	منخفض: يبحر مكان للصف الحالي اللي عليه الدور بس.
أسلوب الجلب	Buffering (غالباً)	Streaming (تدفق)
التشبيه	بتشتري قفص طماطم بحاله وتنقي السليم في البيت.	بتطلب من البياح ينقلك السليم ويبيعتهلك حبة حبة.

⚠️ فخ الـ Connection Pool (تحذير مهم)

بما إن الـ Streaming معتمد إن "الماسورة تفضل مفتوحة" طول ما الـ `foreach` شغالة:

- المشكلة: لو الكود اللي جوه اللوب بياخد وقت طويل (مثلاً بتبعت إيميل لكل يوزر، أو بتكلم API ثانية)، أنت كده حاجز الـ Database Connection ومعطله معاك.
- النتيجة: لو فيه ترافيك عالي، الـ Connections هتخلص ويحصل Connection Pool Exhaustion.
- الحل: في العمليات الطويلة، أعمل `.ToList()` الأول (اقفل الماسورة وهات الداتا عندك)، وبعدين لوب براحتك في الميموري.

💡 الخلاصة

- هل بيجيب الداتا كلها الأول؟ لا، بيجيبها Stream (واحد ورا الثاني).
- هل بيعمل Logic في الميموري؟ لا، الداتا بتوصل مفلتره وجاهزة من السيرفر.
- الميزة: توفير ميموري + سرعة استجابة (لأن أول نتيجة بتوصل فوراً).

مقارنة تفصيلية: IQueryable ضد IEnumerable

وجه المقارنة	IEnumerable	IQueryable
المكان المناسب	التعامل مع (LINQ to Objects Arrays, Lists) ميموري	التعامل مع داتا بيز (LINQ to SQL / Entities) خارجية
مكان الفلتر	في الميموري بعد سحب (Client-side الداتا)	(في الداتا بيز قبل سحب الداتا) Server-side
طريقة العمل	بيستخدم <code><Func<T, bool></code> (Delegate بينفذ كود)	بيستخدم <code><<Expression<Func<T, bool></code> (شجرة توصف الكود)
الامتداد (Extension)	موجود في كلاس Enumerable	موجود في كلاس IQueryable (وييرث من IEnumerable)
مكونات الانترفيس	فقط (للمرور على العناصر) GetEnumerator	ElementType , Expression , Provider

الفروقات في تعريف الدوال (Method Signatures)

لو بصيت على دالة Where في الـ LINQ هتلاقي فرق جوهري:

- في حالة IEnumerable:

```
1 // بتأخذ دالة تنفيذية عادية
2 public static IEnumerable<T> Where(this IEnumerable<T> source, Func<T,
  bool> predicate)
```

هنا هو بياخد "طريقة التنفيذ" و هينفذها بنفسه.

- في حالة IQueryable:

C#

```
1 // بتأخذ تعبير (وصف للكود)
2 public static IQueryable<T> Where(this IQueryable<T> source,
  Expression<Func<T, bool>> predicate)
```

هنا هو بياخد "شجرة" بتوصف الكود، عشان الـ Provider يترجمها لـ SQL.

💡 الخلاصة والنصيحة النهائية

1. لو بتكلم داتا بيز (Remote Data Source): استخدم IQueryable. ده هيخلي الفلترة تحصل في الداتا بيز، وتقل البيانات اللي راجعة عبر الشبكة، وده أفضل بكتير لـ Performance.
2. لو الداتا أصلاً عندك في الـ ميموري (In-Memory Collection): استخدم IEnumerable. مفيش داعي للتعقيد بتاع الـ Expression Trees لأن الداتا كده عندك.
3. الـ IQueryable هو امتداد للـ IEnumerable: يعني الـ IQueryable بيورث من IEnumerable، بس بيزود عليه قدرات الترجمة للغات تانية زي SQL.

Converting Data Types(AsEnumerable, AsQueryable, Cast, OfType)

1. مقدمة ومفهوم الـ Conversion Methods

الـ Conversion Methods هي مجموعة من الـ Methods موجودة جوا الـ Namespace الذي اسمه `System.Linq`. الـ Methods دي بتديك مرونة عالية جداً أثناء التطبيق عشان تحول من Type لـ Type تاني.

قاعدة التسمية (Naming Convention) وفكرة التنفيذ

مايكروسوفت مسمية الـ Methods دي بطريقة معينة عشان تعرف سلوك الـ Method بمجرد قراءة اسمها:

القاعدة الذهبية

- الـ **Methods** تبدأ بكلمة **As** (زي **AsEnumerable**): التنفيذ بتاعها بيكون **Deferred Execution** (مؤجل). يعني الكود مش هيتنفذ غير لما تعمل **Enumerate** (زي **foreach** أو **ToList**).
- الـ **Methods** تبدأ بكلمة **To** (زي **ToList**, **ToArray**): التنفيذ بتاعها بيكون **Immediate Execution** (فوري). يعني بتنفذ الكويري في لحظتها وترجع النتيجة.

احنا هنتعرف النهاردة على 8 Methods مختلفة.

2. تجهيز بيئة العمل (Project Setup)

عشان نجرب الـ 8 methods، المهندس عصام عمل مشروع تجريبي بيحاكي نظام شحن (Shipping System) فيه وراثة (Inheritance).

هيكل الكلاسات (Class Hierarchy)

- الـ **Shipping (Base Class)**:
 - فيها **protected constructor** عشان محدش يقدر يعمل منها **Object** غير الـ **Child Classes**.
 - البيانات: **UniqueId**, **Sender**, **ShippingDate**, **Description**.
 - فيها **Method** اسمها **Start (Virtual)** بتطبع تفاصيل الشحنة.
- الـ **AirShipping**: بتورث من **Shipping** وبتعمل **Override** لـ **Description**.
- الـ **GroundShipping**: بتورث من **Shipping**.
- الـ **OceanShipping**: شحن عبر المحيطات.
- الـ **RailShipping**: شحن عبر القطارات.

الـ Repository والبيانات

- عندنا كلاس **ShippingRepository** فيه ليستة من 12 شحنة (Shipping).
- البيانات متنوعة: شحنات امبارح، شحنات اليوم، شحنات بكرة، وأنواع مختلفة (Air, Ground, etc).

كلاس الـ Wrapper المخصص (<ShippingList<T)

عشان نفهم فائدة `AsEnumerable` ، المهندس عمل كلاس مخصص بيورث من `<List<T` .

C#

```
1 // ده كلاس مخصص عشان يوضح فكرة تعارض الميثودز
2 public class ShippingList<T> : List<T>
3 {
4     // بنفس اسم ميثود الـ link Where عملنا ميثود اسمها
5     public IEnumerable<T> Where(Func<T, bool> predicate)
6     {
7         Console.WriteLine("Custom Where Implementation inside
ShippingList");
8         // هنا بيعمل فلتر بسيط بس بيطبع جملة قبلها
9         // الكود الفعلي للفلتر بيكون هنا
10        return base.FindAll(new Predicate<T>(predicate));
11    }
12 }
```

- الملاحظة المهمة: الكلاس ده فيه `Method` اسمها `Where` . والـ `LINQ` أصلاً فيه `Extension Method` اسمها `Where` . يا ترى الكود هينادي على مين فيهم؟ ده اللي هنشوفه.

📌 ملاحظة هامة: هيكله كلاس `ShippingList` والغرض منه

1. الوراثة من `List (Inheritance)`:

الكلاس ده `<ShippingList<T` وارث من `<List<T` ، وده معناه:

- انه **"Is-a" List**: هو عبارة عن قائمة عادية جداً تمتلك كل صفات ووظائف الـ `List` القياسية.
- القوة: نقدر نستخدم معاه كل أوامر الليست العادية (`Add` , `Remove` , `Indexer`) .

2. الغرض من الـ `Custom Where Method`:

المهندس ضاف ميثود اسمها `Where` بنفس اسم ميثود الـ `LINQ` ، والهدف مش بس إضافة خاصية، ولكن هدف تعليمي بحت:

- إحداث تعارض (**Name Collision**): عشان يخلي فيه دالتين بنفس الاسم (واحدة جوه الكلاس وواحدة في الـ `LINQ`).
- إثبات الأولوية: عشان يثبت إن الكومبايلر دائماً بيختار "الميثود اللي جوه الكلاس" (`Instance Method`) وبيهمل "ميثود الـ link" (`Extension Method`) إلا لو أجبرناه باستخدام `AsEnumerable` .

3. الاستفادة (**Benefits**):

بناءً على استخدام `AsEnumerable` لحل هذا التعارض:

- 1. الـ **Code Readability**: الكود بيبقى أوضح ومقروء أكثر.
- 2. الـ **Flexibility**: بتقدر تغير سلوك الكود (`Polymorphism`) وتختار أي `Implementation` للـ `Where` عايز تستخدمه في سطر واحد.

3. شرح الـ Methods بالتفصيل

أولاً: AsEnumerable

- الوظيفة: بترجع الـ Input Type كـ `<IEnumerable<T>`.
- النوع: Deferred Execution (لأنها تبدأ بـ `As`).

المثال العملي: حل مشكلة تعارض الـ Methods

لو عندنا Object من نوع `ShippingList` (اللي احنا عملناه فوق) وكتبنا `.Where` ، الكومبايلر هينادي على الـ `Where` بتاعة الكلاس نفسه مش بتاعة الـ `LINQ`.

الكود قبل استخدام `AsEnumerable` :

C#

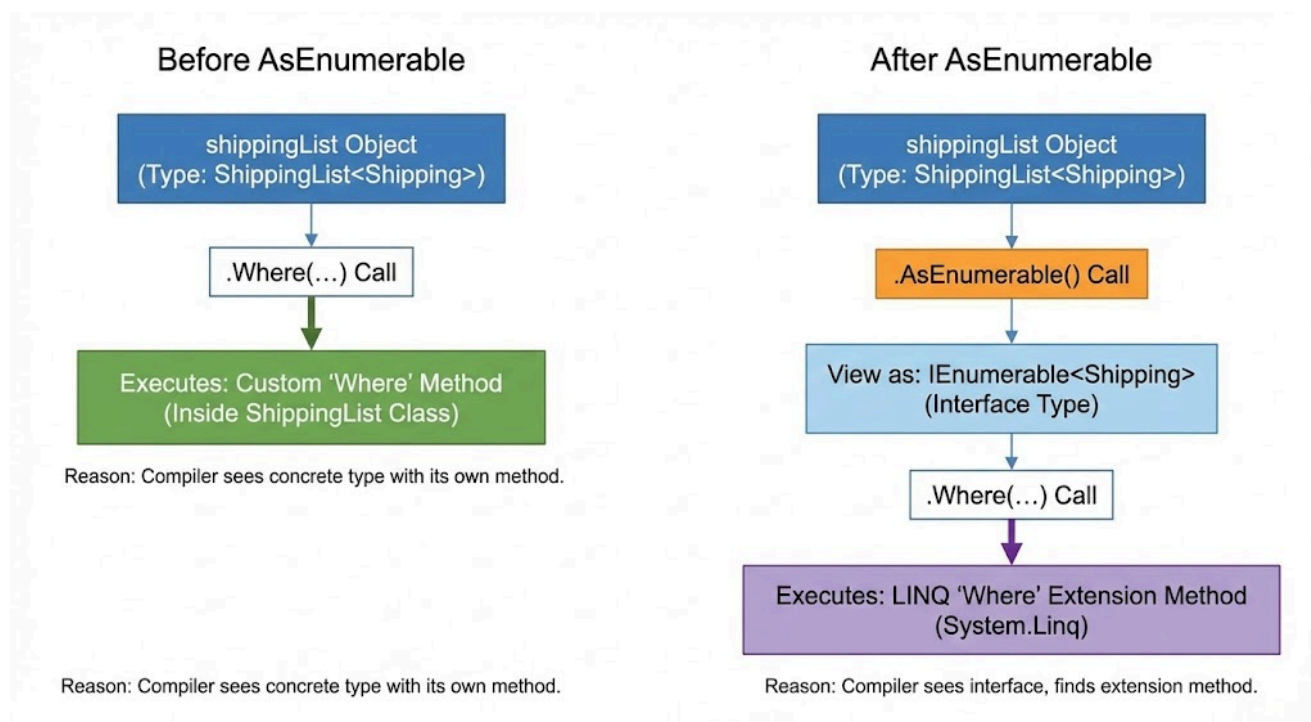
```
1 // ShippingList is the custom class inheriting from List<T>
2 ShippingList<Shipping> shippingList =
  ShippingRepository.AllAsShippingList;
3
4 IEnumerable todayShipping = shippingList.Where(x=>x.ShippingDate ==
  DateTime.Today);
5 // وهيطبع الجملة اللي احنا كاتبينها ShippingList اللي جوه Where النتيجة: هينادي على الـ
```

الكود بعد استخدام `AsEnumerable` : عشان أجبره يستخدم الـ `Where` بتاعة الـ `LINQ`، لازم أحول النوع لـ `IEnumerable` الأول.

C#

```
1 // الأول IEnumerable بنحول الليسته لـ
2 IEnumerable<Shipping> todayShipping = shippingList.AsEnumerable()
3     .Where(x => x.Date == DateTime.Today);
4
5 // الـ Where بتاعة الـ System.Linq (Extension Method) النتيجة: هينادي على الـ
6 // ومش هيطبع الجملة المخصصة اللي في الكلاس
```

في الاستفادة 1. **Code Readability**: الكود بيبقى أوضح ومقروء أكثر. 2. **Flexibility**: بتقدر تغير سلوك الكود (Polymorphism) وتختار أي **Implementation** للـ `Where` عايز تستخدمه في سطر واحد.



توضيح الاستخدامات من Microsoft Documents

ملاحظات (Remarks)

- الميثود `AsEnumerable<TSource>(IEnumerable<TSource>)` فعلياً مابتملش أي حاجة في الـ runtime غير إنها بتغير نوع الـ source وقت الـ compilation (compile-time type). هي بتحول النوع من أي class بيعمل `implement <T> IEnumerable` وتخليه بيان كأنه `<T> IEnumerable` بس.
- ممكن تستخدم `AsEnumerable` عشان تختار طريقة تنفيذ الـ Query (الاستعلام). ده بيفيد لما يكون عندك sequence (زي لسته أو كولكشن) بيعمل `implement <T> IEnumerable` ، بس في نفس الوقت الـ class ده جواه methods خاصة بيه بنفس أسامي الـ LINQ methods العادية.

مثال للتوضيح:

- افرض عندك class اسمه `Table` ، الـ class ده بيعمل `implement <T> IEnumerable` وعنده methods خاصة بيه زي `Where` و `Select` و `SelectMany`.
- لو جيت كتبت كود وندهت على `Where` علطول من الـ `Table` ، الكود هينفذ الـ method اللي جوه الـ `Table` نفسه (مش الـ extension method بتاعة LINQ).
- لو الـ `Table` ده بيمثل جدول في داتابيز (Database)، غالباً الـ `Where` بتاعته بتاخذ الـ predicate كـ "Expression Tree" وبتحوله لـ `SQL` عشان يتنفذ جوه السيرفر (Remote Execution).
- المشكلة:** افرض إنك مش عايز ده يحصل؟ مثلاً لو الشرط (predicate) اللي انت كاتبه بينادي على method محلية (local method) عندك في الكود، ودي طبعاً ماينفعش تتحول لـ `SQL`.
- الحل:** هنا تستخدم `AsEnumerable`. لما بتكتبها، انت كده بتخبي الـ methods الخاصة بتاعة الـ `Table` ، وبتجبر الـ compiler إنه يستخدم الـ standard query operators (بتاعة LINQ العادية) عشان ينفذ الكود عندك في الميموري (LINQ to Objects) بدل ما يرميه على الداتابيز.

ثانياً: AsQueryable

- الوظيفة: بتحول الـ Collection لـ IQueryable .
- النوع: Deferred Execution.
- الفرق الجوهرى:
- الـ IEnumerable : يتعامل مع (Delegates) Func<T, bool> .
- الـ IQueryable : يتعامل مع (Expression Trees) Expression<Func<T, bool>> .

المثال العملي:

لما تحول الداتا لـ AsQueryable ، اللينك بيتعامل مع الكود كـ "Data" (بيانات قابلة للتحليل) مش مجرد كود للتنفيذ.

C#

```
1 ShippingList<Shipping> shippingList =
  ShippingRepository.AllAsShippingList;
2
3 IQueryable<Shipping> todayShipping = shippingList.AsQueryable()
4   .Where(x => x.Date == DateTime.Today);
5
6 // Expression Tree عشان نتأكد إنها بقت
7 Console.WriteLine(todayShipping.Expression.ToString());
```

النتيجة في الكونسول: هيطبع لك شكل الـ Expression نفسه:

System.Collections.Generic.List...Where(x => x.Date == ...) . لو مكنتش
AsQueryable (كانت IEnumerable عادية)، مكنتش هيكون ليها Property اسمها . Expression.

ثالثاً: <Cast<T>

- الوظيفة: بتعمل Casting لكل عناصر الـ Collection لنوع محدد T .
- الخطورة: لو فيه عنصر واحد بس مقدرش يتحول، الـ Runtime هيضرب Exception.

المثال العملي (الخطأ الشائع):

لو عندي ليستة فيها أنواع شحن مختلفة (Air , Ground , Ocean)، وحاولت أحولهم كلهم لـ GroundShipping .

C#

```
1 IEnumerable<Shipping> allShippings = ShippingRepository.AllAsList(); //
  فيها كل الأنواع
2
3 // محاولة تحويل كل الشحنات لـ GroundShipping
4 IEnumerable<Shipping> groundShippings = allShippings.Cast<GroundShipping>
  ();
5
6 foreach(var item in groundShippings)
7 {
```

```

8      // هنا عند أول عنصر مش Exception هيضرب
9      // الرسالة: InvalidCastException
10     }

```

ليه ده بيحصل؟ لأن الميثود Cast بتعمل Loop على كل العناصر وبتحاول تحول كل عنصر، مفيش فيها فلترة ضمنية. هي بتفترض إنك متأكد إن كل الليسته من النوع ده.

الحل التقليدي (قبل OfType): لازم تفلتر الداتا الأول تتأكد من نوعها، وبعدين تعمل Cast.

C#

```

1  var groundShippings = allShippings
2      .Where(x => x.GetType() == typeof(GroundShipping)) // فلتر الأول
3      .Cast<GroundShipping>(); // دلوقتي التحويل آمن

```

رابعاً: <OfT> (الحل الآمن)

- الوظيفة: بتعمل حاجتين في وقت واحد: فلترة (Filter) و تحويل (Cast).
- الميزة: Safe Casting. لو العنصر مش من النوع T ، بتجاهله ومبتضربش Exception. بترجع بس العناصر اللي نجح تحويلها.

المثال العملي:

C#

```

1  var allShippings = repository.GetAll();
2
3  // هاتلي بس الحاجات اللي نوعها
4  var groundShippings = allShippings.OfT<GroundShipping>();
5
6  // بس، وهتجاهل الباقي بدون أخطاء GroundShipping النتيجة: هترجع ليسته فيها الـ

```

الفرق التقني بين Cast و OfType

- الـ Cast : بتنفذ تحويل مباشر item(T) . لو فشل -> Exception.
- الـ OfType : بتستخدم المشغل is . الكود الداخلي بتاعها بيعمل كده: `if (item is T) return item(T);`

نهاية الجزء الأول. في الجزء الثاني هنكمل الـ Methods اللي بتعمل Immediate Execution (اللي بتبدأ بـ To) وهما:

- ToArray

- ToList
- ToDictionary (مهمة جداً وفيها تفاصيل)
- ToLookup

Converting Data Types(ToArray, ToList, ToDictionary, ToLookup)

4. الـ Immediate Execution Methods (تبدأ بـ To)

زي ما اتفقنا في البداية، أي Method بتبدأ بكلمة To معناها إن اللينك هينفذ الكويري حالياً (Right Now) ومش هيسنتى.

خامساً: ToArray

- الوظيفة: بتحول الـ Sequence أو الـ List لـ مصفوفة عادية (Array).
- نوع التنفيذ: Immediate Execution.
- الاستخدام: لو محتاج تتعامل مع داتا بخصائص الـ Array زي الـ Indexing الثابت أو الـ Performance في حالات معينة.

المثال العملي:

C#

```
1 // بننادي الـ Repository
2 IEnumerable<Shipping> shippings = ShippingRepository.AllAsList;
3
4 // التحويل لـ Array
5 Shipping[] shippingArray = shippings.ToArray();
6
7 // التعامل مع الخصائص
8 Console.WriteLine(shippingArray.Length); // Array uses Length
9
10 // الوصول للعناصر بالـ Index
11 // Error عشان ميعرض (Valid Range) موجود Index ملحوظة: لازم نتأكد إن الـ
12 Shipping firstItem = shippingArray[0];
13 firstItem.Start();
```

سادساً: ToList

- الوظيفة: بتحول الـ Collection لـ <List<T> .
- نوع التنفيذ: Immediate Execution.
- الفرق عن Array:
- بنستخدم Property اسمها Count بدل Length .
- مرونة أكثر في الإضافة والحذف بعد التحويل.

المثال العملي:

C#

```

1
2 IEnumerable<Shipping> shippings = ShippingRepository.AllAsList;
3
4 // التحويل لـ List
5 List<Shipping> shippingList = shippings.ToList();
6
7 // طباعة العدد
8 Console.WriteLine(shippingList.Count);
9
10 // الوصول للعناصر
11 // ممكن بالـ Index
12 Shipping item1 = shippingList[0];
13 // تانية LINQ methods أو باستخدام
14 Shipping item2 = shippingList.First();

```

سابعاً: ToDictionary

- الوظيفة: بتحول الـ List لـ قاموس (Dictionary) مكون من Key و Value .
- الأهمية: البحث في الـ Dictionary سيكون سريع جداً (Hash-based).
- شرط مهم: لازم تحدد مين هو الـ Key Selector. ولو فيه مفاتيح متكررة (Duplicate Keys)، الميثود دي بتضرب Exception (عكس الـ Lookup).

خلي بالك فيه أكثر من نوع Override لفنكشن ToDictionary:

1. النوع الأول: تحديد المفتاح فقط (Key Selector)

```

1 source.ToDictionary(x => x.PropertyToBeKey);

```

2. النوع الثاني: تحديد المفتاح والقيمة (Key & Element Selector)

```

1 // بنبت 2 باراميتز: الأول للمفتاح، والثاني للقيمة
2 source.ToDictionary(x => x.KeyProperty, x => x.ValueProperty);

```

مثال 1: تحويل بسيط (Key و Object كامل)

هنا هنخلي الـ UniqueId هو المفتاح، والـ Value هي الشحنة كلها.

C#

```

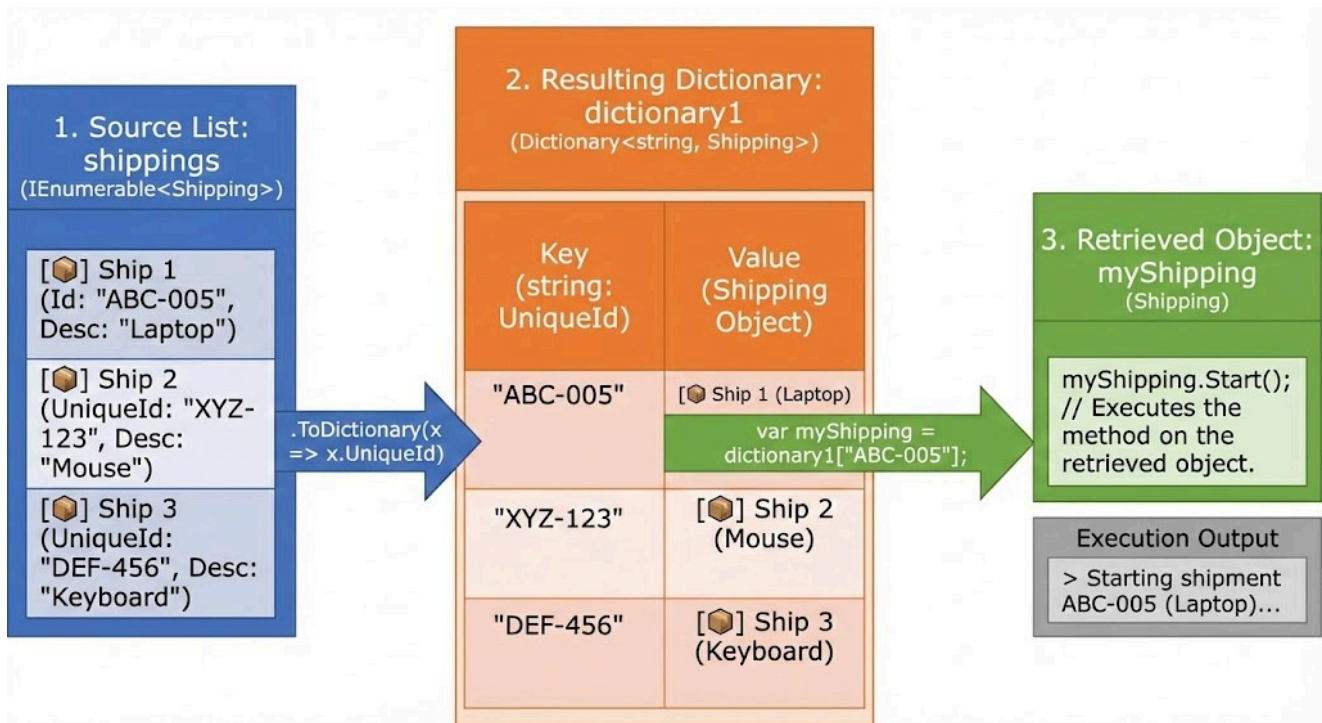
1 // shippings is IEnumerable<Shipping>
2 Dictionary<string, Shipping> dictionary1 = shippings.ToDictionary(x =>
  x.UniqueId);
3
4 // طريقة الاسترجاع

```

```

5  // هاتلي الشحنة اللي رقمها كذا
6  var myShipping = dictionary1["ABC-005"];
7  myShipping.Start(); // نفذ الميثود عليها

```



مثال 2: تحويل معقد (مع GroupBy)

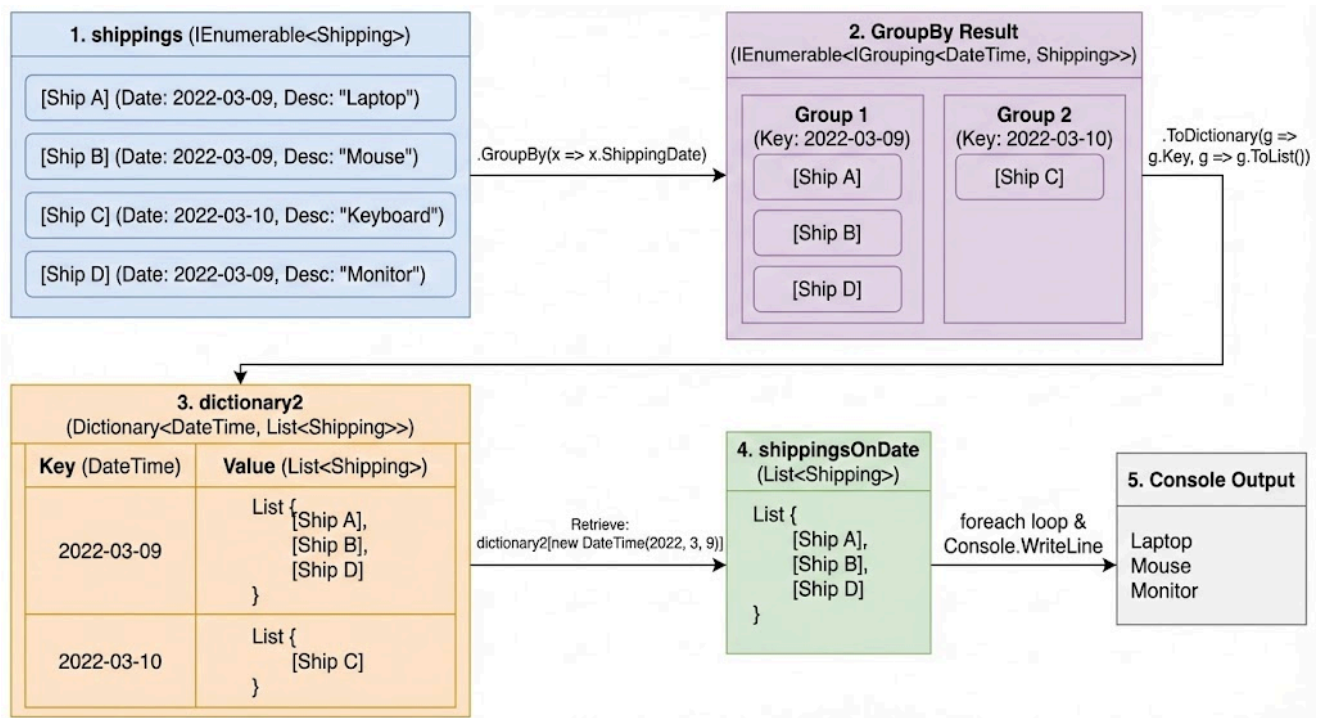
المهندس عصام طرح سيناريو: "عايز أعمل Dictionary المفتاح فيه هو التاريخ، والقيمة هي ليستة من الشحنات اللي حصلت في التاريخ ده".

C#

```

1  // 1. Grouping التاريخ حسب الشحنات الأول بنعمل
2  var dictionary2 = shippings.GroupBy(x => x.ShippingDate).ToDictionary(
3      g => g.Key,          // الـ Key هو التاريخ
4      g => g.ToList()      // الـ Value هي ليستة الشحنات
5  );
6
7  // الاستخدام: هاتلي كل شحنات يوم 9 مارس 2022
8  var targetDate = new DateTime(2022, 3, 9);
9  var shippingsOnDate = dictionary2[targetDate];
10
11 // نلف عليهم ونطبعهم
12 foreach(var ship in shippingsOnDate)
13 {
14     Console.WriteLine(ship.Description);
15 }

```



ثامناً: ToLookup

- **الوظيفة:** شبه الـ Dictionary جداً، بس بتسمح بإن المفتاح الواحد يكون له أكثر من قيمة (One-to-Many) بشكل افتراضي.
- **نوع التنفيذ:** Immediate Execution.
- **الفرق الجوهرى عن GroupBy :**
 - الـ GroupBy : تنفيذه مؤجل (Deferred).
 - الـ ToLookup : تنفيذه فوري (Immediate).
- **المميزات:**
 - مرونة عالية جداً.
 - آمن أكثر في التعامل مع الـ Exceptions مقارنة بالـ Dictionary.

المثال العملي:

نفس المثال اللي فات بتاع التاريخ، بس المرة دي باستخدام ToLookup ، الكود هيبقى أبسط وأكثر أماناً.

C#

```

1 // الأول، هو بيعمل ده ضمناً GroupBy مش محتاج أعمل
2 ILookup<DateTime, Shipping> shippingLookup = shippings.ToLookup(x =>
  x.ShippingDate);
3
4 // الاسترجاع
5 // على طول حتى لو عنصر واحد Collection ميزتها إنها بترجع
6 var items = shippingLookup[new DateTime(2022, 3, 9)];
7

```



```
8 // ممكن تجيب أول عنصر
9 var firstItem = items.First();
```

في خلاصة المقارنة بين **Dictionary** و **Lookup** المهندس أشار إن فيه حلقة كاملة للمقارنة بينهم، لكن باختصار في سياق التحويل:

- لو الـ Key فريد (Unique) وعاليز سرعة -> **Dictionary**. * لو الـ Key ممكن يتكرر (Grouping) أو محتاج أمان أكثر -> **Lookup**.

5. خاتمة الحلقة

- المهندس عصام أكد إن الـ **ToLookup** والـ **ToDictionary** بيطلعوا نفس النتيجة في حالة استخدام **GroupBy** قبل الديكشنري، لكن **ToLookup** مرن أكثر.

Custom LINQ Extension Methods

مقدمة: ليه محتاجين نعمل Custom Extension Methods؟

في الحلقات اللي فاتت (19 حلقة تقريباً) اتعلمنا دوال كثير في LINQ بتساعدنا في الـ Filtering و Projection و Aggregation. لكن ساعات بتيجي لحظات بنحتاج نضيف "التاتش" بتاعنا، لوجيك خاص بينا مش موجود في الـ Built-in methods.

وبما إن أغلب دوال الـ LINQ هي أصلاً عبارة عن **Extension Methods**، فإحنا كمان نقدر نعمل الدوال الخاصة بينا اللي بتخدم الشغل بتاعنا، وده بيخلي الكود:

- يكون **Readable**: مقروء أكثر.
- يكون **Clean**: خالي من التعقيد.
- يكون **Functional**: ماشي بمبدأ الـ Functional Programming.

ملاحظة مهمة: الحلقة دي بتعتمد جداً على فهمك لمبدأ الـ **Extension Methods** في الـ **C#**. لو مش عارفه، لازم تراجعها الأول (هو مشروح في فيديو سابق).

إعداد المشروع (Project Setup)

قبل ما ندخل في الكود، خلينا نشوف الداتا اللي هنتشغل عليها:

1. الـ **Employee Model**: كلاس بسيط فيه بيانات الموظف (الاسم، تاريخ التعيين، القسم، التأمين الصحي HealthInsurance، خطة التقاعد PensionPlan، والمرتب Salary).
2. الـ **Repository**: كلاس static بيرجع لينا 100 موظف ببيانات عشوائية عشان نجرب عليهم.
3. الـ **Existing Extension**: عندنا كلاس جاهز فيه دالة Print بتطبع بيانات الموظفين عشان نشوف النتيجة.

الكود المبدئي للاستدعاء:

C#

```
1 var employees = Repository.LoadEmployees(); // بنجيب 100 موظف
2 employees.Print("All Employees"); // Extension Method دالة طباعة معمولة بـ
```

السيناريو الأول: التصفح (Pagination) بالطريقة العادية

تخيل إن عندنا 100 موظف، وعايزين نقسمهم لصفحات، كل صفحة فيها 10 موظفين.

- المعادلة: عدد الصفحات = $100 / 10 = 10$ صفحات.
- أدوات LINQ: هنستخدم Skip (عشان نتخطى عدد معين) و Take (عشان ناخذ عدد معين).

المعادلة الديناميكية للـ Pagination:

عشان نجيب داتا لصفحة معينة، المعادلة بتكون:

$$Skip = (PageNumber - 1) * PageSize$$

$$Take = PageSize$$

مثال عملي:

- لو عايز الصفحة الأولى (Page 1): $0 = 0 * (1 - 1)$ (مش هنتخطى حاجة).
- لو عايز الصفحة الثانية (Page 2): $10 = 10 * (2 - 1)$ (هنتخطى أول 10 وناخد اللي بعدهم).

الكود العادي (بدون Custom Extension):

C#

```
1 int page = 1;
2 int pageSize = 10;
3
4 var page1 = employees
5     .Skip((page - 1) * pageSize) // المعادلة هنا
6     .Take(pageSize);
7
8 page1.Print("Page 1");
```

المشكلة هنا: الكود شكله بقى معقد شوية (Skip ومعادلات ضرب وطرح)، وشكله مش مريح للعين (Not Readable)، ولو حبيت أكرره في أماكن تانية هكتب نفس السطور تاني.

السيناريو الثاني: الحل باستخدام Custom Extension Method

هنعمل دالة إحنا اللي مألفينها اسمها Paginate ونخليها Extension Method عشان نناديها كأنها جزء من LINQ.

خطوة 1: إنشاء الـ Extension Class

هنعمل كلاس جديد لازم يكون static ، وليكن اسمه IEnumerableExtensions . ونعمل جواه دالة Paginate .

خطوة 2: استخدام الـ Generics

بما إن التصفح (Pagination) ده لوجيك عام ممكن نحتاجه مع Employee أو Customer أو Order ، فلازم نستخدم (<T>) Generics عشان الدالة تشتغل مع أي نوع داتا.

شكل الدالة المبدئي:

C#

```
1 public static class IEnumerableExtensions
```

```

2  {
3      // TSource: عشان نقبل أي نوع داتا
4      public static IEnumerable<T> Paginate<T>(this IEnumerable<T> source,
5      int page, int pageSize)
6      {
7          // اللوجيك لسه هنكتبه
8          return source;
9      }
10 }

```

خطوة 3: إضافة الـ Validation (Sanity Check)

قبل ما نكتب كود الـ Pagination، لازم نتأكد إن القيم اللي جاية من اليوزر منطقية.

1. التحقق من الـ Source: لو الـ Source بـ null، نضرب `ArgumentNullException`.
2. التحقق من رقم الصفحة (Page): لو اليوزر باعت صفحة رقم 0 أو بالسالب، عندنا مدرستين:
 - مدرسة تضرب `Exception`.
 - مدرسة تعمل `Reset` للقيمة وتخليها 1 (وده اللي هنمشي عليه عشان نكون مرنين).
3. التحقق من حجم الصفحة (PageSize): لو القيمة صفر أو سالب، نرجعه للـ Default (مثلاً 10).
4. التحقق لو الليست فاضية: لو الـ Source مفهوش عناصر، نرجع `Empty` ومفيش داعي نكمل تنفيذ.

كود الدالة بالكامل بعد التنظيم:

C#

```

1  public static IEnumerable<T> Paginate<T>(this IEnumerable<T> source, int
2  page, int pageSize)
3  {
4      // 1. Sanity Check for Source
5      if (source == null)
6      {
7          throw new ArgumentNullException(nameof(source));
8      }
9
10     // 2. Validate Page Number (Reset logic)
11     if (page <= 0)
12     {
13         page = 1; // Default to first page
14     }
15
16     // 3. Validate Page Size
17     if (pageSize <= 0)
18     {
19         pageSize = 10; // Default size
20     }
21
22     // 4. Check if source is empty (Performance optimization)

```

```

22     if (!source.Any())
23     {
24         return Enumerable.Empty<T>();
25     }
26
27     // 5. The Core Logic
28     return source.Skip((page - 1) * pageSize).Take(pageSize);
29 }

```

الاستخدام الآن:

C#

```

1  var page1 = employees.Paginate(1, 10);
2  page1.Print("Page 1 with Extension");
3
4  var page7 = employees.Paginate(7, 10); // 70 لـ 61 هيجيب من الموظف
5  page7.Print("Page 7");
6
7  // you can use pipeline
8  employees.Paginate(1, 10).Print("page 1");

```

تحسين إضافي: استخدام Nullable Parameters

يمكن نخلي الـ `page` و `pageSize` يقبلوا `null` ، بحيث لو اليوزر مبعثهمش، الدالة تاخذ القيم الافتراضية لوحدها.

التعديل:

C#

```

1  public static IEnumerable<T> Paginate<T>(this IEnumerable<T> source, int?
2  page, int? pageSize)
3  {
4      // لو مبعثش قيمة، اعتبرها 1
5      int currentPage = page.HasValue ? page.Value : 1;
6
7      // لو مبعثش قيمة، اعتبرها 10
8      int currentPageSize = pageSize.HasValue ? pageSize.Value : 10;
9
10     // بنكمل باقي اللوجيك عادي
11     if (currentPage <= 0) currentPage = 1;
12     if (currentPageSize <= 0) currentPageSize = 10;
13
14     return source.Skip((currentPage - 1) *
15     currentPageSize).Take(currentPageSize);
16 }

```

C#

```

1 // في رقم الصفحة، فاعتبرها الصفحة الأولى أوتوماتيك null هنا بعثنا
2 var result = employees.Paginate(null, 7);
3 result.Print("Page 1 (Default) with size 7");

```

السيناريو الثالث: فلتر مع تصفح (WhereWithPaginate)

لو عايز أعمل شرط (Filter) وبعدين أعمل تصفح للنتيجة. مثلاً: "هات لي الموظفين اللي عندهم تأمين صحي، واعررضهم في صفحات".

بدل ما أكتب Where وبعدها Paginate في الكود الرئيسي، ممكن أعمل Extension Method تدمجهم سوا.

اللوغيك:

1. نستقبل Func<T, bool> predicate (الشرط).
2. نطبق Where الأول.
3. نطبق Paginate على النتيجة.
4. أهم نقطة: نعيد استخدام دالة Paginate اللي لسه عاملينها فوق (Don't Repeat Yourself - DRY).

الكود:

C#

```

1 public static IEnumerable<T> WhereWithPaginate<T>(
2     this IEnumerable<T> source,
3     Func<T, bool> predicate,
4     int page,
5     int pageSize)
6 {
7     // التحقق من المدخلات ...
8     if (source == null) throw new ArgumentNullException(nameof(source));
9
10    // تطبيق الفلتر
11    var result = source.Where(predicate);
12
13    // تطبيق التصفح بإعادة استخدام الدالة السابقة
14    return result.Paginate(page, pageSize);
15 }

```

الاستخدام:

C#

```

1 // هات الصفحة الأولى (7 موظفين) بس بشرط يكون عندهم تأمين صحي
2 var coveredEmployees = employees.WhereWithPaginate(
3     x => x.HealthInsurance == true,
4     1,
5     7);
6
7 coveredEmployees.Print("Covered Employees - Page 1");

```

السيناريو الرابع: اختيار عنصر عشوائي (RandomElement)

آخر نقطة في الحلقة، إزاي نجيب موظف عشوائي من الليسته، ويمكن كمان نطبق شرط عليه قبل الاختيار.

الخطوات:

1. نحتاج Random object (يفضل يكون static عشان الأداء).
2. نستقبل predicate (اختياري) عشان لو عايزين نفلتر الأول.
3. لو الـ predicate مش null ، نفلتر الداتا الأول.
4. نختار رقم عشوائي (Index) في حدود عدد العناصر Count .
5. نستخدم ElementAt(index) عشان نجيب العنصر.

الكود:

C#

```

1 private static Random _random = new Random();
2
3 public static T RandomElement<T>(this IEnumerable<T> source, Func<T,
4     bool> predicate = null)
5 {
6     if (source == null) throw new ArgumentNullException(nameof(source));
7
8     // لو اليوزر باعت شرط، نفلتر الأول
9     // لو مش باعت، نشغل على السورس كله
10    var result = predicate == null ? source : source.Where(predicate);
11
12    // لو مفيش داتا بعد الفلتر نرجع القيمة الافتراضية
13    if (!result.Any()) return default(T);
14
15    // توليد رقم عشوائي من 0 لغاية عدد العناصر
16    int randomIndex = _random.Next(0, result.Count());
17
18    // إرجاع العنصر صاحب هذا الاندكس
19    return result.ElementAt(randomIndex);
20 }

```

C#

```
1 // 1. موظف عشوائي من كل الموظفين
2 var randomEmp = employees.RandomElement();
3 Console.WriteLine(randomEmp.FullName);
4
5 // 2. موظف عشوائي بشرط (معدوش خطة تقاعد)
6 var randomEmpNoPension = employees.RandomElement(x => !x.PensionPlan);
7 Console.WriteLine(randomEmpNoPension.FullName);
```

الخاتمة

- الموضوع شيق وممتع جداً، وبإخليك تقدر تبني المكتبة الخاصة ببيك وتسهل الشغل على نفسك وعلى النtim.
- مهم جداً تطبق بإيدك وتجرب سيناريوهات مختلفة.
- الحلقة الجاية هتكون الحلقة الأخيرة في سلسلة LINQ إن شاء الله.

LINQ Anatomy

مقدمة: الحلقة دي هي الحلقة رقم 20 والأخيرة من سلسلة LINQ التابعة لكورس Full Stack Developer. الموضوع النهاردة حساس جداً ومهم، هنتكلم عن "تشریح" اللينك (Anatomy)، إزاي بيشتغل من جوه؟ إيه الـ Best Practices؟ وإزاي تاخذ أحسن أداء وتتجنب المشاكل.

1. التجهيز للمثال (The Setup)

عشان نشرح المفاهيم دي عملياً، عملنا تطبيق .NET 5 بسيط بيمثل "أوراق الكوتشينة" (Cards).

نموذج الكارت (Card Model)

الكلاس ده بيمثل ورقة الكوتشينة الواحدة ويبحتوي على:

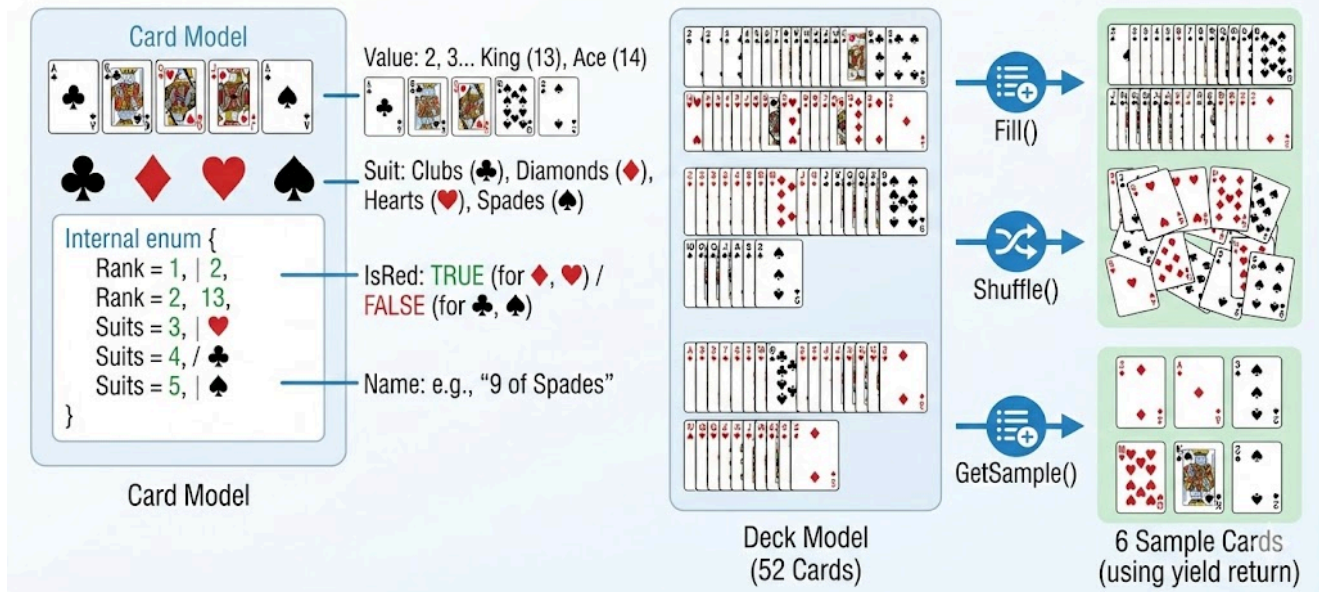
- الـ **Value**: قيمة الورقة (2, 3, ... Ace, King).
- الـ **Suit**: الشكل (Diamond, Spades, Hearts, Clubs).
- الـ **Internal Enum**: للأشكال والقيم (حيث الـ Ace بـ 14 والـ King بـ 13).
- خصائص (Properties)**:
 - الـ **Name**: بترجع اسم الكارت (مثلاً "9 of Spades").
 - الـ **IsRed**: بترجع `true` لو الكارت لونه أحمر (Hearts, Diamonds).

نموذج حزمة الورق (Deck Model)

بيمثل حزمة الكوتشينة (52 كارت):

- فيه ميثود بتعمل `Fill` للـ 52 كارت.
- ميثود `GetSample`: بترجع 6 كروت بس للتجربة باستخدام `yield return`.
- ميثود `Shuffle`: بترتب الورق بشكل عشوائي.

LINQ Anatomy: Card & Deck Logic



2. مفهوم الـ Fluent API

أول نقطة لازم نفهمها هي إنا لما بنكتب LINQ، إنا بنستخدم نمط اسمه **Fluent API**.

يعني إيه **Fluent API**؟

- هي طريقة لكتابة الكود بتخليك تقرأ الكود كأنه جملة إنجليزي عادية (Method Chaining).
- اللينك عبارة عن **Extension Methods** بتنادي بعضها.
- معظم المكتبات المحترمة بتستخدم المبدأ ده زي (EF Core, xUnit, ASP.NET Engine).

مثال بالكود:

C#

```
1 // بنقرأ الكود كأنه جملة: رتب، تجاهل 10، خذ 10، رتب بالشكل
2 var query = deck.Cards
3     .OrderBy(c => c.Value)
4     .Skip(10)
5     .Take(10)
6     .OrderBy(c => c.Suit);
```

ملحوظة: لو حاولت تكتب اللوجيك ده بـ **For Loop** و **While**، الكود هيبقى صعب جداً في القراءة والصيانة، لكن اللينك بيحقق **Readability** عالية جداً.

3. الفرق بين IEnumerable و IQueryable

النقطة دي بتعمل لغط كبير، ولازم تكون فاهمها كويس (مراجعة حلقة 19 ضرورية).

الـ (IEnumerable (LINQ to Objects

- ده بيشتغل على الـ **Memory (In-Memory Collection)**.
- **طريقة التنفيذ:** بينفذ الحاجه حرفياً زي ما أنت كاتبها بالترتيب.
- لو قلت له "رتب وبعدين اعمل صفحات"، هيرتب الأول وبعدين يقطع صفحات.
- لو قلت له "اعمل صفحات وبعدين رتب"، هيعمل صفحات وبعدين يرتب الناتج.
- هو عبد المأمور، بينفذ الخطوات بحذافيرها.

الـ (IQueryable (LINQ to SQL / Providers

- ده بيستخدم مع **Data Providers** زي Entity Framework.
- بيحول الكود بتاعك لـ **Expression Tree**.
- البروفايذر (**Provider**): بياخد الـ Expression Tree دي ويشوف أنسب طريقة لتنفيذها (مثلاً يحولها لـ SQL Query).
- **طريقة التنفيذ:** مش شرط يلتزم بالترتيب الحرفي اللي أنت كاتبه، هو بيعمل Optimization عشان يجيب لك النتيجة اللي أنت عايزها بأحسن أداء.

4. ترتيب التنفيذ (Execution Order)

دي من أهم النقاط وفيها مفاهيم خاطئة كتير.

قاعدة ذهبية: الـ Expressions في C# بتننفذ من الشمال لليمين (**Left to Right**).

محدث يقولك اللينك بيتنفذ من اليمين للشمال. التنفيذ بيبدأ من الميثود الأولى، تسلم للتانية، وهكذا.

مثال عملي لتتبع التنفيذ (Tracing)

تخيل عندنا مصفوفة أرقام، وعايزين نطبق عليها اللوجيك ده:

C#

```
1 var query = numbers
2     .Where(n => n > 5)           // 1. 5 من الأكبر
3     .Select(n => n * n)         // 2. اضرب الرقم في نفسه
4     .Where(n => n % 6 == 0)     // 3. هات اللي بيقل القسم على 6
5     .Take(2);                  // 4. خذ أول رقمين بس
```

سيناريو التنفيذ (خطوة بخطوة): الأرقام: [9, 12, 5, 6, 1, 4, 3, 2, 8]

1. الرقم 8:

- الـ `Where(>5)`: هل `8 < 5`? نعم. -> سلم للي بعده.
- الـ `Select(*)`: `8 * 8 = 64`. -> سلم للي بعده.

• الـ (Where(%6) : هل 64 تقبل القسمة على 6؟ لا. -> ارجع هات اللي بعده.

2. الأرقام 2, 3, 4, 1:

• الـ (Where(>5) : كلهم لا. -> تجاهلهم.

3. الرقم 6:

• الـ (Where(>5) : هل 6 < 5؟ نعم.

• الـ (Select(*) : 6 * 6 = 36 .

• الـ (Where(%6) : هل 36 تقبل القسمة على 6؟ نعم.

• الـ (Take(2) : خدت واحد (باقي لي واحد).

4. الرقم 5:

• الـ (Where(>5) : لا (لأن 5 مش أكبر من 5).

5. الرقم 12:

• الـ (Where(>5) : نعم.

• الـ (Select(*) : 12 * 12 = 144 .

• الـ (Where(%6) : هل 144 تقبل القسمة على 6؟ نعم.

• الـ (Take(2) : خدت الثاني. **STOP!** (عملنا Break).

ملاحظة: الرقم 9 ماتعملش عليه أي عملية، لأن Take اكتفت. التنفيذ كان On Demand (عند الطلب).

5. التنفيذ الفوري vs التنفيذ المؤجل

الـ (Immediate Execution vs Deferred Execution)

أولاً: التنفيذ الفوري (Immediate Execution)

بيحصل لما تنادي ميثودز زي .Sum , .Count() , .ToList() .

• إيه اللي بيحصل؟ الكويري بتننفذ فوراً، وكل البيانات بتتحمل في الميموري في اللحظة دي.

• المميزات:

• لو الداتا اتغيرت في المصدر، اللي معاك في الليست مش هيتأثر (معاك لقطة Snapshot).

• لو هتستخدم الداتا كذا مرة، مش هتروح تكلم الداتا بيز كل مرة، هي مرة واحدة.

• العيوب:

• بتاخذ مساحة من الميموري (ممکن تكون كبيرة جداً).

• لو الداتا اتحدت في الداتا بيز، اللي معاك هيبقى قديم (Not up-to-date).

ثانياً: التنفيذ المؤجل (Deferred Execution)

بيحصل لما تعرف الكويري وترجع IEnumerable أو IQueryable من غير ما تعمل .ToList .

• إيه اللي بيحصل؟ الكويري مش بتننفذ وقت التعريف. بتننفذ بس لما تعمل عليها Loop (تلف عليها بـ foreach).

• المميزات:

• الـ Up-to-date: كل مرة بتناديها بتجلبك أحدث داتا.

- الـ **Low Memory**: مابتأخذ مساحة كبيرة لأنها بتشتغل بنظام العنصر بعنصر (مش بتخزن كله).
- العيوب:
- لو ناديتها 10 مرات، هتنتفذ 10 مرات (ممكن يكون Cost عالي لو الكويري ثقيلة).

6. أخطر جزئية: Streaming vs Non-Streaming

جوه الـ Deferred Execution نفسه، فيه نوعين من العمليات لازم تفرق بينهم عشان الأداء.

1. الـ Streaming (زي Where , Select)

- دي بتشتغل بنظام "الأنبوب" (Pipeline).
- مش محتاجة تشوف كل الداتا عشان تشتغل. بتمسك عنصر، تعالجه، وتسلمه للي بعده فوراً.
- مثال: Where بتمسك رقم، لو نجح تديه لـ Select ، وهكذا.. مش بتستنى تخلص كل الأرقام.

2. الـ Non-Streaming (Buffering) (زي GroupBy , OrderBy)

- دي عمليات ماينفعش تشتغل عنصر بعنصر.
- عشان أرتب (Sort)، لازم يكون معايا كل العناصر الأول.
- النتيجة: العملية دي بتعمل Buffering (تخزين مؤقت) لكل الداتا اللي جاية من المرحلة اللي قبلها قبل ما تطلع أي نتيجة للي بعدها.

كارثة في الأداء: لو حطيت OrderBy في نص الكويري، هي هتضطر تسحب كل الداتا من المراحل اللي قبلها، تخزنها في الميموري، ترتبها، وبعدين تبدأ تطلع نتائج.

```
// Deferred (Non Streaming): i.e. Sorting, grouping must read the whole sequence
// Does not mean "this is a sequence that is ordered"
// This is a sequence that has had an ordering operation applied
// ThenBy to impose additional ordering

var numbers = new int[] { 8, 2, 3, 4, 1, 6, 5, 12, 9 };
var query = numbers
    .Where(x => x > 5)
    .OrderBy(x => x) // Buffering: // 6, 7, 8, 9
    .Select(x => x);

Console.WriteLine($"Where({x} > 5) => {x > 5}");
return x > 5;

// (extension) IOrderedEnumerable<T> IOrderedEnumerable<T>.OrderBy<T, TKey>(Func<T, TKey>, int) (+ 1 overload)
// Returns: An IOrderedEnumerable<T> whose elements are sorted according to a key.
// Exceptions: ArgumentNullException
```

Deferred مع الـ Deferred

يس لازم تلف على كل العناصر لأن الأوردر باي أجبرتها على كده

Where(8 > 5) => True
Where(2 > 5) => False
Where(3 > 5) => False
Where(4 > 5) => False
Where(1 > 5) => False
Where(6 > 5) => True
Where(5 > 5) => False
Where(12 > 5) => True
Where(9 > 5) => True

Select(6 X 6) => 36
Take: 36
Select(8 X 8) => 64
Take: 64

7. أهمية ترتيب العمليات (Position Matters)

مكان Skip و Take

- تغيير مكان Take بغير معنى الكويري تماماً ونتيجتها.
- حاول تحط Take في أقرب مكان ممكن عشان توقف سحب الداتا بدري (Efficiency)، بس بشرط ما تغيرش البيزنس لوجيك.

الفلتر قبل الترتيب (Filter BEFORE Sort)

سؤال: "هات لي أكبر 10 كروت لونهم أحمر".

1. صح: هات الكروت الحمراء -> رتبهم -> خذ 10.
 - هنا رتبت بس الكروت الحمراء (وفرت وقت ومجهود).
 2. غلط: رتب كل الكروت -> هات الأحمر -> خذ 10.
 - هنا رتبت الأحمر والأسود، وده وقت ضايع في ترتيب حاجات هترميها أصلاً.
- لو السؤال "هات لي الكروت الحمراء اللي موجودة في أعلى 10 كروت في الحزمة كلها".

1. المعنى الأول: "أول 10 كروت في إيدك" (Physical Top)

لو قصدك بأعلى 10 يعني "أول 10 ورقات محطوطين على وش الكوتشينة" (بغض النظر هما كبار ولا صغيرين).

- هنا مش محتاج ترتب كله.
- الخطوات: خذ أول 10 (Take 10) ← وبعدين نقي منهم الأحمر (Where).

2. المعنى الثاني (وهو ده المقصود): "أقوى 10 كروت في اللعبة" (Logical Top / Highest Value)

لو سؤالي ليك: "مين الكروت الحمراء اللي نجحت تكون ضمن قائمة أقوى 10 كروت في الكوتشينة كلها؟"

هنا بقى لازم "ترتب كله" الأول، ليه؟ عشان أعرف مين هما الـ "Top 10" (الأقوياء زي الـ Ace والـ King)، لازم أفرش الكوتشينة كلها (الـ 52 كارت) وأرتبهم من الكبير للصغير.

الترتيب هنا إجباري يكون في الأول:

1. الـ OrderByDescending (رتب كله): عشان تطلع الكبار فوق (سواء أحمر أو أسود).
2. الـ Take(10): اقطف بقى الـ 10 الأوائل (صفوة المجتمع).
3. الـ Where(IsRed): شوف لي بقى مين في الصفوة دول لونه أحمر.

8. المثال الشامل (Run Query Pipeline) - تشريح بصري

تعالوا نشوف كويري كاملة ونشوف "الأناتومي" بتاعها بيمشي إزاي كأننا جوه الكمبيوتر.

الكود:

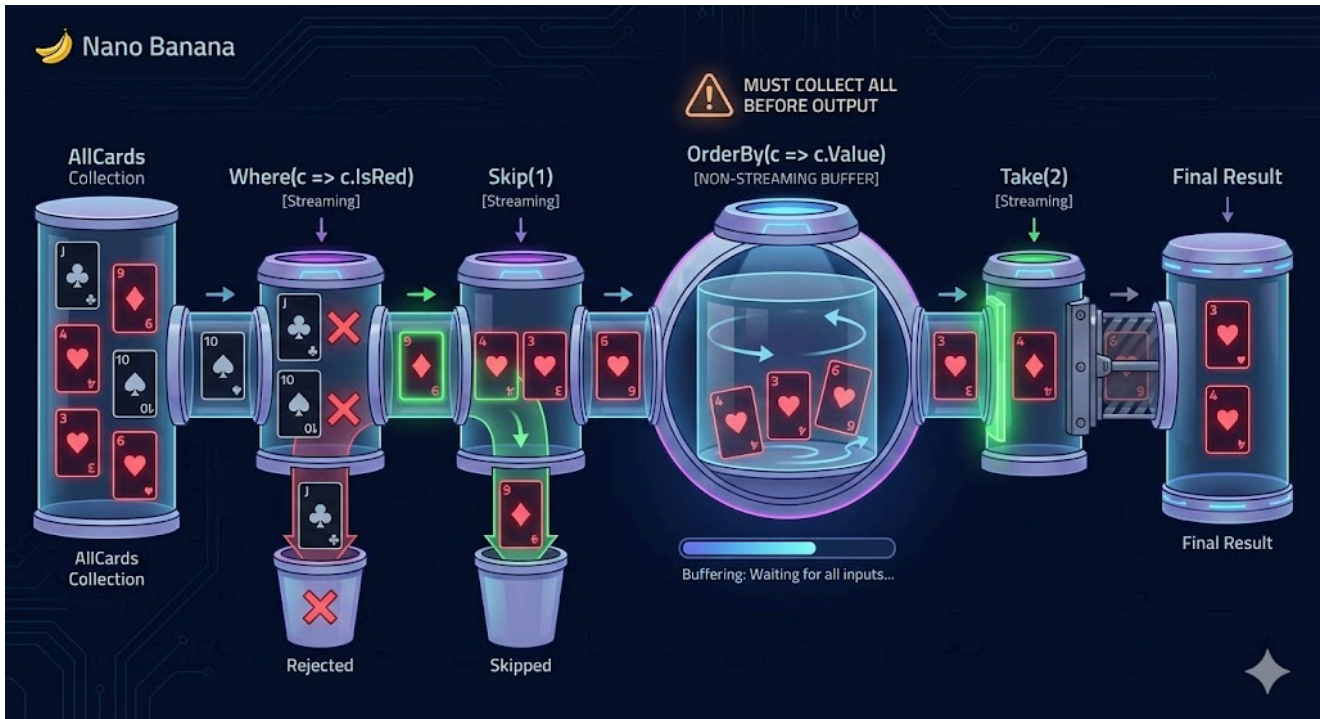
C#

```
1 var query = cards
2   .Where(c => c.IsRed) // فلتر الأحمر
```

```

3      .Skip(1)                // فوت واحد
4      .OrderBy(c => c.Value)   // رتب
5      .Take(2)                // خذ 2
6      .ToList();              // نفذ

```



تخيلها كـ Pipeline (مواسير بتسلم بعض): تخيل كل ميثود عبارة عن "شخص" واقف بيطلب من اللي قبله.

1. الـ AllCards (المصدر): معاه 6 كروت (♥Jack♣, 9♦, 4♥, 10♠, 3♥, 6).

2. الـ Where: بتطلب من المصدر.

الـ - Jack♣ (أسود) <- ❌ ارميه.

• 9♦ (أحمر) <- ✅ عداها.

3. الـ Skip: وصلتلها الـ 9♦.

• أول كارت واصل؟ <- ❌ تجاهله (عشان Skip 1).

4. الاستمرار:

• 4♥ (أحمر) <- عدى من Where <- عدى من Skip (لأن خلاص فوتنا الأول).

• الـ 4♥ وصلت لـ OrderBy.

5. المطب (OrderBy):

• الـ OrderBy هتقول: "استنى عندك! أنا ماقدرش أطلع نتايج دلوقتي. أنا Non-Streaming. لازم تجيب لي كل اللي

عندك يا Where ويا Skip عشان أرتبهم".

• فتضطر تسحب باقي الكروت:

• 10♠ (أسود) <- ماتت في Where.

• 3♥ (أحمر) <- وصلت لـ OrderBy.

• 6♥ (أحمر) <- وصلت لـ OrderBy.

• دلوقتي OrderBy عندها في الـ Buffer: (4♥, 3♥, 6♥).

• ترتبهم: (3♥, 4♥, 6♥).

6. الـ Take(2):

- دلوقتي OrderBy هتبدأ تضخ لـ Take.

- الـ Take تاخذ الـ 3♥.

الـ - Take تاخذ الـ 4♥.

- تكنتفي وتعمل Break.

سؤال عبقرى: ليه سحبنا الـ 6♥ مع إننا عايزين 2 بس، وكان ممكن نكتفي بالـ 3 والـ 4؟ **الإجابة:** بسبب OrderBy .
وجودها أجبر اللينك إنه يسحب كل الداتا اللي بتحقق الشرط (بما فيها الـ 6) عشان يرتب صح، حتى لو الـ Take مش
هتاخذها في الآخر. دي ضريبة الـ Non-Streaming

Material For Practice:

https://github.com/ahmed15reda/Plus-300-Linq-Questions-Northwind?fbclid=IwY2xjawOyKG9leHRuA2FibQIxMABicmlkETFaanlCYUJnSmVIYW1RMmUzc3J0YwZhcHBfaWQQMjlyMDM5MTc4ODlwMDg5MgABHpHedfis1WBV-IruczZ0wNUKM3Jt_ZoY4gU-Wq_QOf6ZaxA6XvQ2V6KEiDGm_aem_-rHR4CU2Gp7vq83RSVJ6dw